
北京科技大学
计算机组成原理课程设计 19-20 秋季
(V1.00)

2019 年 10 月

目录

目录.....	2
一、讲义.....	1
1 讲义一：课程介绍.....	1
1.1 本门课程学习意义.....	1
1.2 本课程与计算机专业其它课程的相互关系.....	2
1.3 课程实验安排.....	5
2 讲义二：Verilog 语言复习.....	6
2.1 怎么学习 Verilog 语言.....	6
2.2 面向硬件电路的设计思路.....	7
2.3 浅谈代码风格.....	8
2.3.1 变量命名.....	8
2.3.2 代码对齐.....	9
2.4 可综合代码.....	10
2.5 推荐的可综合代码风格.....	12
2.6 Verilog 示例.....	12
2.6.1 模块声明和实例化.....	12
2.6.2 基础逻辑门.....	13
2.6.3 译码器.....	13
2.6.4 编码器.....	14
2.6.5 多路选择器.....	14
2.6.6 简单 MIPS CPU 的 ALU.....	15
2.6.7 寄存器.....	17
2.6.8 MIPS CPU 中的寄存器堆.....	18
2.6.9 流水线.....	19
2.7 小结.....	22
3 讲义三：CPU 实验开发环境使用说明.....	23
3.1 CPU 实验开发环境快速上手.....	23
3.2 CPU 实验的一般性开发过程.....	24
3.3 CPU 实验开发环境.....	25
3.3.1 CPU 实验开发环境组织与结构.....	25
3.3.2 TinyMIPS 处理器的目录结构.....	27
3.3.3 SoC_Lite.....	28
3.3.4 功能仿真验证.....	29
3.3.5 func 程序说明.....	32
3.3.6 编译脚本说明.....	35
3.3.7 编译结果说明.....	35
4 讲义四：仿真调试说明.....	37
4.1 调试指导思想概述.....	37
4.2 仿真出错分类.....	38
4.3 出错分类一：波形出错.....	38
4.3.1 信号为“Z”.....	38
4.3.2 信号为“X”.....	39
4.3.3 波形停止.....	40
4.3.4 越沿采样.....	41
4.3.5 波形怪异.....	42

4.4 出错分类二：逻辑出错	42
4.4.1 CPU 逻辑出错调试	42
4.5 总结	44
二、任务书	45
实验：静态 5 级流水 MIPS CPU 实现	45
4.6 实验目的	45
4.7 实验设备	45
4.8 实验任务	45
4.9 实验环境	46
4.10 实验说明	47
4.10.1 CPU 实现指令集	47
4.10.2 CPU 顶层接口	47
4.10.3 CPU 复位 PC	47
4.10.4 CPU 虚实转换	47
4.10.5 CPU 延迟槽实现要求	48
4.10.6 CPU 操作模式	48
4.10.7 CPU 不要求实现项	48
4.10.8 CPU 验证步骤	48
4.10.9 CPU 验证结果	48
三、附录	50
5 附录一：龙芯体系结构教学实验箱（Artix-7）介绍	50
5.1 开发板硬件设计方案	51
5.2 开发板原理图	52
6 附录二：龙芯体系结构教学实验箱（Artix-7）-原理图与引脚列表	54
7 附录三：Vivado 使用说明	55
7.1 数字电路设计的一般流程	55
7.2 Vivado 新建工程	56
7.3 RTL 设计输入	59
7.4 功能仿真	63
7.5 添加约束文件	69
7.5.1 利用 I/O Planning 生成约束文件	69
7.5.2 新建 XDC 文件并手动输入约束	72
7.6 工程实现	75
8 附录四：“计算机组成原理课程设计”MIPS 指令系统规范	81
9 附录五：实验报告模板	82
10 附录六：交叉编译工具链安装	83
10.1 交叉编译环境搭建	83
10.1.1 Windows 下安装	83
10.1.2 Linux 下安装	83
11 附录七：乘除法器 IP 定制	84
11.1 乘法器 IP 定制	84
11.2 除法器 IP 定制	85
12 附录八：SoC_up 使用环境说明	87
12.1 基于 GS232 搭建的 SoC_up 说明	87
12.1.1 GS232 开源版本简介	87
12.1.2 SoC_up 结构	87
12.1.3 地址空间分配	87

12.1.4 NAND DMA 控制器	88
12.1.5 NAND FLASH 控制器	88
12.1.6 CONFREG 模块	88
12.1.7 MAC 控制器	88
12.1.8 DDR3 控制器	88
12.1.1 SPI FLASH 控制器	88
12.1.2 UART 控制器	89
12.1.3 中断连接	89
12.2 串口软件	90
12.2.1 Linux 下	90
12.2.2 Windows 下	90
12.3 Tftp 服务器搭建	93
12.4 Flash 烧写	95
12.4.1 使用 EZP2010 编程器:	95
12.4.2 使用 FPGA 编程 bit 流	95
12.5 FPGA 板上 SoC_up 下载	97
12.5.1 Artix-7 教学实验板固化方法	97
12.5.2 生成 mcs、prm 文件	97
12.5.3 下载 mcs、prm 文件	98
12.6 加载内存到 NandFlash	102
13 附录九: FPGA 在线调试说明	104
13.1 抓取需探测的信号	104
13.2 综合并建立 debug	104
13.3 实现并生产 bit 流文件	107
13.4 下载 bit 流和 debug 文件	108
13.5 上板观察	108
13.6 注意事项	111

一、讲义

1 讲义一：课程介绍

亲爱的同学们，你们现在阅读的是“计算机组成原理课程设计”的课程讲义。是的，我将以这种口语化的风格来写这门课的课程讲义。这样会觉得很亲切，感觉像是在课堂上听老师当面授课一般。所以，我们在本课程的讲义编写中采用这种行文风格，若有不当，欢迎批评指正。

1.1 本门课程学习意义

我们这门课名字叫做“计算机组成原理课程设计”，动手实验确实是我们这门课的主要组成。为什么？因为计算机科学与技术这一学科本身就是工程性很强的学科，光靠课堂讲授、课后习题和笔试考试是难以全面深入掌握相关知识的。我校在数字逻辑、组成原理课程后设置了这样一门课程，并给予了充分的学时，就是基于这个基本观点的。你们刚从大二开始接触一些计算系统结构方面的专业课，这种体会可能还不强烈。很多上到体系结构方向的博士都说在本科阶段很多东西都是知其然不知其所以然，虽然作业会做考试能过，但其实心里不怎么明白。在今年的计组课设中我们打算通过一定工程规模的课程实验，同样让大家通过实践加深对于理论知识的理解。

我们为什么要学好计算机组成原理、系统结构这些课程呢？有的同学可能会说，我将来想搞人工智能、想搞网络，我又不想去做计算机，我干嘛要学好这些课呢？对于这种观点，我来请教同学们两个问题。首先，计算机领域最大的价值还是在于能够极大地提高人类的生产力，也就是说，要用起来，而且要越来越好用。这几年，人工智能火到不行，但是每个产品背后的算法都是多年之前的研究成果，为什么现在才这么火？其次，这一轮人工智能火热的潮流中，是哪些大公司在主导。是谷歌、英伟达、赛灵思、英特尔。为什么？

随着半导体工艺日益接近其物理极限，传统通用处理器设计提升性能或者面积效率的方式也将很快碰到其天花板，但是应用需求与现有硬件性能仍存在巨大的差异，于是专用化硬件设计作为一种新的性能优化手段策略得到越来越多的重视。从某种意义上讲，专用化硬件模糊了原有的软件和硬件的界面。像我们现在所熟知的基于通用处理器的计算机系统，其软硬件界面就是指令集（ISA），向上向下又有各个不同的抽象层次，各个层次的界面很清楚。界面清楚的好处是什么？就是大家可以各干各的，拼起来还能跑。但是现在不一样了，软硬件之间的界面被专用化硬件方法模糊了，同样是基于卷积神经网络的深度学习算法的应用实现，各家的硬件做法都不一样，所以各家的软件自然也就不一样了。以前上层应用软件人员主要考虑满足客户需求把功能做对，余下的性能问题都交给底层库、操作系统、编译系统和硬件去了，也就是说交给“造计算机”的人去管了。但现在情况变了，一个有别于普通“码农”的应用软件开发人员，他/她可能面对一个需求，首先要规划哪些交给硬件做、哪些交给软件做，并基于此开展后续的软件开发和调试。所以，即便是应用软件开发人员，都要具备系统观，更不用说应用层之下各层次的研发人员了。怎么具备系统观呢？别着急，学好我们在本科阶段开设数字电路、组成原理、体系

结构、操作系统和编译原理这一系列课程，你自然就有系统观了。

啰嗦了这么多。动机很简单。希望同学们不要一上来就从心底抵触这门课。只要你愿意来上课，我还是有信心不让你失望的。

1.2 本课程与计算机专业其它课程的相互关系

同学们可能已经发现，我总是喜欢把“数字逻辑、组成原理、体系结构、操作系统和编译原理”这五门课放在一起说。这是因为这些专业课程的知识都是围绕“如何造计算机”来组织的，相互之间彼此关联呼应，构成一个有机的整体。计算机科学与技术专业涉及的各门课程之间的关系大致如图 1-1 的示意，可以看到课程之间存在的知识概念的相互关联，表 1-1 列出这些相互关联的重要知识概念。

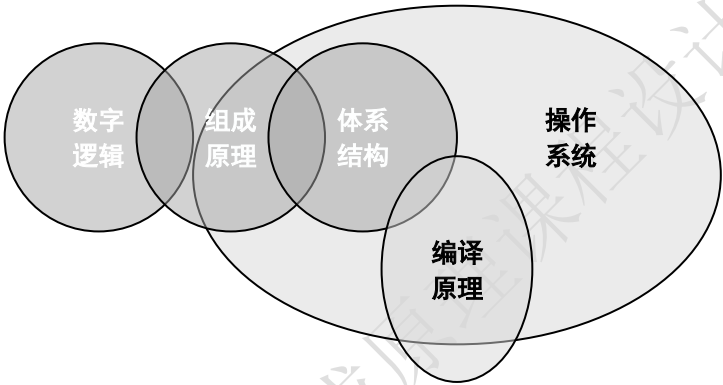


图 1-1 计算机科学与技术专业主要课程相互关系示意

表 1-1 计算机科学与技术专业主要课程重要概念关联

	数字逻辑	组成原理	体系结构	操作系统	编译原理
数字逻辑		数值、数制及编码，算术运算函数（ALU），寄存器传输与控制（ Regs ， RTL ），存储器，状态机设计	数值、数制及编码，算术运算函数（ALU），寄存器及传输与控制（ Regs ， RTL ），存储器，状态机设计	计数器	
组成原理	Datapath （ ALU/ 乘法器 ），控制器（寄存器传输状态机设计方法），Memory		指令集，Datapath，控制器，算术运算函数（ALU/乘法器），单/多周期/流水 CPU	指令集，单/多周期/流水 CPU	指令集
体系结构				指令集，单/多周期/流水 CPU，中断、DMA，Cache，Memory，虚拟存储，总线	指令集
操作系统		内存管理，应用程序加载，虚拟存储器	内存管理，应用程序加载，虚拟存储器		内存管理，应用程序加载，进程调试
编译原理		程序链接	系统优化	程序加载运行	

上述课程中与本课程实验关系最紧密的是三门课程：“数字逻辑”、“计算机组成”（本文有的地方也称之为“组成原理”）和“体系结构”，需要同学们较为系统的学习掌握。此外希望同学们有一定的“C 语言”和“汇编语言”的程序设计基础，至少要达到看到代码有点头绪的程度，当然会写会调就更好了。实验有部分内容与“操作系统”和“编译原理”两门课有关，但涉及的知识点不多且以用为主，所以不需要同学们提前系统学习这两门课，相关知识点我们会在教材中必要的地方予以简单讲解。

围绕本实验课程，“数字逻辑”、“计算机组成”和“体系结构”三门理论课的大体呈现一种递进关系：“数字逻辑”课教会大家设计逻辑电路的基本知识和技能，能设计加法器、移位器这类简单部件；“计算机组成”介绍 MIPS 指令集和简单 CPU 设计；“体系结构”从支撑系统运行和性能分析优化两个角度出发介绍 CPU 的扩展和优化。“计算机组成原理课程设计”在计算机组成和体系结构之间起到承上启下的作用。下面我们把与实验紧密相关的主要知识点都一一罗列出来，算是给同学们提供一个索引。作为实验课，每个知识点的具体内容我就不在讲义中再展开来讲了，同学们应该都可以在自己的理论课教材中查阅到相关内容。不过请同学们注意的是，此处列出的知识点仅是从本课程实验所需来考虑的，没有列出来的并不是不重要。譬如我们实验中没有实现浮点运算部件，但是浮点数相关的知识点还是希望大家掌握的。

“数字逻辑”方面

- 1) 数值表示和数制，重点是二进制数、十六进制数。
- 2) 数值的原码表示和补码表示，有符号数、无符号数，溢出。
- 3) 基本逻辑门：与、或、非。
- 4) 布尔代数：逻辑表达式，真值表，逻辑运算的常用运算律。
- 5) 组合逻辑电路和时序逻辑电路的原理。
- 6) n - 2^n 变量译码器，如“3-8 译码器”等。
- 7) 不同位宽的数据选择器，如“1 位宽 4 选 1”、“8 位宽 32 选 1”等。
- 8) 一位全加器，串行进位多位全加器。
- 9) 锁存器与触发器。
- 10) 触发器时序分析，触发器的 Setup 延迟、Hold 延迟和 Clock-to-Q 延迟的含义及原理
- 11) 典型状态机设计。
- 12) 计数器设计，如二进制计数器、模- n 计数器。
- 13) 移位寄存器设计。
- 14) 只读存储器 ROM 基本原理。
- 15) 随机存储器 RAM 基本原理。

16) 动态存储器 DRAM 基本原理。

17) 现场可编程门阵列 FPGA 基本原理。

“计算机组成”方面

1) “冯·诺依曼”体系结构。

2) RISC 指令集的基本特点。

3) 以 MIPS 指令集为例的指令格式：操作码、操作数、寻址。

4) 并行进位多位全加器。

5) 桶形移位器。

6) 补码乘法器。

7) 补码除法器。

8) 基于多路选择器的数据通路。

9) 算术逻辑单元 ALU。

10) 寄存器堆，两读一写的 32×32 寄存器堆。

11) 指令的取指数据通路、读操作数数据通路、执行数据通路、写回数据通路。

12) 指令译码与 CPU 控制逻辑。

13) 取指、译码、执行、访存、写回五周期的多周期 CPU 数据通路与控制逻辑设计。

14) 取指、译码、执行、访存、写回五级流水 CPU 数据通路与控制逻辑设计。

“体系结构”方面

1) 异常：异常事件与异常事件处理。

2) 中断：中断识别、中断优先级、中断响应和中断返回。

3) 基于页的虚拟存储管理原理：页、页缺失、页分配、页地址映射。

4) TLB：TLB 结构、TLB 缺失维护。

5) 存储器层次结构：多级 Cache、Cache 基本结构、映射机制、查找过程、替换算法、写策略。

6) 总线：基本概念、总线结构、总线的仲裁和通信控制。

7) I/O：I/O 寻址方式（独立编址、统一编址），I/O 访问模式（轮询、中断），I/O 数据传输方式（PIO、DMA）。

8) 计算机系统性能分析：性能评价指标、常见性能分析建模方法

1.3 课程实验安排

在龙芯实验箱上可完成下列所有实验。但是计组课设只关注实验 0。按要求完成现场验收。

实验 0 静态 5 级流水简单 MIPS CPU 实现

实验任务：

1. 完成 TinyMIPS 处理器的仿真测试和 FPGA 测试。
2. 结合给出的 TinyMIPS 工程完成 CG 平台的模块评测。

实验目的：

1. 理解流水线数据通路和控制逻辑的基本设计原理。
2. 理解 CPU 流水线设计中将指令信息逐级传递这一设计关键。
3. 理解流水线 CPU 中各类相关的产生机理和解决方式。
4. 理解流水线 CPU 中各类相关的产生机理和解决方式。

时间安排：

第 10、11、12 周

实验 3 CPU 增加例外和中断的支持

实验任务：

1. 在实验 2 所实现 CPU 基础之上实现对于例外和中断的支持。
2. 例外包括保留指令、溢出、断点、系统调用、地址错例外。
3. 中断包括软件中断、硬件中断（含时钟中断）。

实验目的：

1. 掌握流水线 CPU 中例外处理的设计思路。
2. 熟悉 MIPS 架构下例外处理的软硬件交互界面和交互流程。

实验 4 CPU 增加总线接口

实验任务：

1. 在实验 3 所实现 CPU 基础之上实现 AHB-Lite 总线接口。

实验目的：

1. 掌握片上总线的一般性原理。
2. 掌握总线接口与 CPU 内部流水线之间的相互关系。

实验 5 CPU 增加 TLB MMU

实验任务：

1. 将实验 4 所实现 CPU 的存储管理机制由直接地址映射改为 TLB 映射方式。
2. 软件可见 TLB 共 16 项，支持 4KB 页大小。
3. 实现指令 TLB Cache 有额外奖励。

实验目的：

1. 深入理解基于 TLB 的存储管理机制，掌握其中软硬件交互的界面和交互流程。

实验 6 用 CPU 搭建 SoC 跑小型操作系统

实验任务：

1. 完善自己设计的 CPU，用其替换 SoC 系统中的龙芯 CPU，在 FPGA 上正常启动操作系统内核。
2. 在采用自己 CPU 核的 SoC 上运行 Dhrystone、Coremark 等性能测试，各组进行性能比赛。

实验目的：

1. 在真实系统环境下理解 CPU 各设计要素的互动关系。
2. 初步培养性能评估、优化的能力。

2 讲义二：Verilog 语言复习

本课程要求所有设计采用 Verilog 语言进行描述。我们这样选择是因为 Verilog 是一种常见的硬件描述语言，当前很多厂家的设计都采用 Verilog 语言开发，初学者学习 Verilog 也不会比学习 VHDL 花费更多时间。有的同学可能会问为什么不采用 SystemVerilog 语言。在我们看来，Verilog 语言和 SystemVerilog 语言之间的关系有点类似于 C 语言和 C++ 语言，两种语言都有其应用的地方，并没有谁一定替代谁的趋势，并且学习 Verilog 语言之后能帮助学习 SystemVerilog 语言，所以我们还是选择了 Verilog 语言。我们也没有采用脚本化语言的设计风格，脚本化语言的设计风格的优势在于提高设计效率，但是考虑到课程设计规模并不复杂，同时我们希望同学们能够对于设计的电路实现有更清晰的认识和掌控，所以我们还是坚持采用 Verilog 语言，且在 RTL 级进行设计建模。

目前已经有很多优秀的关于 Verilog 语言方面的书籍和文献，理论上讲同学们学会使用 Verilog 语言应该没有任何问题，但是从我们已有的一些教学经验来看，发现事实并非如此。所以我们在这里对一些典型问题进行简单讲解，希望能帮助同学们在使用 Verilog 语言的时候少走一些弯路。

2.1 怎么学习 Verilog 语言

我们把 Verilog 语言的学习要求分为两档，低一档的叫“学会”，能满足本课程实验任务，高一档的叫“学好”，能够熟练掌握 Verilog 语言进行工程开发。要想学好 Verilog 语言，重点已经不在语言本身，更多的涉及电路设计、系统思维、编程风格、项目管理，需要在实践中不断总结提高。将来有志于从事 IC 前端设计开发的同学们，可以把“学好 Verilog 语言”作为一个长期任务执行下去，这里就不展开进行讲述。接下来我们主要告诉同学们如何“学会” Verilog 完成实验。

本课程实验中有两个地方用到 Verilog 语言，一个地方是要求同学们用 Verilog 语言进行设计，另一个地方是我们用 Verilog 语言给大家写的测试平台（testbench）。前一个地方要求大家会看会写，后一个地方只要求大家看得懂。testbench 的 Verilog 代码中我们已经添加了相应的注释，所以各段代码干什么的同学们应该是知道的，如果有什么关键字、系统调用的不知道具体什么含义，就请自行查阅 Verilog 参考资料。那么同学们真正需要花力气学习的是用设计部分所需的 Verilog 语言。

我们建议同学们采用 RTL（Register Transfer Level，寄存器传输级）设计，且要求设计可以被 EDA 工具综合成最终的电路，那么大家实际开发过程中只会用到 Verilog 语言的一个子集（即所谓的可综合子集）。同时由于 Verilog 语言自身用法比较自由，导致容易出现一些易混淆和易出错的地方，出现竞争条件、前后仿真不一致。为了那么为了避免同学们在实验过程中在这些语言方面花费过多的调试时间，我们又会对可综合子集的使用方式做出限制，从形式上降低代码出错的风险。在上述这一系列限制之下，同学们实际需要学习的 Verilog 语言要素已经

非常非常少。为了帮助同学们迅速掌握这些必须的 Verilog 语言要素，我们会在后面给出一系列具体的代码示例，同学们只要把这些例子中涉及的语言要素都掌握了，则至少完成本课程的设计就不是问题了。事实上，工程师在进行龙芯 CPU 的前端设计时，也就只用到 Verilog 的这些语言要素。

总之，语言只是个工具，是手段不是目的。即使有一把绝世名剑，也并不意味着你就是个好剑客。反之，真正的高手，草木竹石皆可为剑。我们花费了很多心思就是希望同学们不要花费过多的时间去学习 Verilog 语言本身，掌握最有用的一小部分就可以开始实践了，其余部分视你的学习、工作需要，可以在日后继续学习。我们要把精力更多的花在这些提到能力的方面，所以下面我们先讨论设计思路和代码风格，然后再介绍可综合 Verilog 代码并进行代码示例讲解。

2.2 面向硬件电路的设计思路

Verilog 语言的很多语法要素与 C 语言很像，而且 Verilog 语言也支持行为级建模。这就使得很多同学习惯于用 C 程序开发的思路去使用 Verilog 语言，即总是有个过程的概念在脑海里挥之不去。教学中常听到一些同学这样对我描述它的代码——“先调用 X 模块，然后如果条件 C1 成立，就调用 Y 模块，如果条件 C2 成立，就调用 Z 模块……”——是不是还是 C 程序编程那个调调。这种串行的过程化的思维对于 Verilog 语言设计来说是有很大弊端的。我们是用 Verilog 语言来设计硬件电路的，而硬件电路不是串行化过程化的，它是并行并发的。所以 Verilog 语言里面，一前一后两句 assign 语句，不是写在前面的先执行写在后面的后执行，在忽略底层电路延迟因素的情况下，这两句对应的逻辑是同时并发执行的。在使用 Verilog 语言设计电路时，无论你是采用门级建模、数据流建模还是行为级建模，都是对电路建模，不是对算法建模。

怎么让同学们从 C 的那套思维方式转到面向硬件电路的设计思路呢？实践中我们总结出了一条经验，要求学生必须采用这样的设计步骤：**先进行电路结构设计，再进行 Verilog 代码编写**。这样就从根本上避免了学生们被 Verilog 语言中的行为级建模方法带偏了方向。当你在考虑电路的结构及其逻辑的时候，自然会产生并行并发的意识，也会采用起“数据通路+状态机”的设计思路。当电路设计已经被清晰地分解为结构图中的各个模块和模块之间的连接、模块内部的数据通路和状态机、数据通路中的电路逻辑以及状态机中的状态转换图，那么接下来的 Verilog 代码设计就只是一个简单的“翻译”而已。

采用上述设计步骤设计一个较为复杂的电路系统时，同学们可能会觉得电路结构设计的工作量很大，不知道从何处下手。我们建议同学们采取“**自顶向下、模块划分、逐层细化**”的设计步骤。划分过程中，我们推荐**优先沿着数据流的路径进行划分**。一个实用的电路系统，肯定是有输入也有输出的，自然也就存在从输入到输出的数据流路径，沿着这个路径进行切分总是不会有大偏差的。具体如何划分要根据实际情况，总的来说，**功能上相对集中的逻辑应该放在一个模块内，高层次模块与模块之间的数据交互尽可能呈现单方向流动，最底层模块的规模控制在千行代码之内**。

上述电路结构设计过程通常不可能只考虑一遍就能获得很好的设计方案，需要反复迭代，多次调整。所谓

“谋定而后动”，我们强烈建议同学们一定要先把电路结构设计想妥当了，再开始动手写 Verilog 代码。有的同学喜欢“写一点，试一下，改一点，再试一下”这种所谓的增量代码开发方法，尽管听上去很好，但从我们自身的实践来看，这种设计方式不太适合电路系统的设计，至少不适合 CPU 的设计。这主要是因为 CPU 中各部分之间的相互关系十分紧密，常出现牵一发而动全身的情况，所以往往一改就要前前后后改很多地方。改的地方越多，出错的概率也就越大，导致改的地方更多，这样代码开发过程就很容易呈现发散的状态，不容易在可控的时间内收敛到一个稳定状态。本实验课程需要大家设计的 CPU 相对简单，大概不到两千行 Verilog 代码就能实现，测试程序集也很少很不充分，同学们用试错的代码开发方法在规定时间内说不定也能搞定，但我们还是希望同学们养成“谋定而后动”的设计习惯，把迭代放到设计方案的制定阶段而不是放到代码编写阶段。记得有一个学生曾和我聊他做实验的过程，总是要“纠结”好几天甚至一两周，等到都想顺了，不“纠结”了，真正动手写代码一天不到就写完了，调起来也快，主要是一些笔误引入的语法错，基本没有什么逻辑错误，偶尔有一两个逻辑错误确实是因为之前就没有想到这种情况的组合。我觉得这个学生的设计开发方法就是我所推崇的，也希望大家都朝这个方向努力。

2.3 浅谈代码风格

代码风格是个很重要的问题，关乎代码质量、生产效率，是区分普通设计人员和优秀设计人员的重要指标之一。代码风格也是个见仁见智的问题，没有唯一正确，只有相对合理。我们安排实验的首要任务是培养同学们的能力，所以有必要在这里谈一下代码风格的问题，让大家多少对这个问题重视一下。

代码风格之所以是个问题是因为，代码写出来不仅仅是给机器看的，更多的时间里代码是给人看的。如果只是给机器看，数万行代码可以放在一个文件，不加任何缩进和换行，而且所有的变量名都可以命名为“vzi101dba3832”这种形式。这种风格机器看起来一点困难都没有，但是人看起来就是一本“天书”。所以**务必记住**：代码写出来，要让自己过了一年半载还能看得懂改得了，要让同项目组的其它成员能看得懂改得了，要让项目无关的本领域设计人员在看了你的设计文档之后也能看得懂改得了。这就是好的代码风格的最基本的评价标准。

好的代码风格有很多细节，篇幅有限，这里主要提一下变量命名和代码对齐。

2.3.1 变量命名

变量命名要注意在名字中体现出该变量的含义，最好能够让人“望文知义”。如果变量的名字定义的好，很多时候没有注释和设计文档都可以把代码看得差不多明白。例如，进行五级流水 CPU 设计的时候，按照每一级的功能，把模块的名字定义为 fe_module、dec_module、exe_module、mem_module 和 wb_module 就比定义为 pipe1_module、pipe2_module、pipe3_module、pipe4_module 和 pipe5_module 要好的多。五级流水 CPU 设计中译码流水级中的相关判断，将执行级、访存级、写回级各级的目标寄存器号分别定义为 exe_dest、mem_dest 和 wb_dest 就比定义为 dest1、dest2、dest3 要好的多。

可以看出来好的变量名通常是一个短语而不是一个单词，那么一个短语中若干单词之间如何区分呢。常见的做法有两种，一种是下划线，一种是大小写。我个人是习惯于下划线的，这可能与我对英文大写字母不敏感有关，大小写区分的写法我一眼看过去划不出单词。当然用下划线隔开只是我个人喜好，同学们可以自行选择自己喜欢的方式。

将变量名定义为一个短语会碰到的问题是变量名容易过长，这就需要我们采用一些缩写形式。如 `counter` 可以缩写为 `cnt`，`forward` 可以缩写为 `fwd`，`source` 可以缩写为 `src`，`dest` 可以缩写为 `dst`，`buffer` 可以缩写为 `buf`，`XX_queue` 可以缩写为 `XXq`，等等。缩写没有一个所谓的通行标准，一般来说一家公司内部的缩写是统一的。同学们至少要保证一个设计中的缩写是一致的，譬如，如果一个设计中有的同学用 `wb` 代表 `write back`，而其它同学用 `wb` 代表 `write buffer`，那么这样的设计代码阅读起来就容易让人产生困惑。此外建议大家对于采用了缩写的变量名，最好在变量声明的地方把缩写前的短语记录在注释里面，这样会大大帮助别人理解你的缩写风格。

2.3.2 代码对齐

代码对齐主要有两个目的：一是容易规避语法错误，二是美观。

同学们在学习 C 语言程序设计的时候，可能已经接触过缩进的概念。同一缩进层次的代码的起始列对齐有助于我们一眼区分出代码的逻辑层次。此外像 Verilog 中的 `begin...end`、`case...endcase`、`module...endmodule` 都是配对出现，如果将这些关键字对齐在其所在的层次上，那么就比较容易检查出来有没有出现配对确实的情况，否则即使仿真综合工具报错，你定位错也要比较长的时间。

美观这件事情，很多同学可能不以为然，觉得只要写的代码是对的不就可以了吗，干嘛要看得好看。还是最初那句话，代码更多的时候是让人看的，而人看待一件事物的时候是抛不开主观感受的。同学们高考的时候，语文老师一定向你们强调过作文部分字迹要工整，道理是一样的。再精巧的设计，代码写得七零八落、参差不齐，别人也看不出个好来，甚至会曲解你的设计意图。又常说“字如其人”，对于程序员来说，代码反映了你的思维状态。作为我来说，我很难相信一段排列混乱的代码背后会是一套清晰的思维。所以美观这件要求，不是为了美观而美观，首要条件还是你的思路清晰了，在此基础之上稍加注意，则代码自然就体现出清晰简洁之美。

此外强调一个小细节，上下行**对齐的时候不要使用 Tab，而要用空格**。因为不同的文本编辑器对于 Tab 算作多少个空格可能是不一样的，这就有可能导致你写的代码在你自己看来已经排列得很整齐了，但是别人打开来看的时候就完全不是那么回事了。

Verilog 代码风格中还有很大一部分是用于强调可综合性、前后仿一致性和跨平台一致性的，也是很重要的。我们把其中一些关键内容融入到接下来关于可综合性的讨论和代码示例中，详细的原因我们不在此展开解释，感兴趣的同学可以自行阅读 Verilog 专家 Clifford E. Cummings 的一系列经典论文：

- A Proposal To Remove Those Ugly Register Data Types From Verilog

- full_case parallel_case, the Evil Twins of Verilog Synthesis
- Nonblocking Assignments in Verilog Synthesis, Coding Styles That Kill
- RTL Coding Styles That Yield Simulation and Synthesis Mismatches
- Verilog Nonblocking Assignments With Delays, Myths & Mysteries

2.4 可综合代码

Verilog 和 VHDL 这类硬件描述语言设计最初是用于大型数字电路的建模、仿真，由基于 HDL 的设计转换为逻辑门相互连接的电路图的工作仍是由设计人员手工完成的，这个过程既费时费力又容易出错。后来逻辑综合工具的出现和发展改变了这一状况。设计者可以使用 HDL 在 RTL 级对电路进行描述，然后选定标准单元库并定义相关设计约束，那么逻辑综合工具就会自动的将 HDL 语言转换为门级网表，这个转换的过程称为逻辑综合。

在逻辑综合过程中，工具能够支持的语言要素是 Verilog 语言的一个子集，也就是说并不是所有的 Verilog 语言要素都可以进行逻辑综合。表 2-1 列举了 Verilog-1995 中可综合的语言要素。表 2-2 列列举了 Verilog-2001 中可综合的语言要素。

表 2-1 Verilog-1995 中可综合语言要素（来源于 Stuart Sutherland）

Verilog 结构	描述
模块声明 module, endmodule	完全支持
端口声明 input, output, inout	完全支持，且支持任意向量大小
线网数据类型 wire, wand, wor, supply0, supply1	完全支持，且支持标量和向量
变量数据类型 reg, integer	可以是标量、向量或变量数组 变量赋值只能来自于一个结构化过程语句（procedure） integer 类型缺省为 32 bits
参数常量（parameter constants）	仅限于 integer 类型
整型数（literal integer numbers）	完全支持，且支持任意大小和进制
模块实例化	完全支持； 同时支持位置相关和名字相关两种端口实例化方式
原语实例化 and, nand, or, nor, xor, not, buf, bufif1, bufif0, notif1, notif0	完全支持
连续赋值语句 assign	完全支持； 且同时支持显式连续赋值和隐式连续赋值
过程赋值语句	完全支持； 但不支持 deassign 关键字
函数定义 function	仅支持部分结构
任务定义 task	仅支持部分结构
always 结构化过程语句 always	必须有敏感列表
begin ... end 块	完全支持； 命名块和非命名块都支持； 但是不支持 fork ... join 形式的块
阻塞赋值（=）和非阻塞赋值（<=）	完全支持； 同一个变量的所有赋值只能采取同一类型的赋值
条件判断语句	X and Z 均只视作“无关”位

Verilog 结构	描述
<u>if, if...else, case, casex, casez</u>	
<u>for 循环</u> <u>while 循环</u>	步进量的赋值必须为递进和递减
<u>disable statement group</u>	must be used within the same named block that is being disabled
操作符 & ~& ~ ^ ^~ ^~^ == != < > <= >= ! && << >> { {}} ?: + - * /	操作数可以是变量或向量，可以是常量或变量 不支持==和!=操作符
向量比特选择 向量部分选择	出现在赋值语句右侧时，完全支持； 出现在赋值语句左侧时，仅支持选择量为常量。

表 2-2 Verilog-2001 中新增的可综合语言要素（来源于 Synopsys 文档）

Verilog 结构	描述
敏感列表中用逗号分隔	完全支持
组合逻辑敏感列表用@*	完全支持
将端口和数据类型集成在一起的声明方式	完全支持
ANSI C 形式的端口声明	完全支持
连续赋值中的隐藏线网	完全支持
多维数组	完全支持
数组位选或部分选择运算符+: 和 -:	完全支持
<u>有符号数据类型</u>	完全支持
<u>有符号数字</u>	完全支持
<u>算术移位<<<, >>></u>	完全支持
<u>幂运算符**</u>	完全支持
sized parameters	完全支持
以名字相关传递的参数（parameter）	完全支持
本地参数（local parameter）	完全支持
编译制导`ifndef, `elsif, `undef	完全支持
<u>自动化任务和函数（automatic tasks and functions）</u>	完全支持
常数函数（constant functions）	完全支持
generate 表达式	完全支持

针对同学们完成课程实验的实际需要，我们对表 2-1 和表 2-2 的 Verilog 可综合语言要素做进一步精简，所有斜体下划线标记的部分也禁止同学们在 RTL 设计中使用。这里进一步禁止是为了避免作为初学者的同学们遇到一些“设计陷阱”。譬如 function、task、casex、casez 虽然是可综合的，但是如果使用不当，则容易出现前后仿真不一致的情况；又譬如 Verilog-2001 中的有符号数据类型虽然对于混合有大量有符号、无符号数运算的 datapath 设计很有帮助，但是如果对于运算表达式的符号判定规则不是特别明确，则容易出现设计错误；再譬如算术移位运算符“<<<, >>>”虽然 Synopsys 的工具是支持的，但是我们发现有其它工具并不支持，会出错。我们希望同学们首先把 Verilog 中最基本的可综合结构彻底掌握好以后，有余力的情况下再去掌握其它内容，在某些情况下提到编码效率。大家不要小看了这些最基本的，龙芯处理器 RTL 设计所用到的 Verilog 语言要素也没有超过我们推荐给同学们的范围。

2.5 推荐的可综合代码风格

上面介绍了 Verilog 语言的可综合语言要素。那么在进行课程实验的时候，我们对于如何使用这些语言要素写代码又有一个建议的代码风格，其中主要内容列举如下：

1. 建议对于数据通路（datapath）上的组合逻辑采用数据流建模的方法，通俗一点来说就是数据通路上的组合逻辑用 assign 语句写，禁止用 always 语句写。
2. 用 always 语句写组合逻辑只允许出现在生成状态机 next state 的时候，该语句中只允许出现阻塞赋值（=）。
3. 写时序逻辑的 always 语句中只允许出现非阻塞赋值（<=）。
4. 寄存器堆封装成单独的模块，以实例化方式使用。
5. case 语句任何情况下都要有 default 分支。
6. 模块实例化时的参数和端口只允许用名字相关的方式进行赋值和连接。
7. 数据通路的组合逻辑中，1 比特的逻辑运算用 &、|、~、^ 这类位运算符；控制信号的组合逻辑中，1 比特的逻辑运算用 &&、||、! 这三个逻辑运算符。
8. 如果你对于表达式中运算符的优先级没有确切把握的话，要么立刻去找一本参考书明确它，要么按照你需要的优先级层次加上括号；我们推荐前者，因为括号太多影响代码的可阅读性。
9. 不要在 RTL 代码中加入用于仿真的延迟信息；时序是否满足的问题通过综合阶段的 STA 保证，不是通过仿真来保证。
10. 禁用 //synopsys parallel case、//synopsys full case。

2.6 Verilog 示例

在本小节中，我们将给同学们提供一些 Verilog 的示例。这些示例都是一些基本的电路结构，如译码器、编码器、多选一等。一个简单 CPU 中所需的电路逻辑，除去加法器、乘法器这些较为复杂的运算部件外，余下都可以分解为这些基本的电路结构。

2.6.1 模块声明和实例化

```
module bottom #  
(  
    parameter A_WIDTH = 8,  
    parameter B_WIDTH = 4,  
    parameter Y_WIDTH = 2  
)  
(  
    input wire [A_WIDTH-1:0] a,  
    input wire [B_WIDTH-1:0] b,  
    input wire [      3:0] c,  
    output wire [Y_WIDTH-1:0] y,  
    output reg          z  
)
```

```

);
.....
endmodule

module top;

wire [15:0] btm_a;
wire [ 7:0] btm_b;
wire [ 3:0] btm_c;
wire [ 3:0] btm_y;
wire      btm_z;

bottom #(
    .A_WIDTH (16),
    .B_WIDTH ( 8),
    .Y_WIDTH ( 4)
)
inst_btm(
    .a (btm_a), //I
    .b (btm_b), //I
    .c (btm_c), //I
    .y (btm_y), //O
    .z (btm_z) //O
);

endmodule

```

2.6.2 基础逻辑门

```

wire [7:0] a;
wire [7:0] b;

assign y1 = ~a;    //反相器
assign y2 = a & b;  //与
assign y3 = a | b;  //或
assign y4 = a ^ b;  //异或
assign y5 = ~(a & b); //与非
assign y6 = ~(a | b); //或非

```

2.6.3 译码器

3-8 译码器。

```

module decoder_3_8(
    input [2:0] in,
    output [7:0] out
);
assign out[0] = (in == 3'd0);
assign out[1] = (in == 3'd1);
assign out[2] = (in == 3'd2);
assign out[3] = (in == 3'd3);
assign out[4] = (in == 3'd4);
assign out[5] = (in == 3'd5);
assign out[6] = (in == 3'd6);
assign out[7] = (in == 3'd7);
endmodule

```

相信通过上面的例子，你能够很容易的模仿写出 2-4 译码器、4-16 译码器。当然如果碰到 6-64、7-128 这种规格的译码器，用上述写法就会有点“费事”。感兴趣的同学可以自行查阅资料，用 generate 语句改善编码效率。

2.6.4 编码器

8-3 编码器，写法一：

```
module encoder_8_3(  
    input [7:0] in,  
    output [2:0] out  
);  
assign out = in[0] ? 3'd0 :  
             in[1] ? 3'd1 :  
             in[2] ? 3'd2 :  
             in[3] ? 3'd3 :  
             in[4] ? 3'd4 :  
             in[5] ? 3'd5 :  
             in[6] ? 3'd6 :  
             3'd7;  
endmodule
```

上面这样写，功能是没有问题的。但是它实际上实现得有点冗余了，因为这其实是一个优先级编码器，如果你能保证设计输入 in 永远至多只有一个 1，即所谓 **at-most-1-hot** 向量，那么可以像下面这样写：

```
module encoder_8_3(  
    input [7:0] in,  
    output [2:0] out  
);  
assign out = ({3{in[0]}} & 3'd0)  
            | ({3{in[1]}} & 3'd1)  
            | ({3{in[2]}} & 3'd2)  
            | ({3{in[3]}} & 3'd3)  
            | ({3{in[4]}} & 3'd4)  
            | ({3{in[5]}} & 3'd5)  
            | ({3{in[6]}} & 3'd6)  
            | ({3{in[7]}} & 3'd7);  
endmodule
```

2.6.5 多路选择器

我们先说 select 信号还没有译码的例子，而且这里故意将选择的路数设为不是 2 的幂次方，当 select 信号超过范围时输出全 0。

```
module mux5_8b(  
    input [7:0] in0, in1, in2, in3, in4,  
    input [2:0] sel,  
    output [7:0] out  
);  
assign out = (sel==3'd0) ? in0 :  
             (sel==3'd1) ? in1 :  
             (sel==3'd2) ? in2 :  
             (sel==3'd3) ? in3 :  
             (sel==3'd4) ? in4 :  
             8'b0;  
endmodule
```

同样，上面的例子引入了不必要的优先级关系，譬如 sel 是 1 的时候自然不是 0。所以，这个多路选择器还可以这样写。

```
module mux5_8b(  
    input [7:0] in0, in1, in2, in3, in4,  
    input [2:0] sel,  
    output [7:0] out  
);
```

```

assign out = ({8{sel==3'd0}} & in0)
            | ({8{sel==3'd1}} & in1)
            | ({8{sel==3'd2}} & in2)
            | ({8{sel==3'd3}} & in3)
            | ({8{sel==3'd4}} & in4);
endmodule

```

上面这种写法一定不要忘了写{8{}}中的8，因为这种笔误工具不会报错，但是功能不对。

如果 select 信号已经是译码后的位向量形式，也很容易写出来。

```

module mux5_8b(
    input [7:0] in0, in1, in2, in3, in4,
    input [4:0] sel,
    output [7:0] out
);
assign out = ({8{sel[0]}} & in0)
            | ({8{sel[1]}} & in1)
            | ({8{sel[2]}} & in2)
            | ({8{sel[3]}} & in3)
            | ({8{sel[4]}} & in4);
endmodule

```

其实回顾一下前面译码器的写法。可以看到第二个例子是把译码器和基于译码后向量的多路选择器合并在一起写出来的。

2.6.6 简单 MIPS CPU 的 ALU

我们用一个简单 MIPS CPU 中的 ALU 作为一个较复杂的组合逻辑设计的例子。整个 ALU 的控制信号 alu_control 采用 12 位的独热码，暂不考虑溢出例外的判定。

```

module simple_alu(
    input [11:0] alu_control,
    input [31:0] alu_src1,
    input [31:0] alu_src2,
    output [31:0] alu_result
);
wire op_add; //加法操作
wire op_sub; //减法操作
wire op_slt; //有符号比较，小于置位
wire op_sltu; //无符号比较，小于置位
wire op_and; //按位与
wire op_nor; //按位或非
wire op_or; //按位或
wire op_xor; //按位异或
wire op_sll; //逻辑左移
wire op_srl; //逻辑右移
wire op_sra; //算术右移
wire op_lui; //高位加载

assign op_add = alu_control[11];
assign op_sub = alu_control[10];
assign op_slt = alu_control[ 9];
assign op_sltu = alu_control[ 8];
assign op_and = alu_control[ 7];
assign op_nor = alu_control[ 6];
assign op_or = alu_control[ 5];
assign op_xor = alu_control[ 4];
assign op_sll = alu_control[ 3];
assign op_srl = alu_control[ 2];

```

```

assign op_sra = alu_control[ 1];
assign op_lui = alu_control[ 0];

wire [31:0] add_sub_result;
wire [31:0] slt_result;
wire [31:0] sltu_result;
wire [31:0] and_result;
wire [31:0] nor_result;
wire [31:0] or_result;
wire [31:0] xor_result;
wire [31:0] sll_result;
wire [63:0] sr64_result;
wire [31:0] sr_result;
wire [31:0] lui_result;

assign and_result = alu_src1 & alu_src2;
assign or_result  = alu_src1 | alu_src2;
assign nor_result = ~or_result;
assign xor_result = alu_src1 ^ alu_src2;
assign lui_result = {alu_src2[15:0], 16'b0};

wire [31:0] adder_a;
wire [31:0] adder_b;
wire      adder_cin;
wire [31:0] adder_result;
wire      adder_cout;
assign adder_a  = alu_src1;
assign adder_b  = alu_src2 ^ {32{op_sub | op_slt | op_sltu}};
assign adder_cin = op_sub | op_slt | op_sltu;
assign {adder_cout, adder_result} = adder_a + adder_b + adder_cin;

assign add_sub_result = adder_result;

assign slt_result[31:1] = 31'b0;
assign slt_result[0]   = (alu_src1[31] & ~alu_src2[31])
    | (~(alu_src1[31]^alu_src2[31]) & adder_result[31]);

assign sltu_result[31:1] = 31'b0;
assign sltu_result[0]   = ~adder_cout;

assign sll_result = alu_src2 << alu_src1[4:0];

assign sr64_result = {{32{op_sra&alu_src2[31]}}, alu_src2[31:0]}>>alu_src1[4:0];
assign sr_result  = sr64_result[31:0];

assign alu_result = ({32{op_add|op_sub}} & add_sub_result)
    | ({32{op_slt   }} & slt_result)
    | ({32{op_sltu  }} & sltu_result)
    | ({32{op_and   }} & and_result)
    | ({32{op_nor   }} & nor_result)
    | ({32{op_or    }} & or_result)
    | ({32{op_xor   }} & xor_result)
    | ({32{op_sll   }} & sll_result)
    | ({32{op_srl|op_sra}} & sr_result)
    | ({32{op_lui   }} & lui_result);
endmodule

```

接下来，我们介绍一些时序逻辑相关的代码例子。

2.6.7 寄存器

例子 1，最普通的上跳沿触发的 D 寄存器。

```
module Dflipflop(
    input    clk,
    input    din,
    output reg q
);
always @(posedge clk) begin
    q <= din;
end
endmodule
```

例子 2，带复位的 D 寄存器。写法一：

```
module Dflipflop_r(
    input    clk,
    input    rst,
    input    din,
    output reg q
);
always @(posedge clk) begin
    if (rst) q <= 1'b0;
    else    q <= din;
end
endmodule
```

写法二：

```
module Dflipflop_r(
    input    clk,
    input    rst,
    input    din,
    output reg q
);
always @(posedge clk) begin
    q <= ~rst & din;
end
endmodule
```

上述两种写法，电路功能都是一样的。我个人推荐写法一。因为 rst 清 0 操作通常具有最高优先级，用 if...else...这种写法，人在阅读代码的时候一目了然，不容易犯错。

例子 3，带使能端的 D 寄存器。写法一：

```
module Dflipflop_en(
    input    clk,
    input    en,
    input    din,
    output reg q
);
always @(posedge clk) begin
    if (en) q <= din;
end
endmodule
```

写法二：

```
module Dflipflop_en(
    input    clk,
    input    en,
    input    din,
    output reg q
```

```
);
always @(posedge clk) begin
    q <= en ? din : q;
end
endmodule
```

上述两种写法，电路功能都是一样的。我个人推荐写法一，原因与前面的带复位的推荐写法一是一样的，人看起来更直观。而且这种写法还可能带来一些额外的好处，即绝大多数综合工具只对写成第一种风格的代码会自动插入门控时钟或者引入带门控的寄存器。

这里举的例子都是单比特的寄存器，同学们应该可以很自然地将其推广到多比特的情况。

2.6.8 MIPS CPU 中的寄存器堆

寄存器堆是采用二维组织形式的“一堆寄存器”。在一个单发射五级流水简单 MIPS CPU 中，GR 对应一个 32 项、每项 32 位的寄存器堆。为了支持流水，该寄存器堆要能够每周期读出两个 32 位的数、写入一个 32 位的数。同时 MIPS CPU 还有一个特殊之处，即 0 号寄存器恒为 0。

```
module regfile(
    input    clk,
    input [4:0] raddr1,
    output [31:0] rdata1,
    input [4:0] raddr2,
    output [31:0] rdata2,
    input    we,
    input [4:0] waddr,
    input [31:0] wdata
);
reg [31:0] rf[31:0];
// WRITE
always @(posedge clk) begin
    if (we) rf[waddr] <= wdata;
end
// READ OUT 1
assign rdata1 = (raddr1 == 5'b0) ? 32'b0 : rf[raddr1];
// READ OUT 2
assign rdata2 = (raddr2 == 5'b0) ? 32'b0 : rf[raddr2];
endmodule
```

在上面的例子中，rf[waddr]、rf[raddr1]、rf[raddr2]这样的写法，实际上是通过综合工具推导出针对写地址和读地址的译码电路，实际上，上面例子对应下面这种表达。这里我们假设 decoder_5_32 是一个 5-32 译码器的模块。

```
module regfile(
    input    clk,
    input [4:0] raddr1,
    output [31:0] rdata1,
    input [4:0] raddr2,
    output [31:0] rdata2,
    input    we,
    input [4:0] waddr,
    input [31:0] win
);
reg [31:0] rf[31:0];
wire [31:0] waddr_dec, raddr1_dec, raddr2_dec;
decoder_5_32 U0(.in(waddr), .out(waddr_dec));
decoder_5_32 U1(.in(raddr1), .out(raddr1_dec));
decoder_5_32 U2(.in(raddr2), .out(raddr2_dec));
```

```
//WRITE
always @(posedge clk) begin
    if (we & waddr_dec[ 0]) rf[ 0] <= wdata;
    if (we & waddr_dec[ 1]) rf[ 1] <= wdata;
    .....
    if (we & waddr_dec[31]) rf[31] <= wdata;
end
//READ OUT 1
assign rdata1 = ({32{raddr1_dec[ 1]}} & rf[ 1])
                | ({32{raddr1_dec[ 2]}} & rf[ 2])
                .....
                | ({32{raddr1_dec[31]}} & rf[31]);
//READ OUT 2
assign rdata2 = ({32{raddr2_dec[ 1]}} & rf[ 1])
                | ({32{raddr2_dec[ 2]}} & rf[ 2])
                .....
                | ({32{raddr2_dec[31]}} & rf[31]);
endmodule
```

上述两种写法，第二种只是为了加深各位同学对于 `rf[addr]` 这种写法的理解，要知道虽然代码很短，但是其对应的逻辑是很多的。在平时的设计中我们还是推荐各位采用第一种写法。

这里顺带问一个问题，如果写有效时 (`we=1`)，且写地址和读地址相同的话，这一拍读出的结果是寄存器中的旧值还是新写入的值？各位不知道有没有看过 Patterson 的《计算机组成与设计》那本教材，其中在五级流水设计中提到了寄存器堆前半周期写，后半周期读，很显然我们的寄存器堆没有这个特性。所以请记住，如果你严格按照那本教材里面的说法去添加 `stall` 和 `forward` 逻辑，就会出问题。后面做到五级流水实验的时候，请切记这一点。

2.6.9 流水线

首先要说明的一点是，流水线电路纯粹就是一个数字电路的概念，不要一谈到流水线就仅仅认为是处理器中的流水线。

我们先来看一个完全不会被阻塞的 3 级流水线电路怎么写。

```
module non_stall_pipeline #
(
    parameter WIDTH = 100
)
(
    input      clk,
    input  [WIDTH-1:0] datain,
    output [WIDTH-1:0] dataout
);
reg [WIDTH-1:0] pipe1_data;
reg [WIDTH-1:0] pipe2_data;
reg [WIDTH-1:0] pipe3_data;

always @(posedge clk) begin
    pipe1_data <= datain;
end

always @(posedge clk) begin
    pipe2_data <= pipe1_data;
end

always @(posedge clk) begin
    pipe3_data <= pipe2_data;
end
end
```

```

assign dataout = pipe3_data;

endmodule

```

但是实际电路设计中，并不总是上面这种无阻塞的流水线。下面给出一个具有阻塞功能的流水线代码。

```

module stallable_pipeline #
(
  parameter WIDTH = 100
)
(
  input      clk,
  input      rst,
  input      validin,
  input [WIDTH-1:0] datain,
  input      out_allow,
  output     validout,
  output [WIDTH-1:0] dataout
);
reg      pipe1_valid;
reg [WIDTH-1:0] pipe1_data;
reg      pipe2_valid;
reg [WIDTH-1:0] pipe2_data;
reg      pipe3_valid;
reg [WIDTH-1:0] pipe3_data;

// pipeline stage1
wire      pipe1_allowin;
wire      pipe1_ready_go;
wire      pipe1_to_pipe2_valid;
assign pipe1_ready_go = .....
assign pipe1_allowin = !pipe1_valid || pipe1_ready_go && pipe2_allowin;
assign pipe1_to_pipe2_valid = pipe1_valid && pipe1_ready_go;
always @(posedge clk) begin
  if (rst) begin
    pipe1_valid <= 1'b0;
  end
  else if (pipe1_allowin) begin
    pipe1_valid <= validin;
  end
  if (validin && pipe1_allowin) begin
    pipe1_data <= datain;
  end
end

// pipeline stage2
wire      pipe2_allowin;
wire      pipe2_ready_go;
wire      pipe2_to_pipe3_valid;
assign pipe2_ready_go = .....
assign pipe2_allowin = !pipe2_valid || pipe2_ready_go && pipe3_allowin;
assign pipe2_to_pipe3_valid = pipe2_valid && pipe2_ready_go;
always @(posedge clk) begin
  if (rst) begin
    pipe2_valid <= 1'b0;
  end
  else if (pipe2_allowin) begin
    pipe2_valid <= pipe1_to_pipe2_valid;
  end

  if (pipe1_to_pipe2_valid && pipe2_allowin) begin
    pipe2_data <= pipe1_data;
  end
end
end

```

```

// pipeline stage3
wire    pipe3_allowin;
wire    pipe3_ready_go;
assign pipe3_ready_go = .....
assign pipe3_allowin = !pipe3_valid || pipe3_ready_go && out_allow;
always @(posedge clk) begin
    if (rst) begin
        pipe3_valid <= 1'b0;
    end
    else if (pipe3_allowin) begin
        pipe3_valid <= pipe2_to_pipe3_valid;
    end

    if (pipe2_to_pipe3_valid && pipe3_allowin) begin
        pipe3_data <= pipe2_data;
    end
end

assign validout = pipe3_valid && pipe3_ready_go;
assign dataout = pipe3_data;

endmodule

```

上面表述中 `pipeX_valid` 为 1 表示第 X 级流水级上当前存有有效的数据，为 0 表示第 X 级是空的。因为控制位的好处是，清空流水线的时候不用刷各级流水线 `data` 域的值，可以节约逻辑资源，但是需要注意的是根据流水级的 `data` 域信息产生控制信号时，有时候不要忘记“与上”这一级的 `valid` 信号。

`pipeX_allowin` 为 1 表示第 X 级可以接收前一级流水送来的数据。

`pipeX_ready_go` 为 1 表示第 X 级的信息可以传递到后一级流水级了。例如，假设在执行流水级用迭代方式运算除法，需要多拍才能完成，那么在迭代没有完成前，执行流水级的 `ready_go` 将一直为 0。

上面流水线的 RTL 写法对于流水级的关键控制信号采用了链式写法。这样写的好处是，如果发生流水级功能调整时，譬如将一级再切分为两级，或者将某一级的停顿周期数改变了，都只需要修改局部，不会涉及全局性的调整。有的同学可能会觉得我这种写法很麻烦，觉得自己写出来的简单一些。请你们针对具体情况分析，是否对于你所涉及的流水线，上面给出的代码中，某些 `pipeX_allowin` 和 `pipeX_ready_go` 可以恒置为 1，将其代入上面的表达式中，并将级联的控制信号完全展开，就形成了你们的那种简单版本。这里给出这样的一个流水线电路的“模板”式写法，主要是因为它易于理解不易出错，当同学们在自己进行流水线设计时如果因为流水级控制信号搞得很糊涂时，可以回过头来看看此处给出的例子。

再额外说一点，上面的写法是把流水线的 `control logic` 和 `data path` 写到了一个模块内部。你也可以把两者分在不同的模块中。国外的教科书上在描述流水线 CPU 时，通常会在流水线之外画一个叫做 `pipeline control logic` 的椭圆形框，从这个框引出控制信号去往各级流水所在触发器的使能端。这就是把 `control logic` 和 `data path` 分离的写法。我们在上面给出的例子，`pipeX_data` 就是 `data path` 主体。

2.7 小结

基于我们在工程实践中的一些体会，我们给同学们讲述了一些 Verilog 使用中的注意事项，并且给出了一些典型电路的代码实例，供同学们参考。

北京科技大学计算机组成原理课程设计19-20秋季

3 讲义三：CPU 实验开发环境使用说明

这一讲我们向各位同学介绍本课程中 CPU 实验的开发环境，命名为 `cpu_CDE`。

为了让同学们尽可能少花精力去熟悉掌握实验开发环境，从 CPU 实验开始的绝大多数实验都将使用一套相同的实验开发环境。通过这一讲，希望你们能够：

- (1) 基于所提供的 CPU 实验开发环境迅速开展 CPU 实验；
- (2) 了解 CPU 实验的一般性开发过程；
- (3) 了解我们所提供的 CPU 实验开发环境。

在编写本讲内容的时候，我们假定各位同学已经：

- (1) 基本会用 Verilog 写一个简单的数字电路设计；
 - (2) 基本会用 Vivado 工具进行 FPGA 工程开发，知道“仿真-综合-实现-下载（上板）”这一过程；
 - (3) 知道汇编语言是什么；
 - (4) 知道 Linux 是什么，知道最常用的 Shell 命令，如查看文件、进入某个目录、执行程序之类的；
 - (5) 知道基于 GCC 工具链编译的一般过程，即一个 C 或者汇编程序是如何一步步转化为一个可执行文件的。
- 上述内容如果你还不怎么了解，那么还是**强烈建议**你先去学习（复习）一下。

3.1 CPU 实验开发环境快速上手

下面给出利用我们提供 CPU 实验开发环境的：

- (1) **【解压环境】**把我们提供的实验开发环境解压到一个路径中没有中文字符的位置上，要确保你在这个位置下能运行 Vivado。
- (2) **【设计 myCPU】**用你习惯使用的文本编辑器（Vivado 中集成的文本编辑功能实在是不怎么样）将处理器核的 Verilog 代码编写好，重点注意顶层模块的模块名和接口信号必须按照规定要求定义。
- (3) **【myCPU 加入】**将写好的 CPU 代码拷贝到 `mycpu_verify/rtl/myCPU/`目录下。
- (4) **【编译 Func】**注意：环境中预置的功能测试程序（即 **Func**）已经默认完成了编译，除非你需要修改功能测试程序的源码（不建议），否则无需再次执行此步骤。

如果你是在 Windows 下面运行 Vivado：将 `func` 目录整体拷贝到一个已经安装了 MIPS-GCC 交叉编译工具的 Linux 环境中(或者将 `soft/func` 目录设置为虚拟机共享目录)，进入 `soft/func` 目录，先运行 `make clean`，再运行 `make`。将当前 `soft/func/obj` /整个目录的内容覆盖步骤（1）解压所在位置下的 `func/obj/`目录。（有关 `func` 编译的内容详见本章 3.3.5 到 3.3.7 节）。

如果你是在 Linux 下运行 Vivado：先确保你的 Linux 系统已经安装了 MIPS-GCC 交叉编译工具。然后进入 `func` 目录先运行 `make clean`，再运行 `make` 就可以了。

- (5) **【生成 Trace】**注意：环境中已经预置了生成好的 **Trace** 文件，除非你在第（4）步中修改了功能测试程序，并已经重新编译得到了结果，否则无需再次执行此步骤。但如果有时间，此处依然推荐你按以下步骤操作，以熟悉功能测试环境的大致使用方法和运行结果的观察方法。

进入 `cpu132_gettrace/run_vivado/cpu132_gettrace/`目录，打开 Vivado 工程 `cpu132_gettrace`，进行仿真，生成参考结果 `trace_ref.txt`。重点注意此时 `inst_ram` 和 `data_ram` 加载的是第（4）步编译出的结果。（参考模型生成 Trace 参见本章 3.3.4 节）

- (6) **【myCPU 仿真】**进入 `mycpu_verify/run_vivado/mycpu/`目录，打开 Vivado 工程 `mycpu`，将你在第（2）步新加的文件添加到工程中，进行仿真。看仿真输出 `log` 是否与给出的正确 `log` 一致。如果结果异常，则进行调试直至通过。重点注意此时 `inst_ram` 和 `data_ram` 加载的是第（4）步编译出的结果，即 `soft/func/obj/`目录下的内容。

- (7) 【myCPU 上板】回到第 (6) 步打开的 mycpu 这个工程中，进行综合实现，进行上板验证，观察实验箱上数码管显示结果，判断是否正确。**注意上板时，板子上的所有拨码开关都应该拨下。**如果结果与要求的一致，则 myCPU 验证成功至此结束，恭喜你；否则转到第 (8) 步进行问题排查。
- (8) 【反思】请按照下列步骤排查，重复第 (7)、(8) 步直至正确。
- 复核生成、下载的 bit 文件是否正确。
 - 如果判断生成的 bit 文件不正确，则重新生成 bit 文件。
 - 如果判断生成的 bit 文件正确，转 b)
 - 复核仿真结果是否正确。
 - 如果判断仿真结果不正确，则回到前面步骤 (6)。
 - 如果判断仿真结果正确，转 c)。
 - 检查实现时的时序报告 (Vivado 界面左侧 “IMPLEMENTATION” → “Open Implemented Design” → “Report Timing Summary”)。
 - 如果发现实现时时序不满足，则在 Verilog 设计里调优不满足的路径，或者降低 SoC_lite 的运行频率，即降低 clk_pll 模块的输出端频率，做完这些改动后，回到前面步骤 (7)。
 - 如果实现时时序是满足的，转 d)。
 - 认真排查综合和实现时的 Warning
 - 如果有尚未修正的 Warning，修正它们了，然后回到前面步骤 (7)。
 - 如果没有 Warning 了，转 e)。
 - 排查 RTL 代码规范，避免多驱动、阻塞赋值乱用、模块端口乱接、时钟复位信号接错，避免把模块调用处的输入输出接反了，查看那些从别处模仿来的“酷炫”风格的代码，查找有没有仿真时被 force 住的值导致仿真和上板不一致……如果怎么看代码都看不出问题，转 f)。
 - 如果你会使用 Vivado 的逻辑分析仪进行板上在线调试，那么就去调试吧；如果调试了半天仍然无法解决问题，转 g)。
 - 反思。真的，现在除了反思还能干什么？

3.2 CPU 实验的一般性开发过程

上这门课，同学们应该最关心每个 CPU 实验任务通过老师和助教检查，能拿到成绩得个高分。我们如何判断你的实验通过呢？只要你的 CPU 设计在 FPGA 实验板上运行规定程序能够得到期望的结果，就认为通过。这个判定方式意味着两件事：首先，你要用 Verilog 语言设计一个 CPU 并最终将其转化为 FPGA 上的电路实现；其次，这个电路实现上运行规定程序能够得到期望的结果，即所谓的“跑对”。

第一件事情相对来说是简单的，你只要一步一步完成“综合-实现-下载（上板）”这些 FPGA 工程开发中的步骤就可以了，勤劳的 EDA 工具会帮你将 Verilog 描述转化为最终的电路。

第二件事情是大部分同学相对头疼的，我们将多花一些篇幅来聊一聊。为了“跑对”，我们先要知道为什么会“跑错”。我们习惯将错误分为两类，一类叫功能错，另一类叫实现错。前者是指你所设计的数字电路从逻辑上就是不符合设计规格的；后者主要是指所设计的电路从功能逻辑上看是对的，但是最终实现的电路在 FPGA 板卡上的行为并不符合预期。为什么要区分这两种错？因为无论哪一类错，都会造成你的设计实现在 FPGA 板卡上出现不符合预期的行为。如果你没有从错误引入的源头进行分而治之，那么就会“眉毛胡子一把抓”，越调越乱，“欲速则不达”。

搞清楚错误分为功能错和实现错两类，就能理解为什么我们强调先要保证仿真通过，然后再去综合电路板上验证。因为仿真的时候，一切都是理想的，只看你的电路设计的逻辑是否正确。先保证逻辑正确，再保证实现未出错。我们自己的工程实践体会是，大多数的电路行为异常都是由于功能错而非实现错。所以，尽量在设计过程的早期（仿真验证阶段）将错误发现并纠正，而不是等到设计过程的晚期（实际电路验证阶段），可以大幅度的节约时间，提高工作效率。有些同学常常羡慕别人每个实验都完成得很轻松，自己每次都折腾得“死去活来”，

其实并不是你的同学真的就比你聪明多少，而是他们按照工程开发的一般性客观规律办事情，踏踏实实地先做过仿真再综合上板。

要实现一个有意义（可演示我们要求的实验功能就是一种意义）的计算机硬件系统，仅仅有你们设计的 CPU 是不够的，还要有其它组件（譬如外设）。我们进行功能仿真验证，目的是为了确保最终实现的计算机硬件系统是“对”的，所以我们干脆搭建一个硬件系统来进行功能仿真验证。我们提供给同学的实验环境中，mycpu_verfiy 目录下除了你需要添加的 CPU 设计的 RTL 代码外，所有其余 Verilog 代码都是仿真所需要硬件系统的其它组件。采用直接功能仿真最终的硬件系统的方式，好处是直截了当，缺点是定位错误费点劲。我们选用这种功能仿真验证的方式就是希望同学们能够更直观的分析自身设计的行为。不过为了降低大家定位错误的工作量，我们还在功能仿真阶段添加了指令执行结果踪迹（trace）逐条对比的机制，帮助大家直接定位到出错的指令，不再需要从系统对外的输出（如串口打印信息或 LED 灯的亮灭）沿着逻辑链条逐级反推到出错的指令。

在正常的工程开发过程中，功能仿真正确了，最终实现的电路基本上就是对的。但是从过往的教学经历来看，同学们碰到的实现错也不少。我最常听到同学们问我的一句话就是：“老师，我仿真都是对的，怎么上板子就不对了？”这里面绝大多数问题都是由于同学们写代码的时候没有严格遵守一些约定，并且在实现过程中也没有认真检查每个阶段 EDA 工具的提示信息所造成的。在我们的实验过程中，从 Verilog 描述到硬件电路的转换过程是通过 EDA 工具（Vivado）完成的。EDA 工具虽然勤劳，但是毕竟没那么智能，所以我们之前反复强调写 Verilog 代码要面向电路而非行为，就是为了确保 EDA 工具推导出的电路和你期望的是是一致的。另外一方面，EDA 工具不是上帝，不是说你要风，它就能给你创造出风。它在将你的 Verilog 描述实现为电路的过程中，发现你的代码出了错（报 Error）、有极大的出错风险（报 Critical Warning）、电路逻辑级数过长或是主频定得太高（报 Timing Violation）的时候，都是会及时向你报告的，告诉你它可能无法满足你的需求。可是不少同学往往选择主观忽视这些过程报告，只关心最后上板子之后对不对，其结果可想而知。所以我们再次强调，尽量严格按照 LEC02 介绍的 Verilog 语言使用注意事项进行代码开发，并且在用 Vivado 工具进行仿真、综合、实现的过程中确保没有报 Error、报 Critical Warning 以及报 Timing Violation，那么最终下载到 FPGA 实验板上的电路的行为基本上就会与你功能仿真的情况是一致的。

不过有时候我们也不能排除是板卡本身出了硬件问题。如果你怀疑这方面，那么可以多找几块其它的板卡（尤其是别的同学能够正常演示它的设计的板卡）试一下，如果出错的情况是一样的话，那么基本可以认为还是设计实现过程中出了问题。

总之，不要一碰到问题就觉得它时“诡异”的、“玄妙”的、“无法解释”的，我们实现的是数字电路，出了问题一定有原因，去定位问题！

最后，总结一下 CPU 实验的一般性开发过程。

第 1 步，想清楚你的设计，用 Verilog 描述出来，把写好的 Verilog 代码拷贝到所提供实验环境的指定位置。

第 2 步，进行功能仿真，一遍一遍地修改你的设计，直到仿真通过。

第 3 步，进行综合实现，一遍一遍地修改你的设计和约束，直到所有的 Error、Critical Warning 和 Timing Violation 都不出现。

第 4 步，将生成的 bit 流文件下载到 FPGA 上进行实际演示操作，如果出现异常，首先确保你第 2 步和第 3 步真的做对了。

3.3 CPU 实验开发环境

3.3.1 CPU 实验开发环境组织与结构

整个 CPU 实验开发环境 cpu_CDE 的目录结构及各主要部分的功能如下：红色部分为大家自实现的 CPU，黑色部分是我们已经搭建好的其余部分。

cpu	目录，存放 TinyMIPS 的实现
cpu132_gettrace	目录，存放获取 trace 的 Verilog 设计
rtl	目录，SoC_lite 的 RTL 源码
BRIDGE	目录，存放 bridge_1x2 模块，充当 SRAM Crosbar
CONFREG	目录，存放 confreg 模块，控制开发板上的 GPIO
cpu132	目录，存放龙芯开源 GS132 处理器核
xilinx_ip	目录，存放 Xilinx 的 IP 核
soc_lite_top.v	SoC_lite 的顶层模块
run_vivado	目录，存放 Vivado 工程相关的文件
cpu132_gettrace	目录，存放 Vivado 工程
cpu132_gettrace.xpr	gettrace 的 Vivado 工程
soc_lite.xdc	Vivado 使用的设计约束文件，定义 FPGA 的 IO 约束
testbench	目录，存放仿真文件
tb_top.v	仿真顶层，该模块会抓取 debug 信息并且生成 trace 文件
golden_trace.txt	预生成的 trace
soc_axi_func	AXI 版功能测试，默认请使用此目录中的文件进行功能测试
rtl	目录，存放 SoC_lite 的源码
axi_wrap	目录，存放处理器 AXI 接口的包装模块
CONFREG	目录，存放 confreg 模块的实现
myCPU	目录，此处放置你的 CPU 实现
ram_wrap	目录，存放 AXI RAM 的包装模块，用于模拟访存随机延迟
xilinx_ip	目录，存放 Xilinx 的 IP 核
soc_axi_lite_top.v	SoC_lite 的顶层模块
run_vivado	目录，存放 Vivado 工程相关的文件
mycpu_prj1	目录，存放 Vivado 工程
mycpu.xpr	myCPU 的 Vivado 工程，可运行功能测试
soc_lite.xdc	Vivado 使用的设计约束文件，定义 FPGA 的 IO 约束
testbench	目录，存放仿真文件
mycpu_tb.v	仿真顶层，该模块会抓取 debug 信息和 trace 进行比对
soc_sram_func	SRAM 版功能测试，当 AXI 版有误时可以尝试
rtl	（介绍略，下同）
BRIDGE	
CONFREG	
myCPU	目录，此处放置你的 CPU 实现
xilinx_ip	
soc_axi_lite_top.v	
run_vivado	
mycpu_prj1	
mycpu.xpr	
soc_lite.xdc	
testbench	
mycpu_tb.v	
soft	目录，存放实验环境的软件部分
func	目录，存放功能测试程序的实现
include	目录，存放编译所需的头文件
cpu_cde.h	开发环境用到的宏定义，例如 LED、数码管的地址
asm.h	MIPS 汇编需要的宏定义，例如 LEAF(x)
regdef.h	MIPS 汇编用到的 ISA 寄存器定义
inst	目录，存放各功能点的验证程序
Makefile	子目录的 Makefile，用于编译功能点测试
inst_test.h	功能点测试使用的头文件
*.S	功能点测试的汇编实现
obj	目录，存放功能测试的编译结果
*	参见 3.3.6 节
start.S	存放功能测试程序的入口，初始化功能测试
bin.lds.S	链接脚本，用于链接目标文件得到最终的二进制文件

```

├──convert.c
├──Makefile
├──rules.make
└──Readme_first.txt

```

将生成的二进制转换为 coe 和 mif 文件的转换程序源码
顶层 Makefile，用于编译整个功能测试程序
部分编译规则，被顶层 Makefile 调用
说明文档

3.3.2 TinyMIPS 处理器的目录结构

在开发环境中，“cpu”目录已经放置了 TinyMIPS 处理器的完整实现，其目录结构如下所示：

```

├──bus
│   └──AXIAdapter.v
├──core
│   └──pipeline
│       └──PipelineController.v
│           ├──PipelineDeliver.v
│           ├──stage
│           │   ├──if
│           │   │   ├──PC.v
│           │   │   └──IFID.v
│           │   ├──id
│           │   │   ├──ID.v
│           │   │   ├──RegGen.v
│           │   │   ├──FunctGen.v
│           │   │   ├──OperandGen.v
│           │   │   ├──BranchGen.v
│           │   │   ├──MemGen.v
│           │   │   └──IDEX.v
│           │   ├──ex
│           │   │   ├──EX.v
│           │   │   └──EXMEM.v
│           │   ├──mem
│           │   │   ├──MEM.v
│           │   │   └──MEMWB.v
│           │   └──wb
│           │       └──WB.v
│           ├──storage
│           │   ├──RegFile.v
│           │   └──RegReadProxy.v
│           └──Core.v
├──include
│   ├──bus.v
│   ├──debug.v
│   ├──funct.v
│   ├──iobuf.v
│   ├──opcode.v
│   ├──pcdef.v
│   ├──segpos.v
│   └──sim.v
├──ip
│   ├──InternalCrossbar
│   │   └──InternalCrossbar.xci
├──mmu
│   └──StaticMMU.v

```

目录，存放总线相关实现
SRAM 转 AXI 接口转换桥
目录，存放处理器核心实现
目录，存放流水线控制相关实现
流水线控制器
流水线中间级使用的 D 触发器
目录，存放各流水级的实现
目录，存放 IF（取指）级的实现
PC 模块，产生取指地址和请求
IFID 中间级
目录，存放 ID（译码）级的实现
ID 模块，用于指令译码
寄存器读写请求译码模块
ALU 操作码译码模块
ALU 操作数译码模块
分支请求译码模块
访存请求译码模块
IDEX 中间级
目录，存放 EX（执行）级的实现
EX 模块，计算需要运算的指令的结果
EXMEM 中间级
目录，存放 MEM（访存）级的实现
MEM 模块，发起访存请求
MEMWB 中间级
目录，存放 WB（写回）级的实现
WB 模块，处理内存读及写回执行结果
目录，存放寄存器堆相关实现
寄存器堆的实现
寄存器读取转发模块，解决数据冲突
处理器核心的顶层模块，SRAM 接口
目录，存放所有头文件（常量定义）
总线宽度相关的常量定义
用于生成调试信息的辅助宏
ALU 操作码相关的常量定义
用于生成三态门 buffer 的辅助宏
指令操作码相关的常量定义
处理器取指的起始 PC 地址定义
指令译码数据域相关的常量定义
仿真用模块相关的常量定义
目录，存放 Xilinx 的 IP 核配置
目录，存放 AXI Crossbar IP
AXI Crossbar IP 的配置，用于功能测试
目录，存放内存管理单元相关的实现
静态 MMU，用于进行静态虚实地址转换

├──sim	目录，存放仿真用的辅助模块
│ ├──ROM.v	ROM 模块的实现，存放指令
│ └──RAM.v	RAM 模块的实现，充当内存，存放数据
├──mycpu_top.v	运行功能测试所需的 CPU 顶层，AXI 接口
├──README.md	处理器实现的相关说明
└──TinyMIPS.v	SoC 中实际使用的 CPU 顶层，AXI 接口

3.3.3 SoC_Lite

我们 CPU 实验课不仅要教同学们设计一个 CPU，还要教你们用这个 CPU 搭建出一个有意义的计算机硬件系统。在现阶段，我们先搭建一个简单的硬件系统。这个系统里面有你们设计的 CPU（mycpu），供 CPU 访问的指令 RAM（iram）和数据 RAM（dram），用于显示和输入的 LED 灯、数码管、按键，以及时钟、复位等。这个硬件系统是通过我们的 FPGA 实验板实现的。其中 LED 灯、数码管、按键以及时钟晶振是焊在 PCB 板上的，其余部分都实现在 FPGA 中。请注意，FPGA 中所实现的内容也可以看作是一个系统（System），而这些内容都实现在一个芯片（Chip）内部，我们通常将这里在 FPGA 中所实现的内容称为 SoC（System On Chip）。图 3-1 给出了整个简单硬件系统的结构示意图，中间 SoC_Lite 区域内包含的就是最终实现在 FPGA 中的内容。可见 SoC_Lite 中除了 mycpu 以外还有其它模块。整个 SoC 的设计代码位于 mycpu_verify/rtl/目录下。

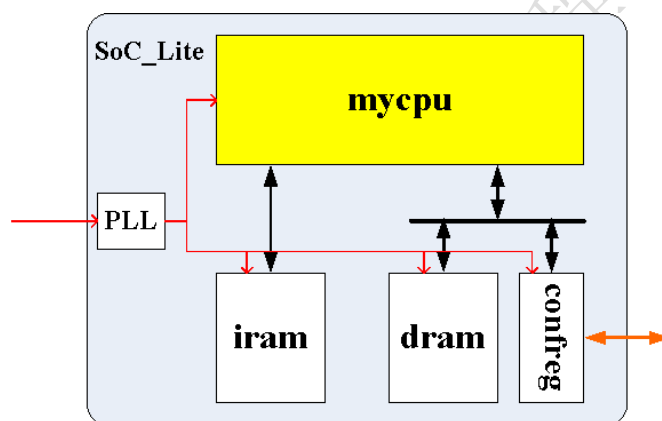


图 3-1 基于 mycpu 的简单硬件系统

SoC_Lite 中的 mycpu、iram 和 dram 各自功能是自明的。这里简单解释一下 PLL、config 以及 mycpu 与 dram、config 之间的二选一。

我们实验板上给 FPGA 芯片提供的时钟（来自于晶振）是 100MHz，如果直接用这个时钟作为 SoC_Lite 中各个模块的时钟，我们担心有些同学设计的 mycpu 无法达到这个频率，所以调用 Xilinx 的 PLL IP，以 100MHz 输入时钟作为参考时钟，生成出一个频率可以调节的时钟。该时钟默认频率 50MHz。

config 是 configuration register 的简称，是 SoC 内部的一些配置寄存器，实验中我们用来操控板上的 LED 灯、数码管，接收外部按键的输入。简要解释一下这个操控的机理：外部的 LED 灯、数码管以及按键都是直接连接到 FPGA 的引脚上的，通过控制 FPGA 引脚上的电平的高低就可以控制 LED 灯和数码管，而按键是否按下也可以通过观察 FPGA 引脚上电平的变化来判断。这些 FPGA 引脚又进一步连接到 config 中的某些寄存器上。所以 CPU 可以通过观察、操控这些寄存器来实现对于 FPGA 相关引脚的观察、操控。

mycpu 和 dram、config 之间有一个二选一，这是因为 config 中的寄存器是 memory mapped，也就是说从 CPU 的角度来看，config 也是内存。dram 是数据 RAM，自然是内存。那么 config 和 dram 如何区分？用地址进行区分，所以就有了一个二选一。

最后再强调一点，SoC_Lite 是进行 FPGA 综合实现时的顶层。

3.3.4 功能仿真验证

说到 Verilog 设计的功能仿真验证，大多数讲 Verilog 的书籍都会给出如图 3-2 的示意图。

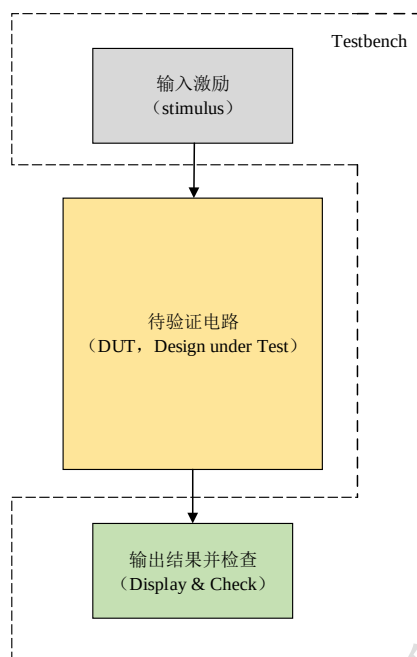


图 3-2 功能仿真验证

这个图其实也蛮好理解的，我们给待验证电路（DUT）一些特定的输入激励，然后观察 DUT 的输出结果是否如我们预期。所谓“不管白猫、黑猫，只要抓到老鼠就是好猫”，这里“老鼠”就是激励，期望结果是“抓到老鼠”，只要被验证对象在这个激励下得到所期望的结果，哪怕它明明是只黄鼠狼，我们也认为它是一只好猫。

我们给 CPU 设计进行功能验证时，沿用的依然是上面的思路，只不过输入激励和输出结果检查与普通的数字电路设计不太一样。这里输入激励是一段测试指令序列，通常是用汇编语言或 C 语言编写，用编译器编译出来的机器代码。输出结果是什么呢？如果我们用整个执行序列最终的结果作为检查的依据，是可以的，但是出错的定位就比较困难。同学们都开发过 C 程序，调试的时候应该都用过单步调试，这种调试方式能够快速定位出错的位置。我们提供给同学们的实验开发环境就使用了这种“单步调试”的策略。具体来说，我们先找一个已知的功能上是正确的 CPU（开源的龙芯 GS132 处理器），运行一遍测试指令序列，将每条指令的 PC 和写寄存器的信息记录下，记为 `golden_trace`；然后我们在验证 `myCPU` 的时候，也跑同样的指令序列，在 `myCPU` 每条指令写寄存器的时候，将设计中的 PC 和写寄存器的信息同之前的 `golden_trace` 进行比对，如果不一样，那么立刻报错停止仿真。

对 MIPS 指令熟悉的同学可能马上就会问：分支指令和 `store` 指令不写寄存器，上面的方式没法判断啊？这些同学提的问题相当对。但是大多数情况下，分支和 `store` 指令执行的错误会被后续的写寄存器的指令判断出来。例如，如果分支跳转的不对，那么错误路径上第一条会写寄存器的指令的 PC 就会和 `golden_trace` 中的不一致，也会报错并停下来。`store` 执行错了，后续从这个位置读数的 `load` 指令写入寄存器的值就会与 `golden_trace` 中的不一致，也会报错并停下来。我们这样设计 `trace`，是在报错的及时性和 CPU debug 接口的复杂度之间作了权衡。

上面我们介绍了利用 `trace` 进行功能仿真验证错误定位的基本思路，下面我们具体介绍一下如何生成 `golden_trace`，以及 `myCPU` 验证的时候是如何利用 `golden_trace` 进行比对的。

参考模型生成 `golden_trace`

功能验证程序 `func` 编译完成后，就可以使用验证平台里的 `cpu132_gettrace` 运行仿真生成参考 `trace` 了。

`cpu132_gettrace` 也支持综合实现，然后上板运行，上板运行效果就是大家自实现 `myCPU` 上板运行的正确的效果。

`cpu132_gettrace` 里是采用 `gs132` 搭建的 SoC_Lite，见图 3-1。仿真顶层为 `cpu132_gettrace/testbench/tb_top.v`，与抓取 `golden_trace` 相关的重要代码如下：

```

.....
`define TRACE_REF_FILE "../../../../../golden_trace.txt" //参考 trace 的存放目录
`define END_PC 32'hbfc00100 //func 测试完成后会 32'hbfc00100 处死循环
.....
assign debug_wb_pc      = soc_lite.debug_wb_pc;
assign debug_wb_rf_wen  = soc_lite.debug_wb_rf_wen;
assign debug_wb_rf_wnum = soc_lite.debug_wb_rf_wnum;
assign debug_wb_rf_wdata = soc_lite.debug_wb_rf_wdata;
.....
// open the trace file;
integer trace_ref;
initial begin
    trace_ref = $fopen(`TRACE_REF_FILE, "w"); //打开 trace 文件
end

// generate trace
always @(posedge soc_clk)
begin
    if(!debug_wb_rf_wen && debug_wb_rf_wnum!=5'd0) //trace 采样时机
    begin
        $fdisplay(trace_ref, "%h %h %h %h", `CONFREG_OPEN_TRACE ,
            debug_wb_pc, debug_wb_rf_wnum, debug_wb_rf_wdata_v); //trace 采样信号
    end
end
end
.....

```

Trace 采样的信号为:

- (1) CPU 写回级（最后一级，记为“wb”）的 PC，要求大家将每条指令的 PC 一路带到 wb 级。
- (2) wb 级的写回使能。
- (3) wb 级的写回的目的寄存器号。
- (4) wb 级的写回的目的操作数。

显然并不是每时每刻，CPU 都是有写回的，因此 Trace 采样需要有一定的时机：wb 级写通用寄存器堆信号有效，且写回的目的寄存器号非 0。（大家可以思考下，为什么此处判断写回目的寄存器非 0 时才采样？）

myCPU 功能验证使用 golden_trace

myCPU 功能验证所使用的 SoC 架构依然是图 3-1 展示的 SoC_Lite，但 testbench 就与 cpu132_gettrace 里的有所不同了，见 mycpu_verify/testbench/mycpu_tb.v，重点部分代码如下：

```

.....
`define TRACE_REF_FILE "../../../../../cpu132_gettrace/golden_trace.txt"
//参考 trace 的存放目录
`define CONFREG_NUM_REG soc_lite.confreg.num_data //confreg 中数码管寄存器的数据
`define END_PC 32'hbfc00100 //func 测试完成后会 32'hbfc00100 处死循环
.....
assign debug_wb_pc      = soc_lite.debug_wb_pc;
assign debug_wb_rf_wen  = soc_lite.debug_wb_rf_wen;
assign debug_wb_rf_wnum = soc_lite.debug_wb_rf_wnum;
assign debug_wb_rf_wdata = soc_lite.debug_wb_rf_wdata;
.....
//get reference result in falling edge

```

```

reg [31:0] ref_wb_pc;
reg [4 :0] ref_wb_rf_wnum;
reg [31:0] ref_wb_rf_wdata_v;
always @(negedge soc_clk) //下降沿读取参考 trace
begin
    if(!debug_wb_rf_wen && debug_wb_rf_wnum!=5'd0 && !debug_end && `CONFREG_OPEN_TRACE) /
//读取 trace 时机与采样时机相同
    begin
        $fscanf(trace_ref, "%h %h %h %h", trace_cmp_flag ,
                ref_wb_pc, ref_wb_rf_wnum, ref_wb_rf_wdata_v); //读取参考 trace 信号
    end
end

//compare result in rsing edge
always @(posedge soc_clk) //上升沿将 debug 信号与 trace 信号对比
begin
    if(!resetn)
    begin
        debug_wb_err <= 1'b0;
    end
    else if(!debug_wb_rf_wen && debug_wb_rf_wnum!=5'd0 && !debug_end && `CONFREG_OPEN_TRAC
E) //对比时机与采样时机相同
    begin
        if ( (debug_wb_pc!=ref_wb_pc) || (debug_wb_rf_wnum!=ref_wb_rf_wnum)
            || (debug_wb_rf_wdata_v!=ref_wb_rf_wdata_v) ) //对比时机与采样时机相同
        begin
            $display("-----");
            $display("[%t] Error!!!", $time);
            $display("    reference: PC = 0x%8h, wb_rf_wnum = 0x%2h, wb_rf_wdata = 0x%8h",
                    ref_wb_pc, ref_wb_rf_wnum, ref_wb_rf_wdata_v);
            $display("    mycpu      : PC = 0x%8h, wb_rf_wnum = 0x%2h, wb_rf_wdata = 0x%8h",
                    debug_wb_pc, debug_wb_rf_wnum, debug_wb_rf_wdata_v);
            $display("-----");
            debug_wb_err <= 1'b1; //标记出错
            #40;
            $finish; //对比出错, 则结束仿真
        end
    end
end
end
.....
//monitor test
initial
begin
    $timeformat(-9,0," ns",10);
    while(!resetn) #5;
    $display("=====");
    $display("Test begin!");
    while(`CONFREG_NUM_MONITOR)
    begin
        #10000; //每隔 10000ns, 打印一次写回级 PC, 帮助判断 CPU 是否死机或死循环
        $display ("    [%t] Test is running, debug_wb_pc = 0x%8h", debug_wb_pc);
    end
end
end

```

```

//test end
wire global_err = debug_wb_err || (err_count!=8'd0);
always @(posedge soc_clk)
begin
    if (!resetn)
    begin
        debug_end <= 1'b0;
    end
    else if(debug_wb_pc==`END_PC && !debug_end)
    begin
        debug_end <= 1'b1;
        $display("=====");
        $display("Test end!");
        $fclose(trace_ref);
        #40;
        if (global_err)
        begin
            $display("Fail!!!Total %d errors!",err_count); //全局出错, 打印 Fail
        end
        else
        begin
            $display("----PASS!!!"); //全局无错, 打印 PASS.
        end
        $finish;
    end
end
end
.....

```

由于验证平台的 trace 比对时机同采样时机，都是要求指令具有写回且写回目的寄存器号非 0，才进行比对。所以存在以下三种情况，CPU 出错，而 trace 比对无法发现：

- (1) myCPU 死机，时钟没有写回；
- (2) myCPU 执行一段死循环程序，且该死循环程序没有写回的指令，比如程序“1: b 1b; nop;”。
- (3) myCPU 执行 store 指令出错。

第(3)种情况只能等后续的 Load 再读出时才能发现出错，前两种情况在我们提供的验证平台上依然可以被发现，这是通过 mycpu_tb.v 中的“monitor test”机制实现的，就是每隔 10000ns，打印一次写回级 PC。如果发现 Vivado 控制台不再打印写回级 PC，或者打印的都是同一 PC，则说明 CPU 死机了；如果 Vivado 控制台打印的 PC 具有很明显的规律，不停重复打印，则说明陷入了死循环。借此，可以帮助大家判断 myCPU 是否执行 func 出错。

3.3.5 func 程序说明

func 程序分为 func/start.S 和 func/inst/*.S，都是 MIPS 汇编程序：

- (1) func/start.S：主函数，调用 func/inst/下的各汇编程序。
- (2) func/inst/*.S：针对每条指令或功能点有一个汇编测试程序。

主函数 func/start.S 中主题部分代码如下，分为三大部分，具体查看红色注释。

```

.....
#以下是设置程序开始的 LED 灯和数码管显示，单色 LED 全灭，双色 LED 灯一红一绿。
LI (a0, LED_RG1_ADDR)
LI (a1, LED_RG0_ADDR)

```

```

LI (a2, LED_ADDR)
LI (s1, NUM_ADDR)

LI (t1, 0x0002)
LI (t2, 0x0001)
LI (t3, 0x0000ffff)
lui s3, 0

sw t1, 0(a0)
sw t2, 0(a1)
sw t3, 0(a2)
sw s3, 0(s1)

#以下是运行各功能点测试，每个测试完执行 wait_1s 等待一段时间，且数码管显示加 1。
inst_test:
jal n1_lui_test  #lui
nop
jal wait_1s
nop
jal n2_addu_test  #addu
nop
.....

```

#以下是显示测试结果，PASS 则双色 LED 灯亮两个绿色，单色 LED 不亮；

#Fail 则双色 LED 灯亮两个红色，单色 LED 灯全亮。

```

test_end:
test_end:
LI (s0, 0x14)
NOP
NOP
NOP
beq s0, s3, 1f
nop

LI (a0, LED_ADDR)
LI (a1, LED_RG1_ADDR)
LI (a2, LED_RG0_ADDR)

LI (t1, 0x0002)
NOP
NOP

sw zero, 0(a0)
sw t1, 0(a1)
sw t1, 0(a2)
.....

```

每个功能点的测试代码程序名为 n*_*_test.S，其中第一个“*”处为编号，如有 15 个功能点测试，则从 n1 编号到 n15。每个功能点的测试，其测试代码大致如下。其中红色部分标出了关键的 3 处代码。

```

.....
LEAF(n1_lui_test)
.set noreorder
lui s0, 0x0100      #加载功能点编号到 s0 的高 8 位
addiu s2, zero, 0x0

```

```

lui t2, 0x1
###test inst
addiu t1, zero, 0x0
TEST_LUI(0x0000, 0x0000)
.....#测试程序，省略
TEST_LUI(0xf0af, 0xf0a0)
###detect exception
bne s2, zero, inst_error
nop
###score ++          #s3 存放功能测试计分，没通过一个功能点测试，则+1
addiu s3, s3, 1
###output a0|s3
inst_error:
or t0, s0, s3        #s0 高 8 位为功能点编号，s3 低 8 位为通过功能点数，相与结果显示到数码管上。
sw t0, 0(s1)         #s1 存放数码管地址
jr ra
nop
END(n1_lui_test)

```

从以上可以看到，测试程序的行为是：当通过第一个功能测试后，数码管会显示 0x0100_0001，随后执行 wait_1s；执行第二个功能点测试，再次通过数码管会显示 0x0200_0002，执行 wait_1s.....依次类推。显示，每个功能点测试通过，应当数码管高 8 为和低 8 位永远一样。如果中途数码管显示从 0x0500_0005 变成了 0x0600_0005，则说明运行第六个功能点测试出错。

最后，再来看下 wait_1s 函数的代码，其实使用一个循环来暂停测试程序执行的，如下：

```

wait_1s:
    LI (t1,SIMU_FLAG_ADDR)
    lui t0, zero
    NOP
    NOP
    lw t2, 0x(t1) //读取 confreg 模块里的 SIMU_FLAG_REG 的值
    NOP          //仿真时，SIMU_FLAG_REG 复位值为全 1
    NOP          //综合实现时，SIMU_FLAG_REG 复位值为全 0
    NOP
    bne t2, zero, 1f
    nop
    lui t0, 0x20 //综合实现时，wait_1s 循环次数为 0x20+0x1
1:
    addiu t0, 1 //仿真时，wait_1s 循环次数为 0x1
    NOP
    NOP
    NOP
2:
    addiu t0, -1
    NOP
    NOP
    NOP
    bne t0, zero, 2b
    nop
    jr ra
nop

```

红色部分为关键部分，其设置了循环的次数：在仿真环境下，软件程序自动设置循环次数为 1；在综合实现环

境下，设置循环次数为 33×2^{16} 。之所以这样设置，是因为 FPGA 运行远远快于仿真的速度，假设 CPU 运行一个程序需要 10^6 个 CPU 周期，再假设 CPU 在 FPGA 上运行频率为 10MHz，那其在 FPGA 上运行完一个程序只需要 0.1s；同样，我们仿真运行这个程序，假设我们仿真设置的 CPU 运行频率也是 10MHz，那我们仿真运行完这个程序也是只需要 0.1s 吗？显然这是不可能的，仿真是软件模拟 CPU 运行情况的，也就是它要模拟每个周期 CPU 内部的变化，运行完这一个程序，需要模拟 10^6 个 CPU 周期，根据我们实际统计发现，Vivado 的仿真器 Xsim 运行 SoC_lite 的仿真，模拟一个周期，大约需要 600us，Xsim 为了模拟 10^6 个周期，其所运行的实际时间约 10min。

同一程序，运行仿真测试大约需要 10 分钟，而在 FPGA 上运行只需要 0.1 秒（甚至更短，比如 FPGA 运行到 50Mhz，则运行完程序只需要 0.02s）。所以我们如果用上板运行时的 wait_1s 函数在仿真时运行，则我们会陷入到 wait_1s 长时间等待中；而如果我们用仿真时的 wait_1s 函数上板运行，则 wait_1s 时间太短，导致我们无法看到数码管累加的效果。在开发环境 cpu_CDE 中，设置了 func 自动获取仿真和综合时的 wait_1s 的循环次数，这样上板和仿真就可以使用同一套软件代码了。

如果大家在自实现 CPU 上板运行过程中，发现数码管累加跳动太慢，请调小 wait_1s 里的循环次数；如果发现数码管累加跳动太快，请调大 wait_1s 里的循环次数。

3.3.6 编译脚本说明

func 编译脚本为验证平台目录下的 func/Makefile，对 Makefile 了解的可以去看下该脚本。该脚本支持以下命令：

- (1) make help : 查看帮助信息。
- (2) make : 编译得到仿真下使用的结果。
- (3) make clean : 删除 *.o, *.a 和 ./obj/ 目录。
- (4) make reset : 执行命令“make clean”，且删除 convert, bin.ld。

3.3.7 编译结果说明

func 编译结果位于 func/obj/下，共有 8 个文件，各文件具体解释见表 3-1，文件生成关系见图 3-3。

表 3-1 编译生成文件

文件名	解释
data_ram.coe	重新定制 data ram 所需的 coe 文件
data_ram.mif	仿真时 data ram 读取的 mif 文件
inst_ram.coe	重新定制 inst ram 所需的 coe 文件
inst_ram.mif	仿真时 inst ram 读取的 mif 文件
main.bin	编译后的代码段，可以不用关注
main.data	编译后的数据段，可以不用关注
main.elf	编译后的 elf 文件
test.s	对 main.elf 反汇编得到的文件

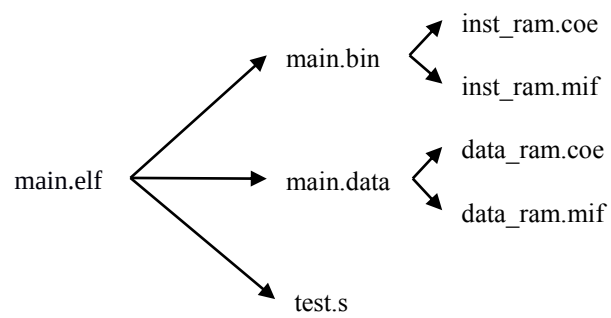


图 3-3 编译得到的文件生成关系

4 讲义四：仿真调试说明

在学习并尝试本章节前，你需要具有以下知识：

- (1) 仿真工具的使用，比如 Vivado 的 Xsim。
- (2) Verilog 的基本语法。

通过本章节的学习，你将获得：

- (1) 各类仿真错误排查的方法。
- (2) CPU 逻辑出错的调试指导。
- (3) Verilog 运算符的优先级。

看完本章节，请问自己是否知道以下知识点，如果不知道，请先找同学讨论，讨论不确定再询问老师或助教：

- (1) 何为多驱动的信号？
- (2) 阻塞赋值和非阻塞赋值的区别？其使用情况分别是？
- (3) 信号只为“Z”和“X”的区别。
- (4) Verilog 中“+”与“&”的优先级。
- (5) Verilog 中，两个 reg 型信号写在一个 always 块里和分开写在两个 always 块里，有何区别？
- (6) 请反思自己编写 Verilog 中是 C 语言编程的思想，还是画电路框图的思想。

强烈建议按照第 2 章的要求来写 Verilog 程序。

本文档在介绍的仿真调试手段是基于 Vivado2017.1 的 Xsim 仿真器的经验总结。对于其他仿真器，主体内容也是通用的，但在部分细节上有所不同，需要大家具体问题具体分析。

4.1 调试指导思想概述

调试是指在我们设计的一个系统在执行功能出现了错误时，定位出错误的原因。比如我们设计了一个 CPU，在运行一个测试程序时发现结果不对，这时就需要进行调试，以便后续进行纠正。可以看到全局上的调试原理是从结果推原因，难点就是定位错误的源头。

本文档编写时采用的调试思路是：时间上先定错，空间上再定错。一个设计在执行功能出现错误时，往往是在一个大片的时间段内该设计的电路的执行都不符合预期。“时间上先定错，空间上再定错”具体解释如下：

- (1) 时间上先定错：在出错的大片时间段里，定位出源头部分，源头部分是一个较小的时间段。
- (2) 空间上再定错：在源头时间段里，查看设计电路的控制部分和数据通路，定位是哪个信号带来的错误，或者是哪几个信号的组合带来的错误，或者是设计上哪里有疏忽带来的错误。

比如一个设计的 CPU 在执行测试程序出错了，这个程序是分很多指令的，这些指令是在时间上顺序执行的，我们首先需要找出第一个错误的指令（也就是时间上定位错误），随后在 CPU 的数据通路和控制信号里定位该指令错误的原因（也就是空间上定位错误）。

相对于空间上的定错，时间上的定错更加困难。特别是对 CPU 调试而言，更是如此，往往 80% 的精力都用于时间上定错了。不过针对本学期的大部分 CPU 设计实验，由于我们提供的验证平台采用了参考模型比较的验证方法，因此大家很容易定位执行出错的第一条指令，也就是完成时间上的定错。

时间上定错和空间上定错，是一种针对设计的整体调试的指导思想。但当我们仿真发现一个错误时，往往需要先去辨别错误时什么类型，并安装一定的方法追宗错误原因。

4.2 仿真出错分类

仿真出错情况按照波形直接观察结果可分为两类：

- (1) 波形出错：从波形图里直接观察，而不需要分析电路设计的功能，就能判断的错，比如波形中信号为“X”。
- (2) 逻辑出错：波形直接观察很正常，但其电路执行结果不符合预期，属于逻辑出错，比如加法器运行结果不对。

“波形出错”为浅层次的出错，都是很容易查找到原因的。“逻辑出错”则是深层次的出错，是真正调试难点，其具体内容也是包罗万象。

4.3 出错分类一：波形出错

波形出错，细分又可归为以下几类：

- (1) 信号为“Z”。
- (2) 信号为“X”。
- (3) 波形停止。
- (4) 越沿采样：上升沿采样到被采样数据在上升沿后的值。
- (5) 其他，波形怪异：仿真波形图显示怪异，与设计的电路功能无关的错误。

4.3.1 信号为“Z”

“Z”表示高阻，比如电路断路了就是显示为高阻，往往是由于以下两个原因之一导致的：

- (1) RTL 里声明为 reg 型的变量，从未被赋值；
- (2) 模块调用的信号未连接导致的信号悬空。

如下图举例了第（2）中情况。

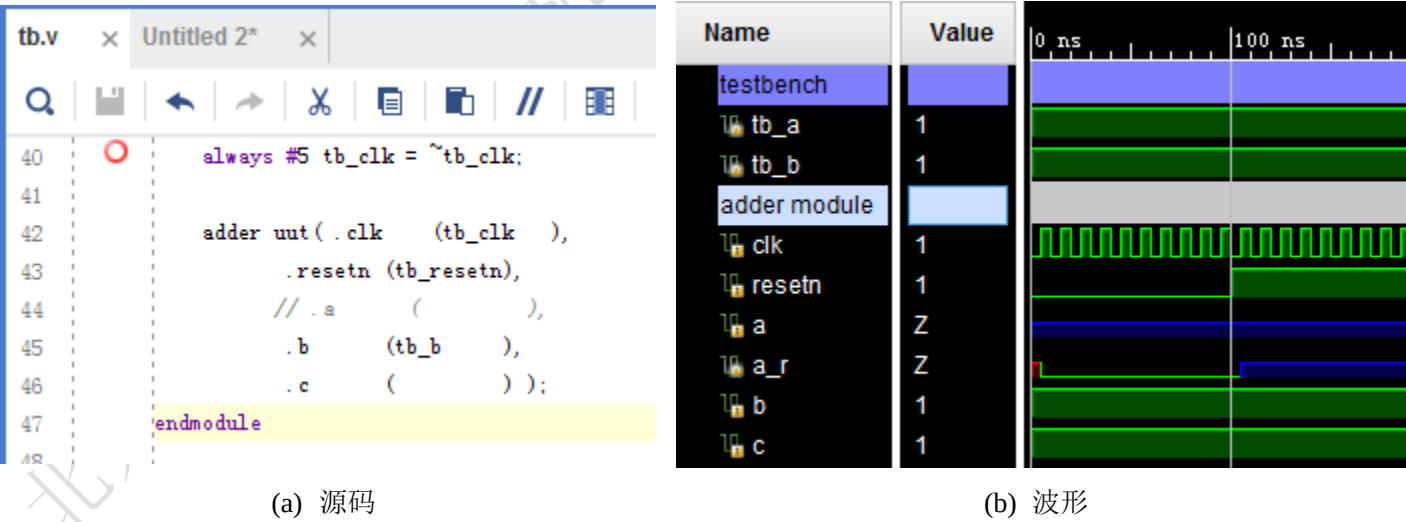


图 4-1 信号为“Z”示例

上图示例中有以下几点比较重要：

- (1) 模块调用是信号未连接，未连接包括两种：显式的未连接，如图 4-1(a)中的.c(); 隐式的未连接，如图 4-1(a)中模块 adder 调用时，a 端口即未连接。“显式的未连接”一般是人为故意设置的，只针对 output 类接口；“隐式的未连接”则是疏忽，属于代码不规范，往往也是导致信号值为“Z”的主要原因。
- (2) adder 模块里，a 端口未连接，导致 a 为“Z”，但 c 端口也未连接，c 却是固定值。这是因为 a 端口是 input，c 端口是 output。output 类接口未连接是母模块里不使用该信号，可能是人为故意设置的。所有的 input 类

接口被调用时不允许悬空。

- (3) adder 模块里 a 信号从 0 时刻开始就是“Z”，而 a_r 信号确实在 100ns 左右才变成“Z”的。这是因为 a 信号为端口，被调用时就未链接，故从 0 时刻就为“Z”，但 a_r 信号是内部寄存器，从 100ns 时刻才使用 a 信号参与赋值，所以也变成了“Z”。

针对以上几点，我们有以下几点建议：

- (1) RTL 编写时注意代码规范，特别是模块调用时，按接口顺序一一对应。
- (2) 所有 input 类接口被调用时不允许悬空。
- (3) 一旦发现一个信号为“Z”，向前追踪产生该信号的因子信号，看是哪个为“Z”，一直追踪下去直到追踪到该模块里的 input 接口，随后进行修正。
- (4) 有可能“Z”只出现在向量信号里的某几位上，也是一样的追踪，有可能调用时某个接口存在宽度不匹配也会带来该接口上某些位为“Z”。

4.3.2 信号为“X”

“X”表示不定值，往往是由于以下两个原因之一导致的：

- (1) RTL 里声明为 wire 型的变量从未被赋值；
- (2) RTL 里写成了多驱动的代码，有时候也可能导致“X”。

第（1）个情况如下图：

```
43  /*
44  always @(posedge clk)
45  begin
46      if(!resetn)
47      begin
48          b_r <= 1'b0;
49      end
50      else
51      begin
52          b_r <= b_r;
53      end
54  end*/
55  assign c = a_r + b_r;
```

(a) 源码



(b) 波形

图 4-2 信号为“X”示例

在上图中，由于 b_r 信号声明后始终未赋值，导致其值为“X”，后续 c 信号由于使用了 b_r 信号，导致其值也为“X”。

Vivado 对于多驱动（2 个及 2 个以上电路单元驱动同一信号），仿真时也会产生“X”，如图 4-3。这种情况下追寻信号为“X”的原因可能不太好追，可以尝试先进行综合，观察下 Critical warning，此时会报出多驱动警告，如图 4-4。

```

56 assign c = a_r&b_r;
57 assign c = a_r + b_r;
58 endmodule
59

```

(c) 源码



(d) 波形

图 4-3 Vivado2017.1 中多驱动引发“X”

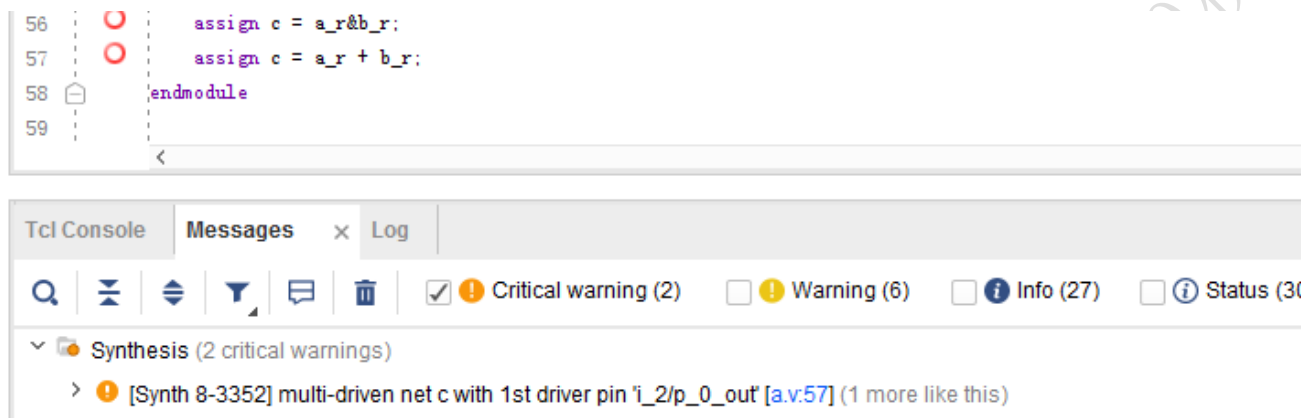


图 4-4 Vivado 中多驱动报出 Critical warning

针对信号为“X”情况，我们有以下几点建议：

- (1) 一旦发现仿真错误来自某个信号为“X”，则向前追踪产生该信号的因子信号，看是哪个为“X”，一直追踪下去直到追踪到某个信号未赋值，随后修正。
- (2) 如果因子信号都没有为“X”的，则很可能是多驱动导致的，则综合排查 Error 和 Critical warning。
- (3) 寄存器型信号如果没有复位值，在复位阶段其值可能也为“X”，但可能这并不会带来错误。
- (4) “X”和 1 进行或运算结果为 1，“X”和 0 进行或运行结果为 0。

4.3.3 波形停止

波形停止是指仿真停止某一时刻，再也无法前进分毫，而仿真却显示不停地在运行，往往是由于以下原因导致的：

- (1) RTL 里存在组合环路。

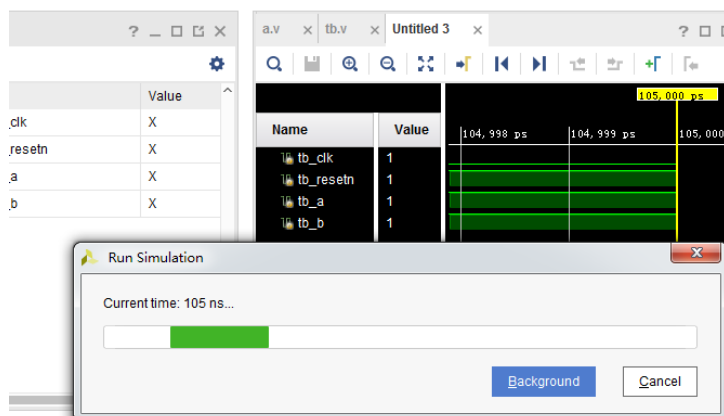
如下图：

```

wire c_t;
assign c_t = a_r&b_r+c;
assign c = a_r + c_t;

```

(e) 源码



(f) 波形

所谓组合环路就是信号 A 的组合逻辑表达式中某个产生因子为 B，而 B 的组合逻辑表达式中又用到了信号 A，如上图源码 c_t 用到了 c，而 c 又用到 c_t。仿真器是在每个周期内计算该周期的所有表达式，组合逻辑循环嵌套，带来的是仿真器的循环计算，导致其无法退出该计算，带来了波形停止的现象。

由于波形停止出现时，并不好排查哪里写出了组合环路，我们建议按以下处理：

- (1) 一旦发现波形停止，则先对设计进行综合。
- (2) 查看综合产生的 Error 和 Critical warning，并尝试修正。比如上图示例中的组合环路，经过 Vivado 的综合后变成了一个多驱动的关键警告，如图 4-5。



图 4-5 组合逻辑报出多驱动的 critical warning

另外，Vivado 工程中有 TCL 命令 `report_timing_summary`，会检查组合环路，并报出检查结果。但很遗憾，对于我们上图的示例，该命令并没有检查出组合环路，很有可能和综合时变成了多驱动有关。

4.3.4 越沿采样

越沿采样是指一个被采样的信号在上升沿采样到了其在上升沿后的值，一般情况下认为这是一个错误，往往是由于以下原因导致的：

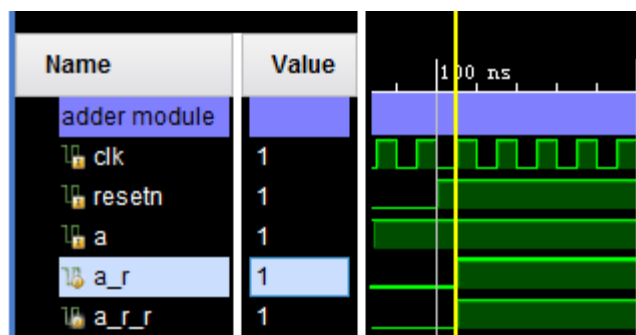
- (1) RTL 里阻塞赋值“=”和非阻塞赋值“<=”使用不当。

越沿采样在波形出错中是一个隐藏较深的出错，往往可能会和逻辑出错混在一起。初看起来，其波形也是很正常的，而且在发生越沿采样后，往往会再执行很长时间才会出错。因此需要大家先按照逻辑出错去调试，最后如果发现数据采样有些异常，就需要甄别下是否是越沿采样的错误了。

如下图：

```
always @(posedge clk)
begin
    if(!resetn)
        a_r = 1'b0;
    else
        a_r = a;
end
always @(posedge clk)
begin
    a_r_r <= a_r;
end
```

(g) 源码



(h) 波形

图 4-6 越沿采样示例

上图示例中在 105ns 时刻, clk 上升沿到来, a_r 和 a_r_r 同时变为了 1 (也就是 a 的值)。a_r 在 105 时刻前是 0, 在 105 时刻后是 1。从源码来看, a_r_r 是在上升沿采样 a_r 的值, 结果其在 105 时刻采样到 a_r 为 1 的值, 也就是采样到了 a_r 在同一上升沿后的值。这就属于越沿采样。

造成这一现象更深层的原因是 Verilog 里阻塞赋值“=”和非阻塞赋值“<=”混用。上图源码中 a_r 采用阻塞赋值, 而 a_r_r 采用非阻塞赋值。

每一次赋值, 分为两步: 为计算等式左侧的表达式和赋值给右侧的信号, 简记为计算和赋值。在一个上升沿到来时, 所有由上升沿驱动的信号按以下顺序进行处理:

- (1) 先处理阻塞赋值, 先完成计算和赋值, 同一信号完成计算后立马完成赋值。同一 always 块里的阻塞赋值从上到下按顺序串行执行, 不同 always 块里的阻塞赋值依赖工具实现确定顺序串行执行, 一一完成计算和赋值。
- (2) 再进行非阻塞赋值的计算。所有非阻塞赋值其等式左侧的值都同时计算好。
- (3) 上升沿结束时, 所有非阻塞赋值同时完成最终的赋值动作。

从以上描述可以看到, 非阻塞赋值是在上升沿的最后一个时间步里完成处理的, 晚于阻塞赋值的处理。所以上图示例中, a_r_r 的赋值晚于 a_r 的赋值, 造成了越沿采样的情况。

越沿采样, 除非特意设计, 一般我们认为是一个设计错误, 针对越沿采样, 我们有以下几点建议:

- (1) RTL 编写时注意代码规范, 所有 always 写的时序逻辑只允许采用非阻塞赋值。
- (2) 一旦发现越沿采样的情况, 追踪被采样信号, 直到追踪到某一个阻塞赋值的信号, 随后进行修正。

4.3.5 波形怪异

本文档将目前未能想到的波形出错的类型都归为波形怪异。

当出现波形怪异类的错时, 需要区分其是仿真工具出错还是 RTL 代码出错:

- (1) 观察出错的信号, 看其生成原因, 如果自我判断 RTL 应该没有, 且波形显示确实太怪异 (比如始终为 32'hxx?x0x?), 则很有可能仿真工具出错。重启电脑甚至重建工程试试。
- (2) 实在无法从波形里区分出是什么错。可以尝试先运行综合, 看出综合后的 Error、Critical warning 和 warning。其中 Error 是必须要修正的, Critical warning 是强烈建议要修正的, warning 是建议能修则修的。
- (3) 经常有些不符合规范的代码, Vivado 也不会报出 Warning, 需要大家仔细复核自己的代码。

4.4 出错分类二: 逻辑出错

逻辑出错则是包罗万象, 错误类型是设计的电路功能有错, 此时波形界面看起来是很正常的, 我们需要利用波形观察各信号的变化, 结合预定的电路功能进行定错。

以数据和控制分开来看, 逻辑出错可分为两类: 数据通路出错和控制信号出错。其中数据通路通常属于较简单的错, 比如加法器算两个加数的和, 结果不对; 而控制信号出错则往往比较难调, 往往是设计时的边角问题考虑不周导致的, 比如 CPU 的访存系统出错。这些都是逻辑上出错了, 但是很不幸的是, 在我们未能定位出该错误的源头时, 我们往往不能判断出其是数据通路出错, 还是控制信号出错。电路设计者在设计之初应当对整个电路有较全面的认识和考虑, 尽量减少控制信号出错的情况。

逻辑出错时, 不同的电路设计有其特定的调试手段, 难以总结出统一的调试手段, 但他们的指导思想是一致的: 时间上先定错, 空间上再定错。

以下我们将针对 CPU 的逻辑出错调试作简单的说明, 主要以流水线 CPU 为例进行说明。

4.4.1 CPU 逻辑出错调试

CPU 逻辑出错调试时, 首先需要在波形窗口里抓取必要的信息: 指令的 PC 值和机器指令编码、指令的写回结

果、顶层的访存接口、指令执行的数据通路上的信息。

对于流水线 CPU，需要各流水级信号分组抓出，比如抓出每级流水里的 PC 值、指令编码和执行结果。流水级间的进入和退出的控制信号也尤其重要，必须抓出，CPU 初期调试往往都是流水线控制出错了。还有顶层总线接口也很重要，一旦取指出错了，或者调试访存指令出错了，就需要关注顶层总线接口上的信号了，CPU 后期调试往往都是访存系统出错了。

CPU 逻辑出错调试同样按“时间上先定错，空间上再定错”进行。

(1) 定位出错时间源头

此时需要结合执行的测试程序和 CPU 设计进行定位。定位错误前，需要大家做好以下三项准备：

首先，需要先理解测试程序的大致行为。理解过程中需要阅读测试程序的源码，本学期 CPU 设计实验中会运行的程序有两类：功能测试程序 func 和性能测试程序 coremark、dhrystone。功能 func 行为很简单，所有编号的功能点依次测试，每两个功能点测试之间插入 wait_1s 函数。性能测试程序行为则偏复杂，可以不用弄懂，但在调试过程中如有需要应当能很快找到函数间的调用关系。

再者，需要对 mips 汇编和 func 编译环境有一定了解。我们在功能和性能测试程序的编译环境中会生成如下 8 个文件，位于编译目录的 obj/下。其中 test.s 是我们调试过程中重点关注的，它对执行程序的每条机器指令进行了反汇编和注释，对我们的调试很有帮助。

表 4-1 编译生成文件

文件名	解释
data_ram.coe	重新定制 data ram 所需的 coe 文件
data_ram.mif	仿真时 data ram 读取的 mif 文件
inst_ram.coe	重新定制 inst ram 所需的 coe 文件
inst_ram.mif	仿真时 inst ram 读取的 mif 文件
main.bin	编译后的代码段，可以不用关注
main.data	编译后的数据段，可以不用关注
main.elf	编译后的 elf 文件
test.s	对 main.elf 反汇编得到的文件

最后，需要对 CPU 内部逻辑比较熟悉。因为是大家自实现的 CPU，相信大家应该比较熟悉。在定位时间源头时，需要明确 CPU 正在执行的指令的 PC 值、编码和写回结果，将这些信息抓取到波形窗口中，进行对比。

具体调试时，可以采用以下方案：

- (1) 在波形最后出错处，确认取会的指令和 PC 值是对应正确的，也就是确认取指正确，这时就需要对照反汇编程序 test.s。
- (2) 如果取指不正确，则往前追溯，直到第一个取指正确的地方。追溯的方法也有讲究，有时不能简单一条条指令追溯，因为第一个取指正确的地方可能在很早之前，必要的时候，应跨越一大段指令段，去确认取指是否正确。追溯过程就是程序段不停的压缩，直到找到第一个取指正确的地方，此时往往要用到仿真工具中加标签的方法。
- (3) 找到第一条指令正确的地方后，可以先确认该指令执行结果是否正确。随后我们的调试目标是确认时间上第一个出错的地方是在该指令前还是在该指令后。确认方法就需要结合测试程序，比如判断该指令位于的函数，确认该函数是否应该执行，其进入条件是否正确。这里有很多种调试方法，需要根据具体情况具体分析，无法很好的总结分类，需要大家在实践中进行总结。总而言之，需要将测试程序代码和 CPU 结合起来联调。

以上方法是由后往前追，如果追溯过程中发现无法再追了，则可以考虑由前往后追。这里的“前”就需要大家好好定位了，甚至可能存在运气的成分，一定要确保这个“前”之前的程序执行时对，这样往后追才能追到正确的第一个错误的店，否则，只会将自己引向错误的方法。

时间上的定错，要求大家对测试程序有一定的了解和掌握，具体调试方法需要大家多多在实践中总结。

值得一提的是，我们的 CPU 实验开发环境 cpu_CDE 仿真时一旦出错，验证平台会很快停止，并打印对比出错

的信号，大家最多往前追溯一会就会找到第一条出错的指令。

(2) 定位出错空间源头

在完成时间上的定错后，也就是找到一个执行出错的指令后，需要进行空间定错了。

空间定错时需要大家对 CPU 微结构有更深入的理解。建议大家以空间划分的视角去理解 CPU，特别是流水线 CPU，每一流水级都是有对应的部件的，应当理解清楚各流水级的划分。

空间定位时，有两种方法：

- (1) 从 CPU 流水前端向流水后端排查，确认指令在哪个流水级开始出错。首先要排查的就是取指是否正确。
- (2) 从 CPU 流水后端想流水前端排查，确认指令是从哪个流水级出错的。首先要排查的就是写回结果是否正确。

4.5 总结

大家在仿真过程中碰到错了可以按需索引本文档，调试过程中记住“时间上先定错，空间上再定错”，不要手忙脚乱，各种信号乱抓，要注意理清调试思路。

设计初期，应当具有全局思路，尽量考虑周到，特别是边角情况，这样可以大大减少后期调试的工作量。当然也不可钻牛角尖，深陷设计考虑中而迟迟不肯动笔，很多设计往往是写着写着就清晰了。

编程过程中，应当注意编码规范，仿真、综合实现中很多怪异现象，以及“仿真通过，上板不过”等很多都是由于代码不规范导致的，强烈建议按照文档“LEC02_Verilog 复习.pdf”中的第 1.5 节来编程。

最后，给大家强调下 Verilog 的运算符的优先级，见图 4-7。其中“+”和“-”的优先级是很高的，需要大家留意，比如“assign res[31:0] = a[31:0] + b[0]&& c[0];”，表达式右侧运行结果是一个 1bit 的结果，而不是我们认为的 32bit 的结果，上述语句等效于“assign res[31:0] = (a[31:0] + b[0]) && c[0];”。

优 先 级 别	
! ~ * / % + - << >> < <= > >= == != === !== & ^ ^~ && ?:	高 优 先 级 别 ↓ 低 优 先 级 别

图 4-7 Verilog 运算符优先级

二、任务书

实验：静态 5 级流水 MIPS CPU 实现

在学习并尝试本章节前，你需要具有以下环境和能力：

- (1) 装有 Vivado 的电脑一台。
- (2) 较为熟练使用 Vivado 工具。
- (3) 熟悉龙芯体系结构实验箱（Artix-7）。

通过本章节的学习，你将获得：

- (1) CPU 设计中流水线切分的原理和实现。
- (2) CPU 设计中三类相关（数据、控制和结构）的产生原因、影响和处理。
- (3) 一个小型较完备指令集的理解。
- (4) 嵌入式处理器中功能测试程序和原理。

在本章节的学习过程中，你可能需要查阅：

- (1) 附录三 CPU 实验开发环境使用说明。
- (2) 附录四 MIPS 指令系统规范。
- (3) 附录六 交叉编译工具链安装。
- (4) 附录九仿真调试说明。

4.6 实验目的

1. 进一步熟悉实验平台。
2. 熟悉并运用 verilog 语言进行电路设计。
3. 掌握 CPU 的仿真测试和 FPGA 测试方法。

4.7 实验设备

1. 装有 Xilinx Vivado 的计算机一台。
2. 龙芯体系结构教学实验箱（Artix-7）一套。

4.8 实验任务

1. 根据指导书下了解静态 5 级流水简单 TinyMIPS CPU 的实现过程。
2. 注意：
 - (1) CPU 复位从虚拟地址 0xbfc00000 处取指。
 - (2) CPU 虚实地址转换采用：虚即是实。
 - (3) CPU 对外访存接口为取指、数据访问分开的同步 SRAM 接口。
 - (4) CPU 只实现一个操作模式：核心模式，不要求实现其他操作模式。
 - (5) 不支持例外和中断。

(6) CPU 顶层连出写回级的 debug 信号，以供验证平台使用。

4.9 实验环境

本次实验实验环境为 cpu_CDE。实验环境介绍请参考 CPU 实验开发环境使用说明。

cpu_CDE 中 func 放置在 soft 目录下。

北京科技大学计算机组成原理课程设计19-20秋季

4.10 实验说明

4.10.1 CPU 实现指令集

本次实验四个阶段，总共要求实现 MIPS I 指令集中的 22 条指令。

关于指令的详细定义可以参考文档“A02_MIPS 指令系统规范”。

4.10.2 CPU 顶层接口

本实验要求 CPU 核心部分（Core）按以下接口封装。

表 0-1 CPU 核心部分顶层接口信号描述

名称	宽度	方向	描述
时钟、复位和暂停			
clk	1	input	时钟信号，来自 clk_pll 的输出时钟
rst	1	input	复位信号，高电平同步复位
stall	1	input	暂停信号，高电平有效
取指端访存接口（ROM 控制信号）			
rom_en	1	output	ROM 使能信号，高电平有效
rom_write_en	4	output	ROM 字节写使能信号，高电平有效
rom_addr	32	output	ROM 读写地址，字节寻址
rom_read_data	32	input	ROM 读数据
rom_write_data	32	output	ROM 写数据
数据端访存接口（RAM 控制信号）			
ram_en	1	output	RAM 使能信号，高电平有效
ram_write_en	4	output	RAM 字节写使能信号，高电平有效
ram_addr	32	output	RAM 读写地址，字节寻址
ram_read_data	32	input	RAM 读数据
ram_write_data	32	output	RAM 写数据
debug 信号，供验证平台使用			
debug_pc_addr	32	output	写回级（流水线最后一级）的 PC，需要在核心里将 PC 一路带到写回级
debug_reg_write_en	1	output	写回级写寄存器堆(RegFile)的写使能，为字写使能。
debug_reg_write_addr	5	output	写回级写 RegFile 的目的寄存器号
debug_reg_write_data	32	output	写回级写 RegFile 的写数据

4.10.3 CPU 复位 PC

要求 CPU 复位从 0xbfc00000 处取指（MIPS 架构规范要求如此，以后解释）。由于本实验 ROM 和 RAM 地址宽度有限，并没有使用到 32 位地址的高 12 位，因此 CPU 复位 PC 为 0xbfc00000 和为 0x00000000 效果一样，但我们还是要求尽量符合规范。

4.10.4 CPU 虚实转换

MIPS 常用虚实转换机制包括：固定地址映射（FMT）和 TLB 机制。但本次实验均不要求大家自行实现这两种机制，我们默认在 CPU 核心内，虚地址即是实地址。实验环境内为大家封装了完整的 CPU 顶层，在该顶层内我们已经实现了 FMT 机制。

4.10.5 CPU 延迟槽实现要求

本次实验要求实现延迟槽技术，延迟槽里可存放任何指令。（请思考延迟槽里放分支指令会如何？可以查阅 MIPS 官方手册。）

4.10.6 CPU 操作模式

MIPS 架构 CPU 操作模式有核心模式、监管模式和用户模式。但本次实验只要求实现核心模式，即认为程序用于运行于核心模式。

4.10.7 CPU 不要求实现项

本次实验不要求实现中断和例外，不要求实现任何系统控制寄存器。

4.10.8 CPU 验证步骤

参考 lab1 和开发环境的使用。

4.10.9 CPU 验证结果

本次实验验证结果的检查类似 lab1，详细请阅读 lab1 的文档或者讲义三。

(1) 仿真验证结果

仿真结果正确判断有两种方法。

第一种方法，也是最简单的，就是看 Vivado 控制台（Tcl Console）打印 Error 还是 PASS。正确的控制台打印信息如下图。

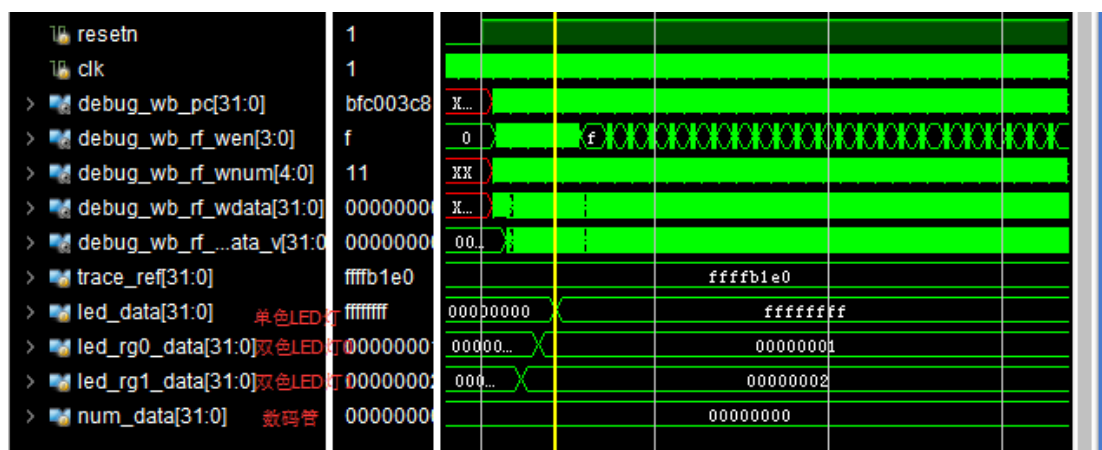
```
[2112000 ns] Test is running, debug_wb_pc = 0xbfc17530
[2122000 ns] Test is running, debug_wb_pc = 0xbfc17d78
[2132000 ns] Test is running, debug_wb_pc = 0xbfc185e0
[2142000 ns] Test is running, debug_wb_pc = 0xbfc18e08
[2152000 ns] Test is running, debug_wb_pc = 0xbfc19634
[2162000 ns] Test is running, debug_wb_pc = 0xbfc19e8c
[2172000 ns] Test is running, debug_wb_pc = 0xbfc1a6d4
[2182000 ns] Test is running, debug_wb_pc = 0xbfc1af1c
----[2184405 ns] Number 8' d15 Functional Test Point PASS!!!
----[2189465 ns] Number 8' d16 Functional Test Point PASS!!!
[2192000 ns] Test is running, debug_wb_pc = 0xbfc105fc
----[2194525 ns] Number 8' d17 Functional Test Point PASS!!!
----[2197365 ns] Number 8' d18 Functional Test Point PASS!!!
----[2200405 ns] Number 8' d19 Functional Test Point PASS!!!
[2202000 ns] Test is running, debug_wb_pc = 0xbfc1b1d4
----[2203645 ns] Number 8' d20 Functional Test Point PASS!!!
=====
Test end!
----PASS!!!
$finish called at time : 2205425 ns : File "E:/loongson/arch_ucas/17-18/lab3/ucas_CDE_v0.2/mycpu_v
run: Time (s): cpu = 00:00:31 ; elapsed = 00:02:02 . Memory (MB): peak = 1217.645 ; gain = 0.000
```

图 0-1 测试通过的控制台打印信息

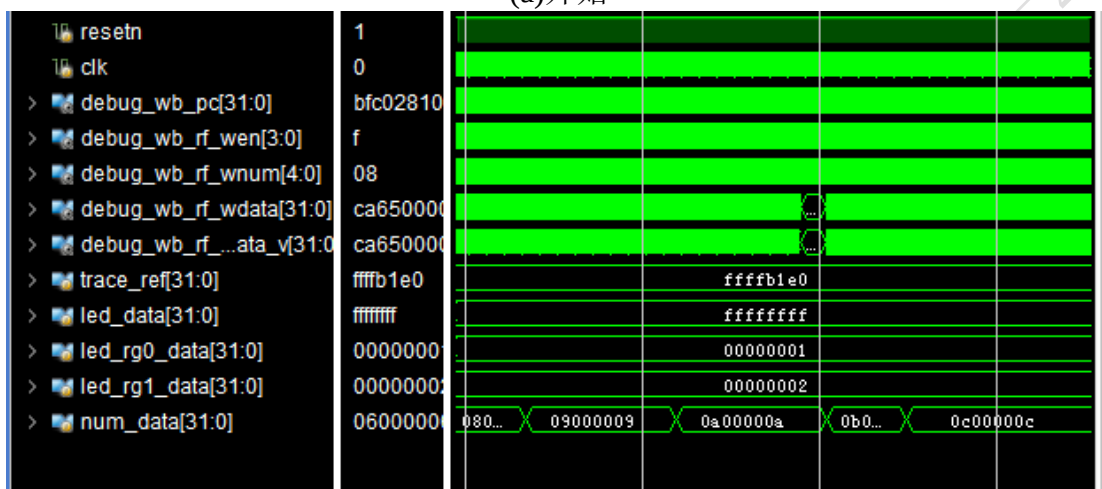
第二种方法，是通过波形窗口观察程序执行结果 func 正确的执行行为，抓取 confreg 模块的信号 led_data、led_rg0_data、led_rg1_data、num_data:

- (1) 开始，单色 LED 写全 1 表示全灭，双色 LED 写 0x1 和 0x2 表示一红一绿，数码写全 0；
- (2) 执行过程中，单色 LED 全灭，双色 LED 灯一红一绿，数码管高 8 位和低 8 位同步累加；
- (3) 结束时，单色 LED 写全 1 表示全灭，双色 LED 均写 0x1 表示亮两绿，数码管高 8 位和低 8 位数值相同，

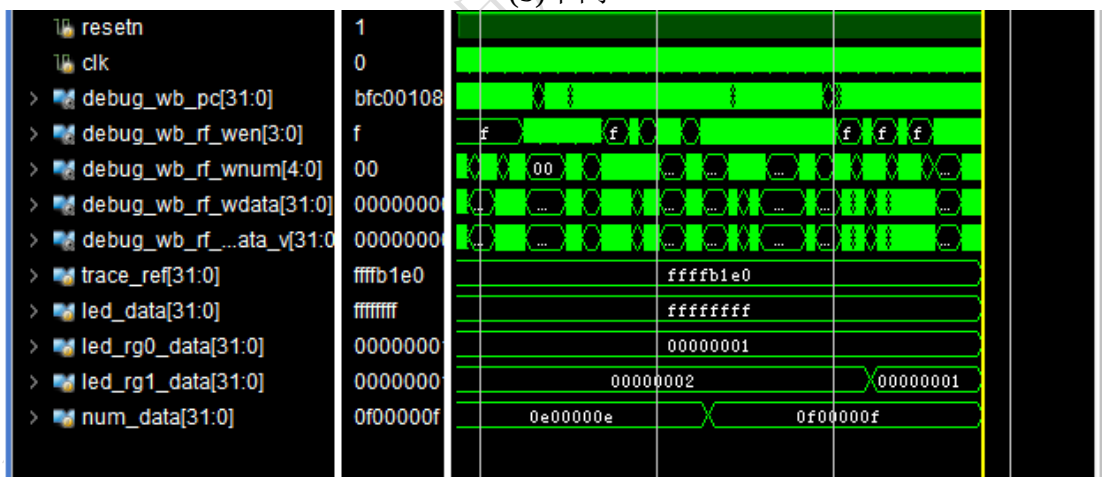
对应测试功能点数目，lab1 中为 0x13，lab2 中各阶段测试功能点数目不同。



(a)开始



(b)中间



(c)结束

图 0-2 正确的仿真波形图

(2) FPGA 验证结果

在 FPGA 上验证时其结果正确与否的判断也只有一种方法，func 正确的执行行为是：

- (1) 开始，单色 LED 全灭，双色 LED 灯一红一绿，数码管显示全 0；
- (2) 执行过程中，单色 LED 全灭，双色 LED 灯一红一绿，数码管高 8 位和低 8 位同步累加；
- (3) 结束时，单色 LED 全灭，双色 LED 灯亮两绿，数码管高 8 位和低 8 位数值相同，对应测试功能点数目。

如果 func 执行过程中出错了，则数码管高 8 位和低 8 位第一次不同处即为测试出错的功能点编号，且最后的结果是单色 LED 全亮，双色 LED 灯亮两红，数码管高 8 位和低 8 位数值不同。

最后 FPGA 验证通过的效果图，类似下图，数码管高 8 位和低 8 位显示为运行的功能点数目。

如果大家想看动态的 FPGA 运行过程，可以对 cpu132_gettrace 下的 Vivado 工程进行综合实现并上板运行，其将显示正确的 FPGA 验证运行过程。

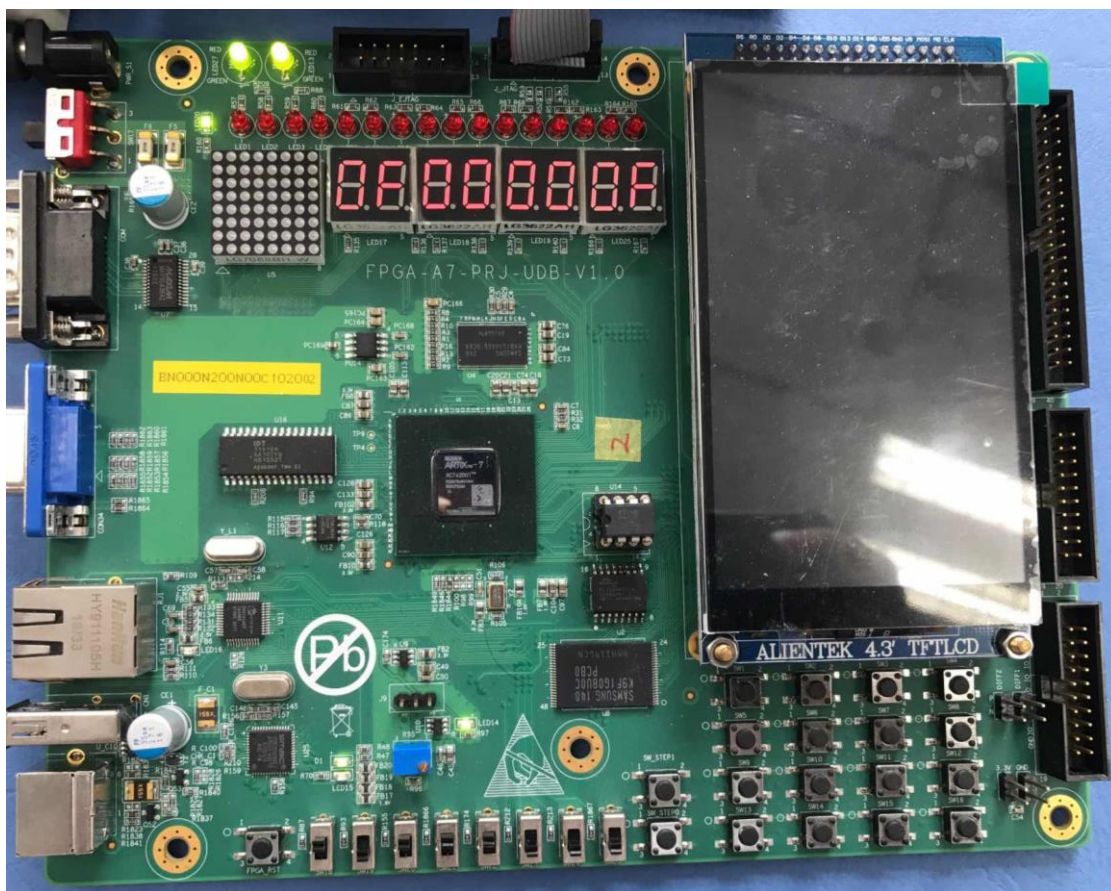


图 0-3 FPGA 验证正确效果图

三、附录

5 附录一：龙芯体系结构教学实验箱（Artix-7）介绍

在学习并尝试本章节前，你需要具有以下环境：

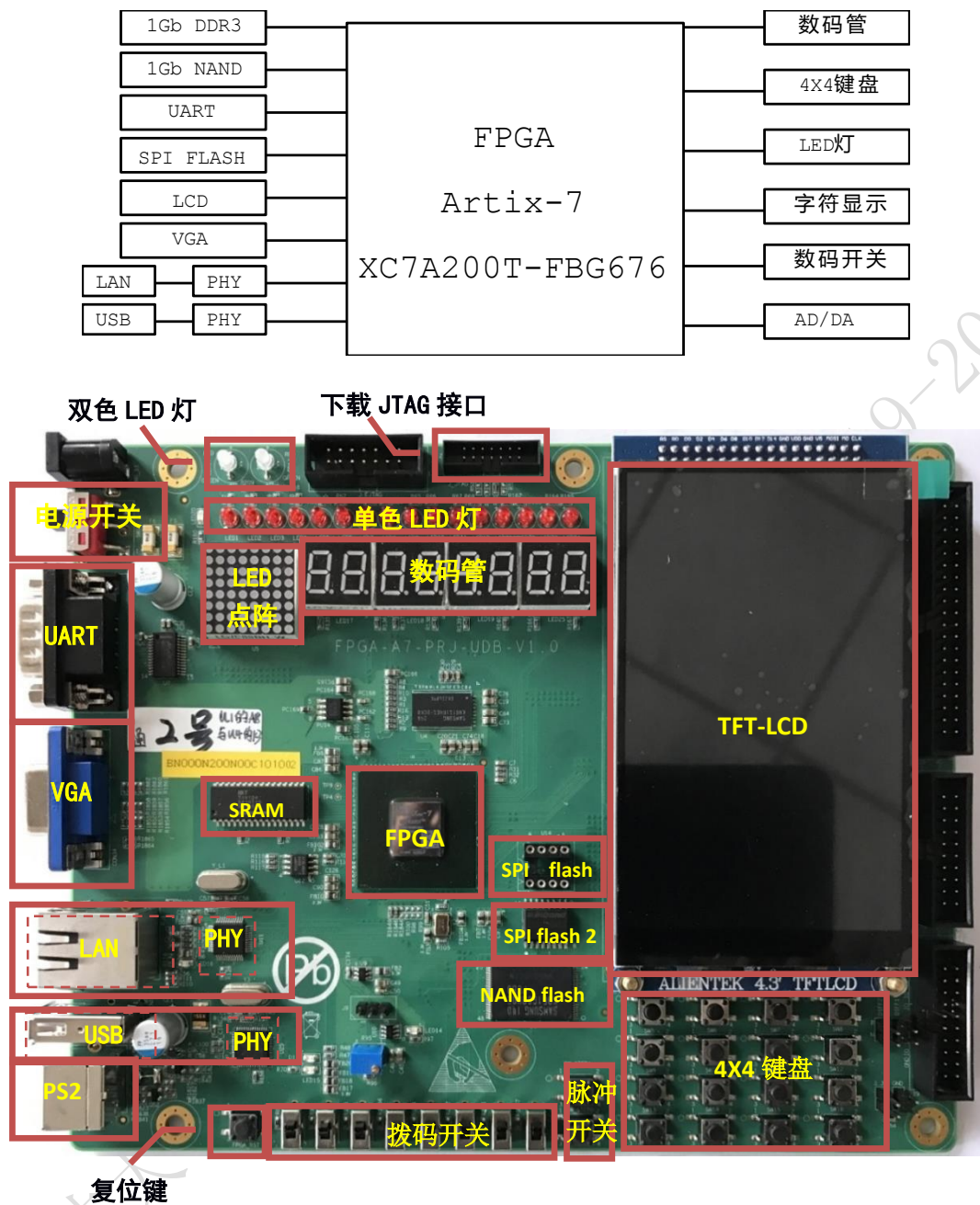
- (1) 龙芯体系结构教学实验箱（Artix-7）一套。

通过本章节的学习，你将获得：

- (1) 初步了解龙芯体系结构教学实验箱（Artix-7）

龙芯体系结构教学实验箱的核心是一块基于 FPGA 芯片的嵌入式系统开发板（以下简称“开发板”），箱内的其它器件为：开发板配套的电源适配器 1 个、JTAG 下载线和适配器 1 套以及串口线 1 根。下面简要介绍开发板的硬件设计方案和时钟设计方案。如果想详细了解开发板，请参看提供的开发板原理图。

5.1 开发板硬件设计方案



上面的两幅图分别给出了开发板的逻辑结构示意图和实景图，可以对照着看以加强感性认识。开发板的主要部分的设计方案列举如下：

功能模块	设计方案概述
FPGA	选用 Artix-7 XC7A200T-FBG676
DDR3	使用 FPGA 实现 DDR3 控制器，板载 K4B1G1646G-BCK0 DDR3 颗粒
SRAM	使用 FPGA 实现 SRAM 控制器，板载 IDT71V124SATY SRAM 芯片
NAND	使用 FPGA 实现 NAND 控制器，板载 K9F1G08U0C-PCB0 闪存颗粒
SPI Flash 1	使用 FPGA 实现 SPI 控制器（支持启动），板载 Flash 芯片插座，Flash 可插拔
SPI Flash 2	板载不可插拔 Flash 芯片，FPGA 设计固化专用
VGA	使用 FPGA 实现数字显示模块，板载 MM74HC573SJ 实现 332 的数模转换，模拟 VGA 的 R、G、B 信号
LCD	使用 FPGA 实现 LCD 显式控制器，板载 TFT-LCD 屏
USB	使用 FPGA 实现 USB 控制器，板载 USB PHY（USB3500），对外提供一个 USB 接口

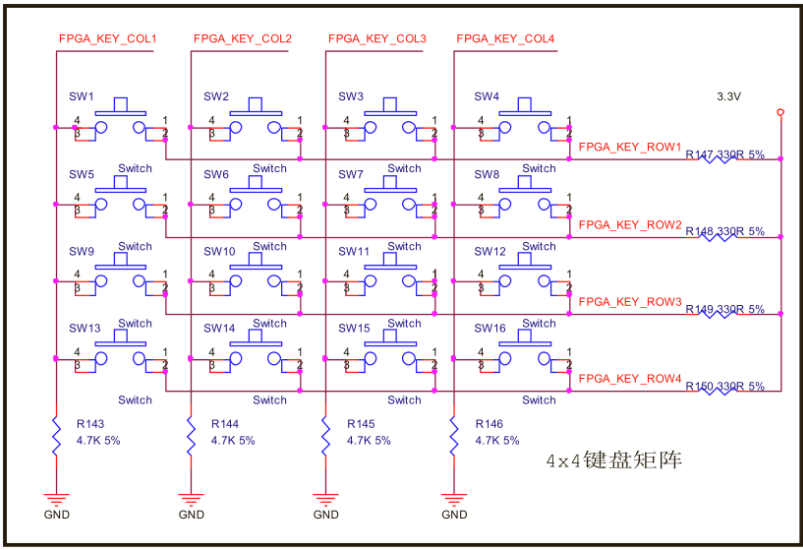
功能模块	设计方案概述
LAN	使用 FPGA 实现 MAC 控制器，板载以太网 PHY（DM9161AEP），对外提供一个 RJ45 网络接口
PS2	使用 FPGA 实现 PS2 控制器，板载 PS2 接口
UART	使用 FPGA 实现 UART 控制器，板载 UART 接口
GPIO	16 个 LED 单色灯； 2 个 LED 双色灯； 8×8LED 点阵（可实现字符显示功能）； 8 个共阴极八段数码管（用于数字显示）； 其余外接通用 I/O 接口。

5.2 开发板原理图

当使用开发板展示实验时，也就是使用 Vivado 工具进行电路实现时，需要将设计的电路顶层的 input/output 接口信号绑定到开发板上 FPGA 芯片的 IO 引脚上，该绑定关系由 Vivado 工程中的约束文件(*.xdc)指定。

因此在使用 Vivado 工具进行电路实现时，需要编写约束文件，此时就需要查找开发板原理图以确定引脚编号。开发板原理图参见原理图目录的“龙芯体系结构实验箱-原理图.pdf”。我们也整理了开发板上常用的 IO 设备的引脚列表，如 LED 灯、数码管等，参见“引脚对应关系.xlsx”。

比如，假设电路实现中使用到了 4X4 矩阵键盘，现需要确定矩阵键盘的接口。我们可以直接从“引脚对应关系.xlsx”中获得矩阵键盘引脚对应的 FPGA 芯片的 IO 编号，也可以查看原理图得到对应的编号。4X4 矩阵键盘原理图如下：



可以看到，4X4 键盘矩阵只用了 8 个引脚：FPGA_KEY_COL_1~FPGA_KEY_COL_4，FPGA_KEY_ROW_1~FPGA_KEY_ROW_4。其中列 FPGA_KEY_COL* 通过一个高电阻接地，行 FPGA_KEY_ROW* 通过一个相对低的电阻接高电平。当有开关闭合时，闭合处对应的列 FPGA_KEY_COL* 和行 FPGA_KEY_ROW* 有相同的电平。

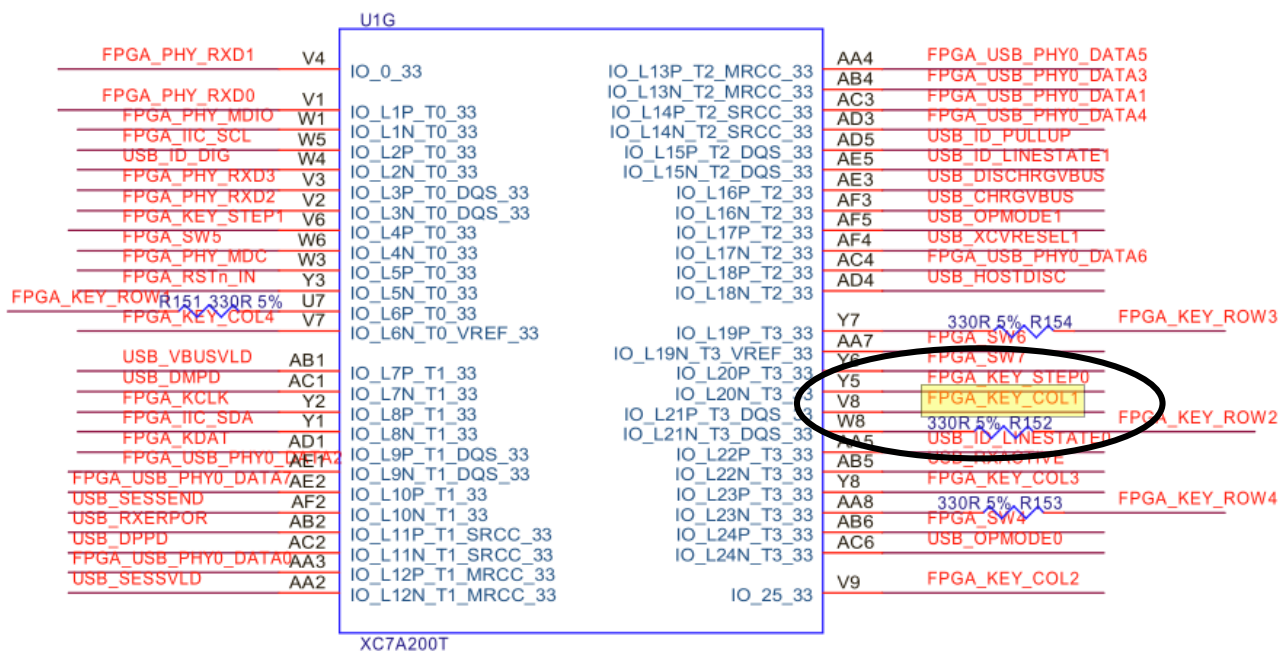
如果只需要使用同一行的按键时，由于列 FPGA_KEY_ROW* 是默认接到高电平的，故当有按键时，相应的行 FPGA_KEY_COL* 会收到一个高电平，即得到一个“1”，而无按键时会得到一个“0”。需要注意，对按键检测时最好加上防抖。

当需要使用多行的按键时，则需要通过扫描方式确认按键位置了。扫描方式是从列 FPGA_KEY_COL* 一次输入低电平“0”，随后检测行 FPGA_KEY_ROW* 处得到的电平值。如果四行均为高电平“1”，则表示按键不在该列；如果有一行为低电平“0”，表示按键在该列，且在该行。

假如在电路实现时，我们使用了键盘左上角的按键，则需要在原理图中查找行 FPGA_KEY_COL_1，可以看

到该列线是连接到 FPGA 芯片中编号为 V8 的 I/O 引脚上(图中引脚名有些许错位), 如下图。

类似可以查找 FPGA_KEY_COL_2~FPGA_KEY_COL_4 和 FPGA_KEY_ROW_1~FPGA_KEY_ROW_4 对应的 FPGA 芯片的 I/O 引脚编号。



6 附录二：龙芯体系结构教学实验箱（Artix-7）-原理图与引脚列表

参考本目录下 Appendix 目录下的 A02 目录里 pdf 文件和 Excel 文件。

北京科技大学计算机组成原理课程设计19-20秋季

7 附录三：Vivado 使用说明

本章是以 vivado2017.1 为例举例说明的，对其他版本 vivado 的安装也同样适用。

在学习并尝试本章节前，你需要具有以下环境和能力：

- (1) 装有 Vivado 的电脑一台。

如果电脑尚未安装 Vivado，请参考文档“A03_Vivado 安装说明”。

- (2) 龙芯体系结构教学实验箱（Artix-7）一套。

实验箱介绍，请参考文档“A01_龙芯体系结构教学实验箱（Artix-7）介绍”。

- (3) 一定的规范的 Verilog 编程能力。

请自学 Verilog，可以参考文档“LEC02_Verilog 复习”。

通过本章节的学习，你将获得：

- (1) 初步熟悉龙芯 Artix-7 实验箱。
- (2) 基本的 Vivado 使用方法。
- (3) 初步了解 Vivado 的 XDC 约束文件。

Vivado 的开发模式有两种：Project Mode 和 Non-Project Mode。两者的主要区别是，Project Mode 使用 Flow Navigation 更加自动化，Non-Project Mode 使用 Tcl 命令直接控制操作。实际中根据开发需求选择这两种模式，一般简单设计中采用 Project Mode。

调用 Vivado 的方法也有两种：图形界面和 TCL shell 界面。Project Mode 同时支持两种调用方法，但是 Non-Project Mode 只支持 TCL shell 界面。

本课程大部分实验应用的是 Vivado 基于图形界面的 Project Mode 开发模式。

下面以 led 实验为例介绍如何使用 Vivado 新建一个工程。led 实验实现开发板上 8 个拨码开关控制 8 个 led 灯亮灭。

7.1 数字电路设计的一般流程

在介绍 Vivado 工具的使用前，先简单介绍下数字电路设计的一般流程，如下图（图片来自网络）。图中的最后步骤“Sign off”为物理签核，就是得到最终的电路版图。将电路版图交付晶元代工厂，就可以生产芯片了。

在图 7-1 中用红色标出了 FPGA 开发的流程，FPGA 开发的流程和一般数字电路设计的流程主体部分是对应的。但在最后步骤时，FPGA 开发不是生成电路版图，而是生成 bit 流文件，并下载到 FPGA 开发板上进行运行。

本章的 7.4 节涉及图 7-1 中的第 2、3 步，7.6 节涉及图 7-1 中的第 4、5、6、7、8 步。

7.5 节“添加约束文件”则是综合与布局布线之间的步骤。在图 7-1 中的“Static Timing Analysis”（静态时序分析，简称为 STA）也需要添加约束文件，只是一般数字电路设计中的约束文件只是时序、cell 单元、布局上的约束。而 FPGA 的设计约束除了这些外，还起到引脚绑定的作用。在本学期的实验中，约束文件更多的作用就只是引脚绑定，具体请详细阅读本章 7.5 节。

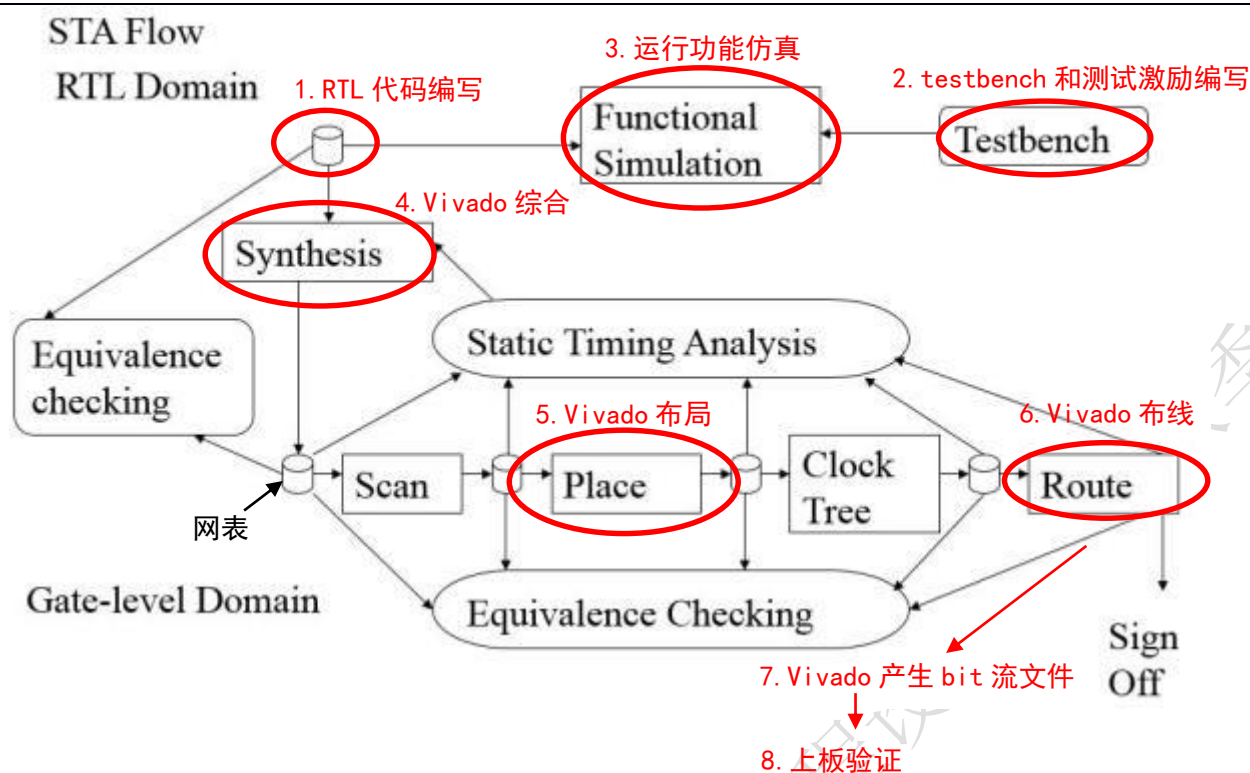


图 7-1 数字电路设计与 FPGA 开发对照图

7.2 Vivado 新建工程

通过双击桌面快捷方式或开始菜单的“Xilinx Design Tools→Vivado 2017.1”打开 Vivado 2017.1，在界面上“Quick Start”下选择“Create Project”。

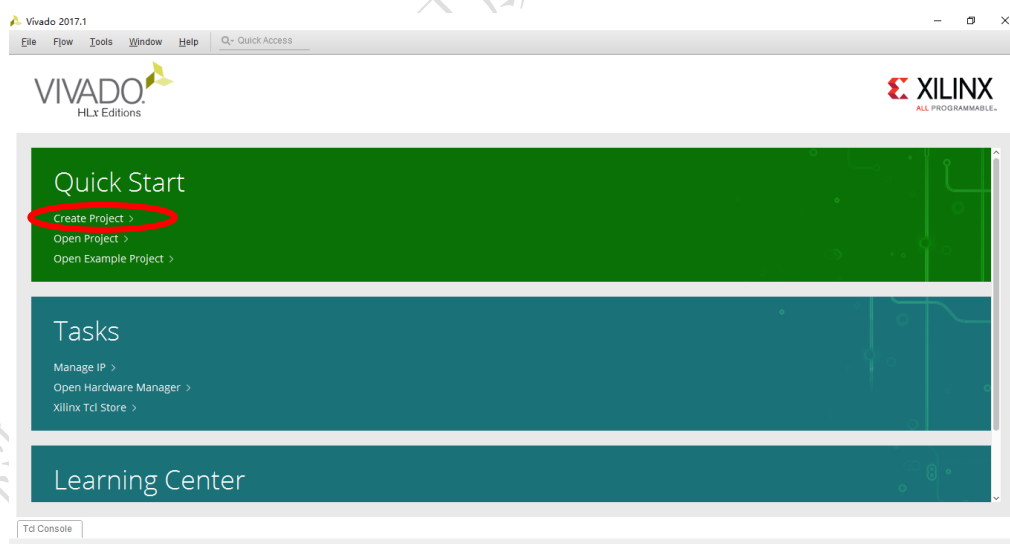


图 7-2 Vivado 启动界面

新建工程向导，点击“Next”。

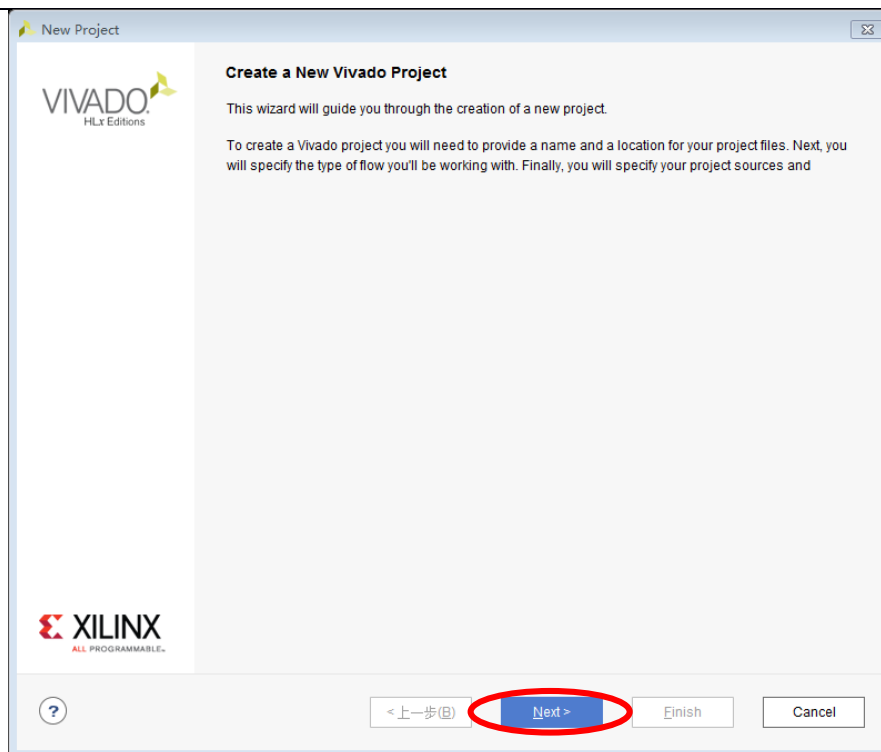


图 7-3 新建工程向导

输入工程名称并选择工程的文件位置，并勾选“Create project subdirectory”选项，为工程在指定存储路径下建立独立的文件夹。设置完成后，点击“Next”。

注意：工程名称和存储路径中不能出现中文和空格，建议工程名称以字母、数字、下划线来组成。

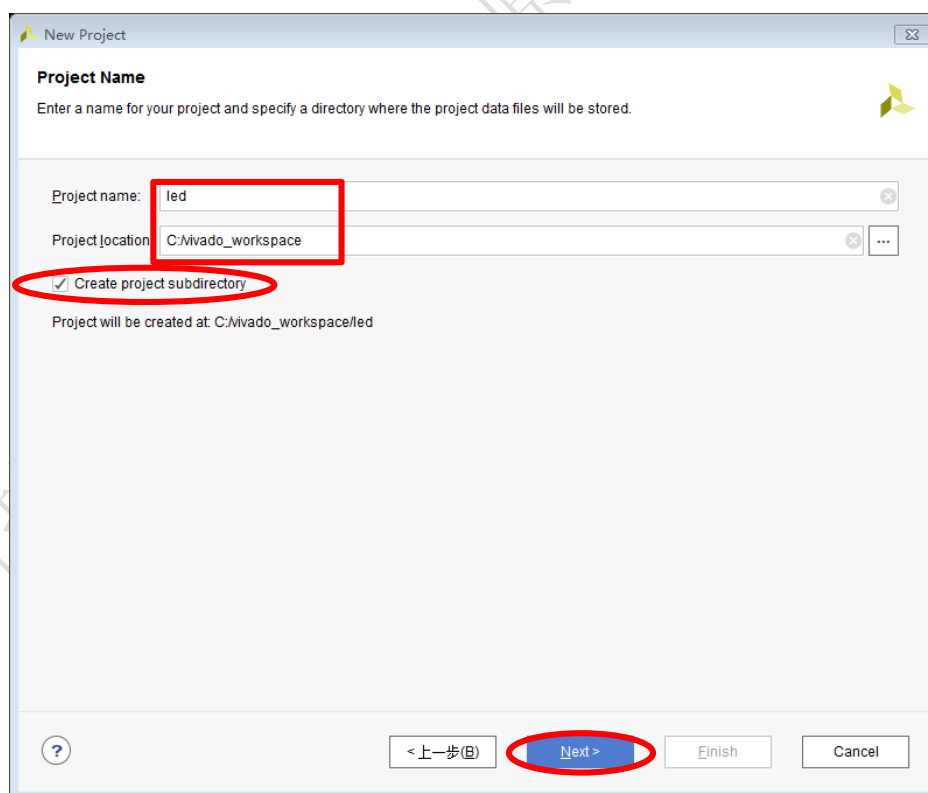


图 7-4 设置工程名称和位置

选择“RTL Project”一项，并勾选“Do not specify sources at this time”，勾选该选项是为了跳过在新建工程的过程中添加设计源文件，如果要在新建工程时添加源文件则不勾选。点击“Next”。

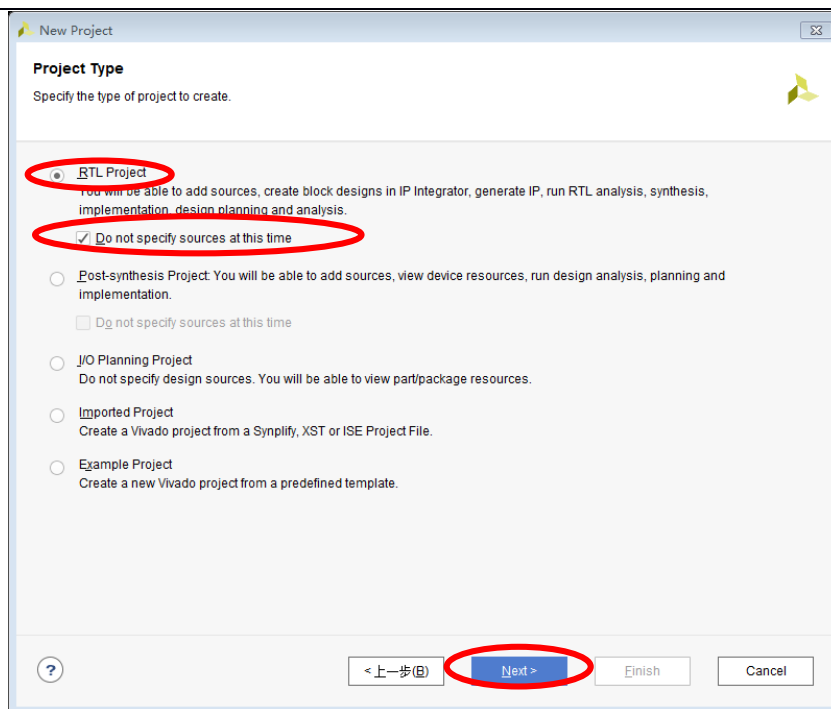


图 7-5 设置工程类型

根据使用的 FPGA 开发平台，选择对应的 FPGA 目标器件。根据实验平台搭载的 FPGA，在筛选器的“Family”选择“Artix 7”，“Package”选择“fbg676”，在筛选得到的型号里面选择“xc7a200t**fbg676-2**”。点击“Next”。

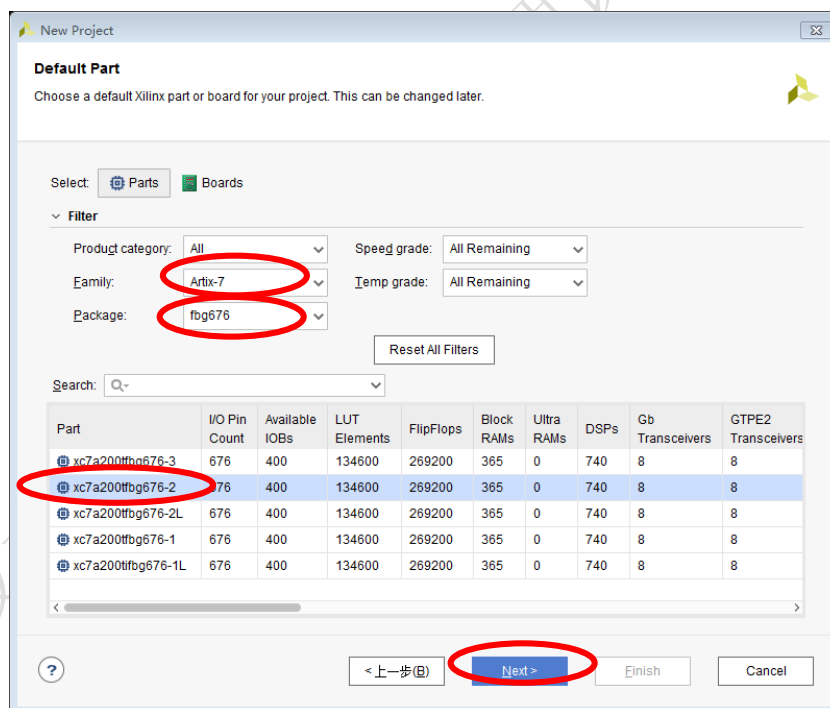


图 7-6 选择目标器件

确认工程设置信息是否正确。正确点击“Finish”，不正确则点击“上一步”返回相应步骤修改。

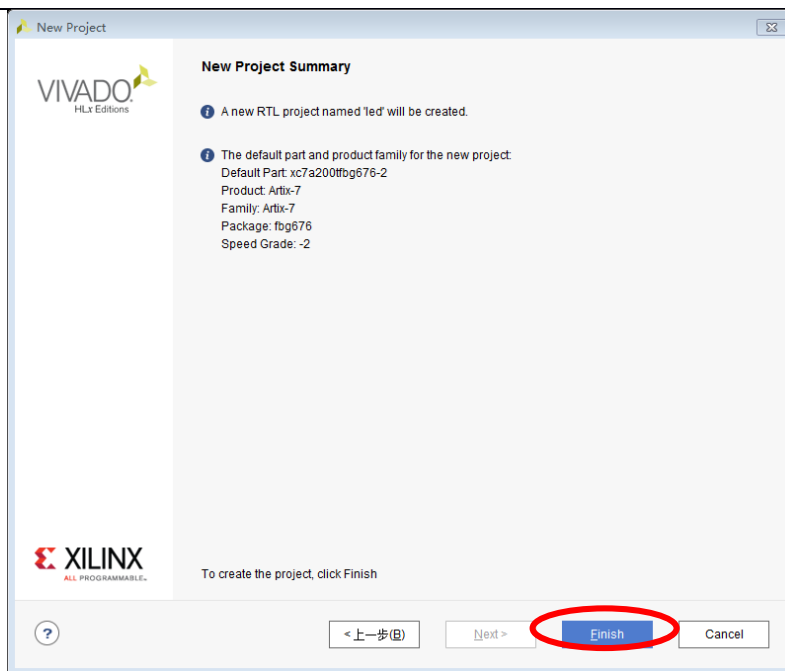


图 7-7 工程信息

完成工程新建。

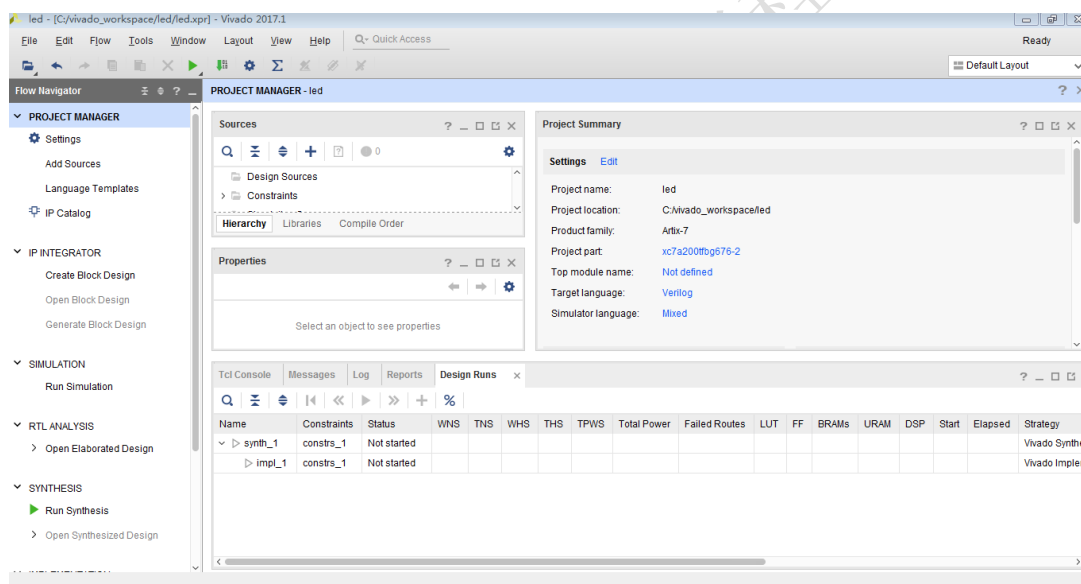


图 7-8 工程新建完成显示界面

7.3 RTL 设计输入

以使用 Verilog 完成 RTL 设计为例。Verilog 代码都是以“.v”为后缀名的文件，可以在其他文件编辑器里写好，再添加到新建的工程中，也可以在工程中新建一个再编辑。

添加源文件。在“Flow Navigator”窗口下的“Project Manager”下点击“Add sources”，或者点击“Source”窗口下“Add Sources”按钮，或者使用快捷键“Alt + A”。

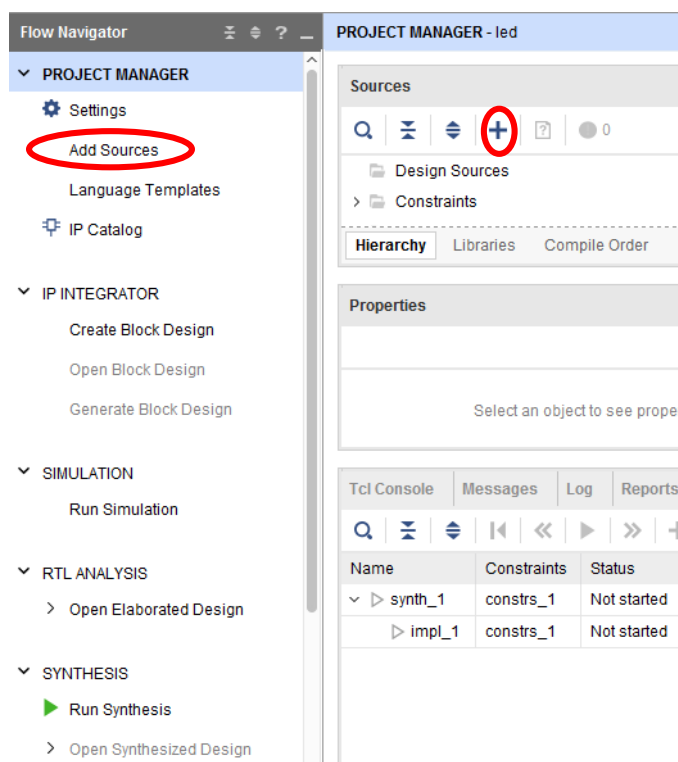


图 7-9 添加源文件

添加设计文件。选择“Add or create design sources”来添加或新建 Verilog 或 VHDL 源文件，点击“Next”。

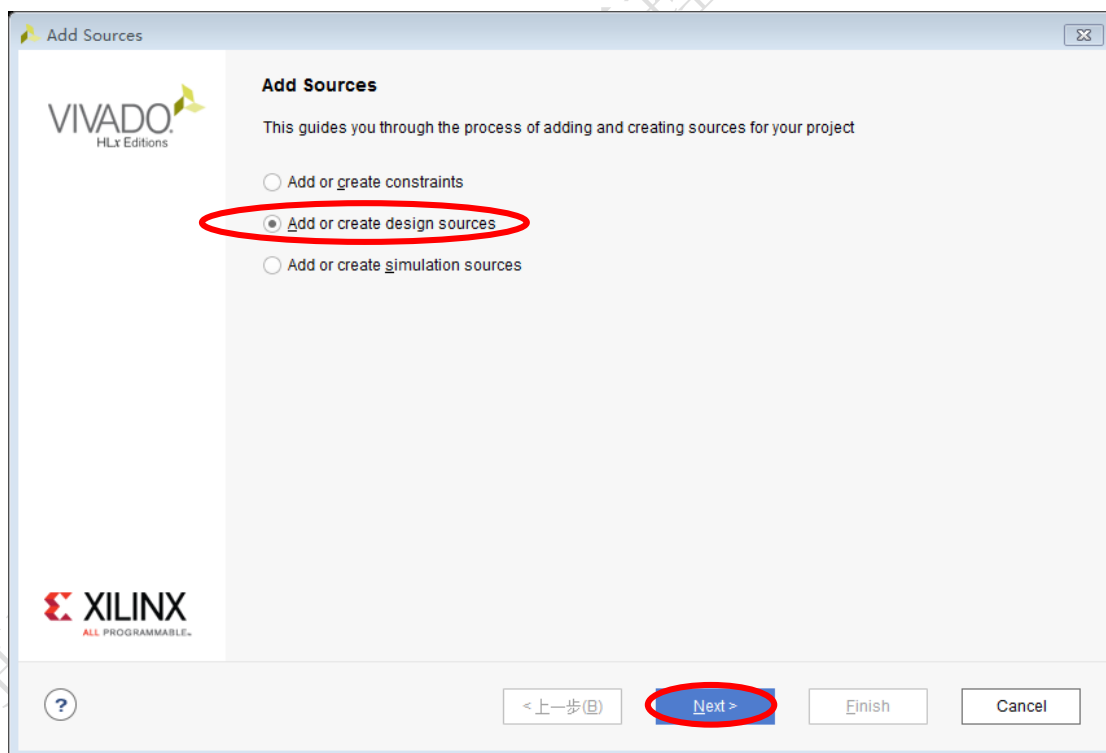


图 7-10 添加设计文件

添加或者新建设计文件。如添加已有设计文件或者添加包含已有设计文件的文件夹，选择“Add Files”或者“Add Directories”，然后在文件浏览窗口选择已有的设计文件完成添加。如创建新的设计文件，则选择“Create File”。这里新建文件。

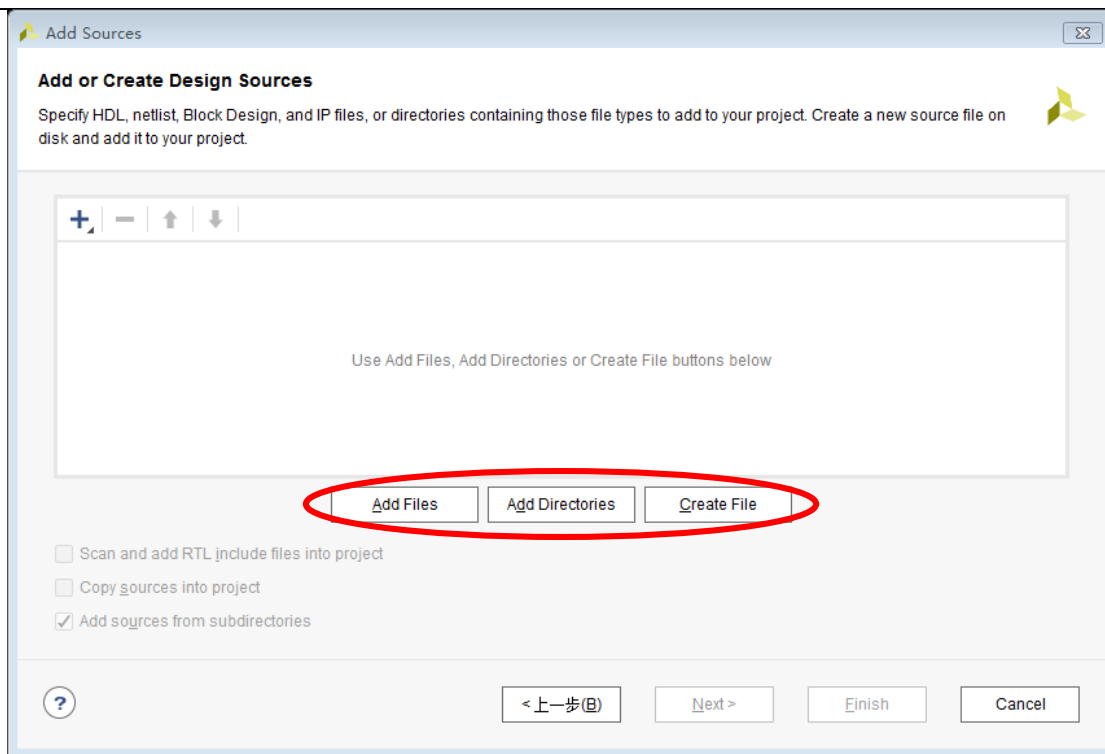


图 7-11 添加或者新建设计文件

设置新创建文件的类型、名称和文件位置。注意：文件名称和位置路径中不能出现中文和空格。

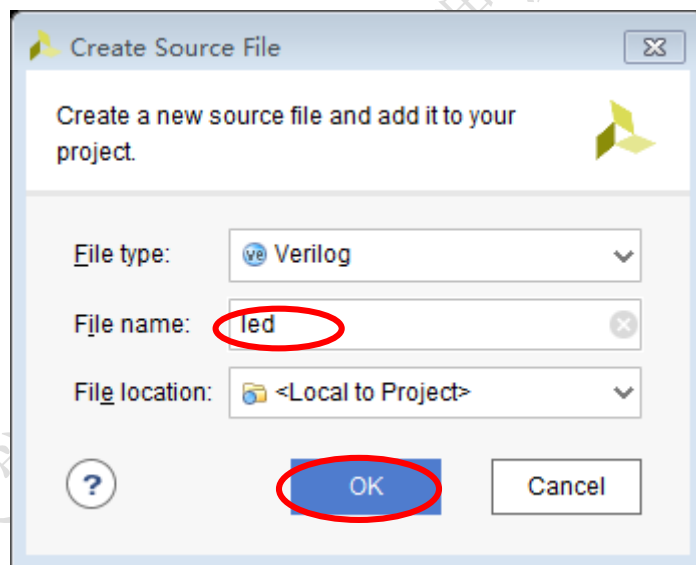


图 7-12 设置新的设计文件

继续添加设计文件或者修改已添加设计文件设置。点击“Finish”。

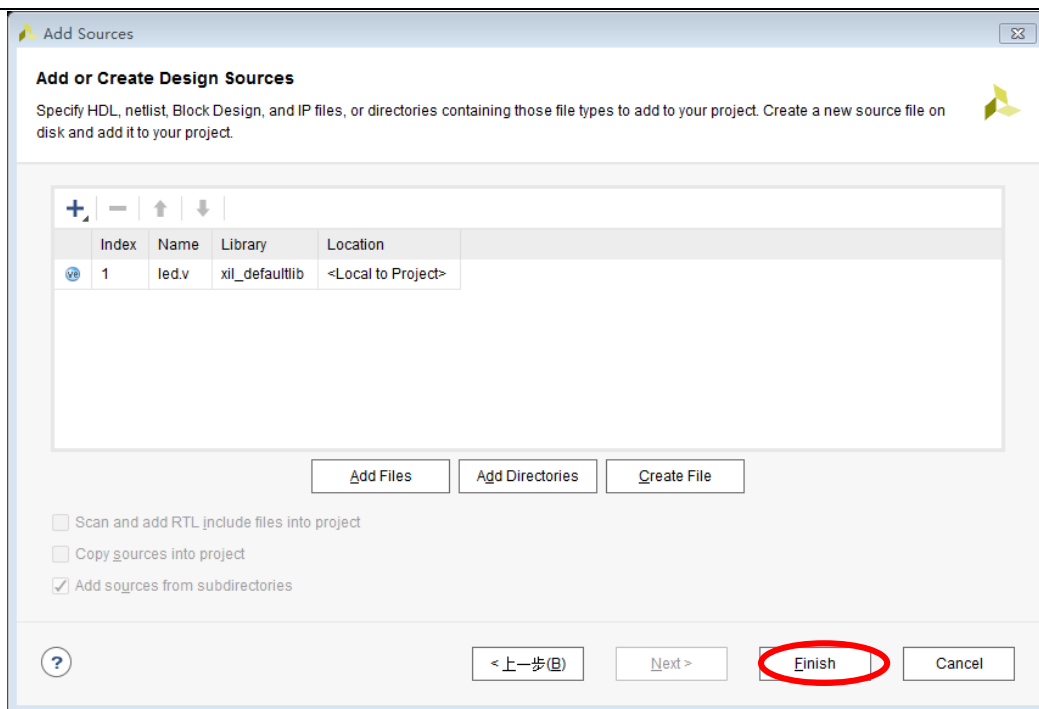


图 7-13 完成设计文件添加

模块端口设置。在“Module Definition”中的“I/O Port Definitions”，输入设计模块所需的端口，并设置端口方向，如果端口为总线型，勾选“Bus”选项，并通过“MSB”和“LSB”确定总线宽度。完成后点击“OK”。端口设置也可以在编辑源文件时完成，即可以在这一步直接点“OK”跳过。

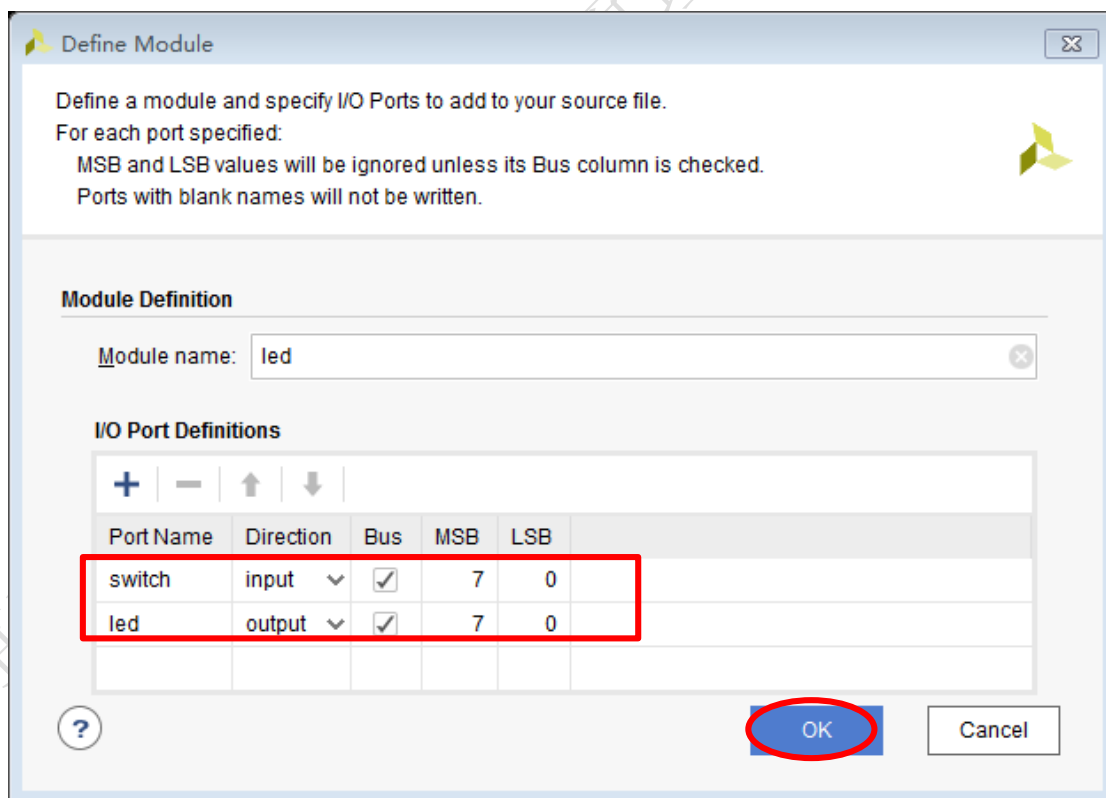


图 7-14 模块端口设置

双击“Sources”中的“Design Sources”下的“led.v”中打开该文件，输入相应的设计代码。如果设置时文件位置按默认的<Local to Project>，则设计文件位于工程目录下的“\led.srcs\sources_1\new”中。完成的设计文件如下图所示。

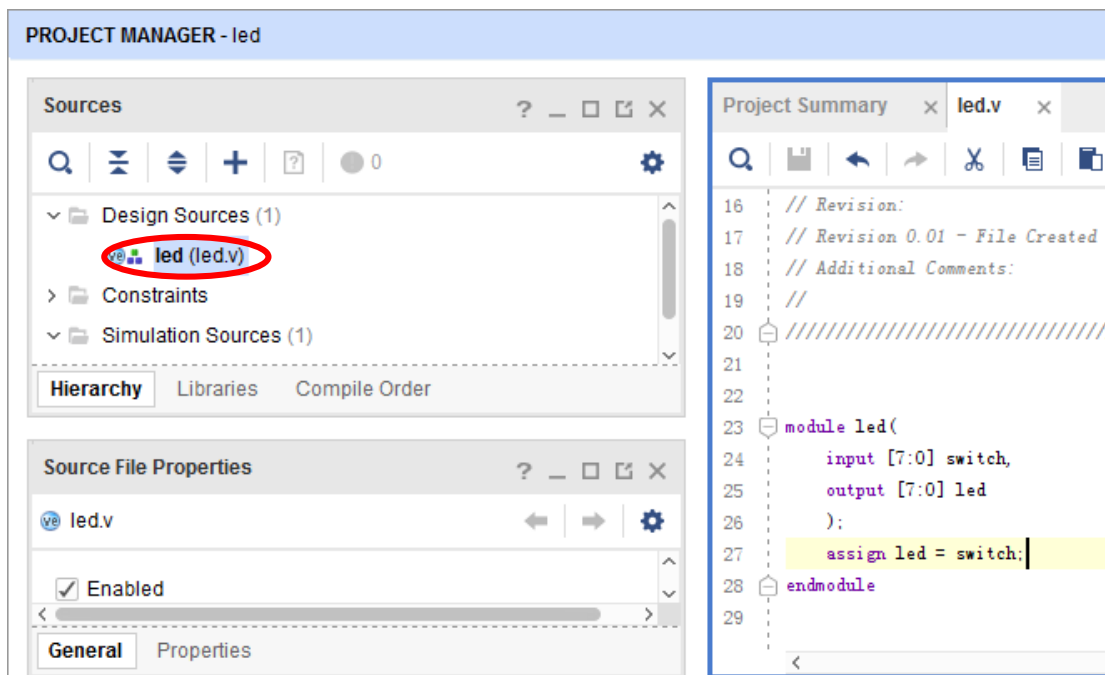


图 7-15 编辑设计文件

7.4 功能仿真

Vivado 集成了仿真器 Vivado Simulator，这里介绍使用 Vivado 仿真器仿真的方法。也可以使用 ModelSim 进行仿真。

首先添加测试激励文件。在“Source”中“Simulation Sources”右击选择“Add source”。

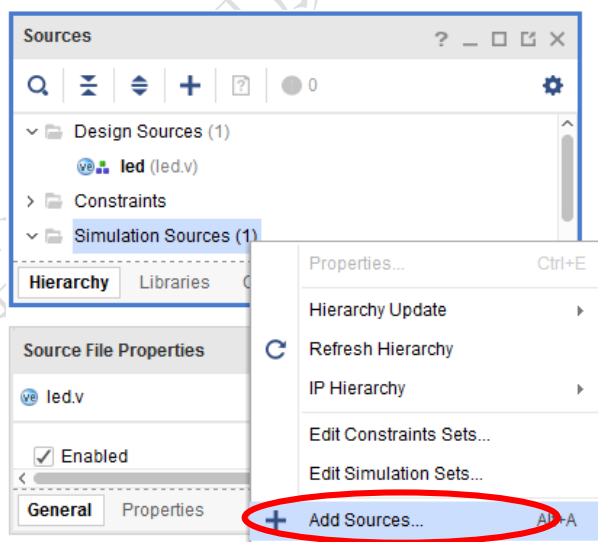


图 7-16 添加测试激励文件

在“Add Source”界面中选择“Add or Create Simulation Sources”，点击“Next”。

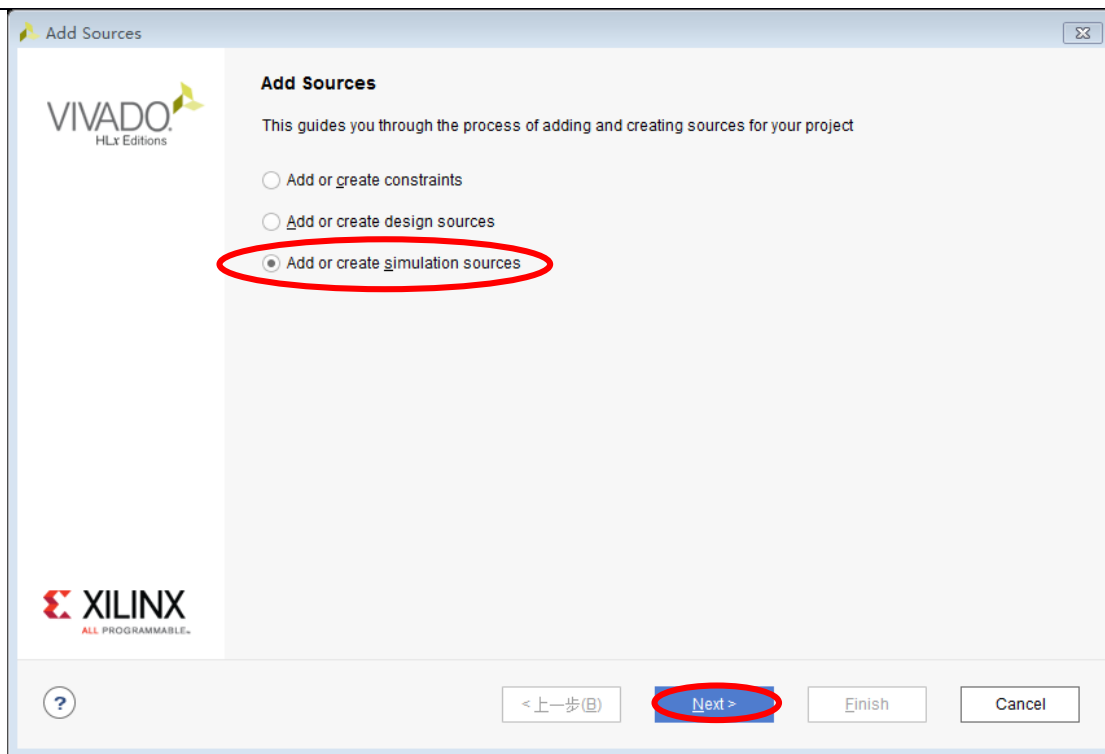


图 7-17 添加和新建测试文件

选择“Create File”，新建一个激励测试文件。

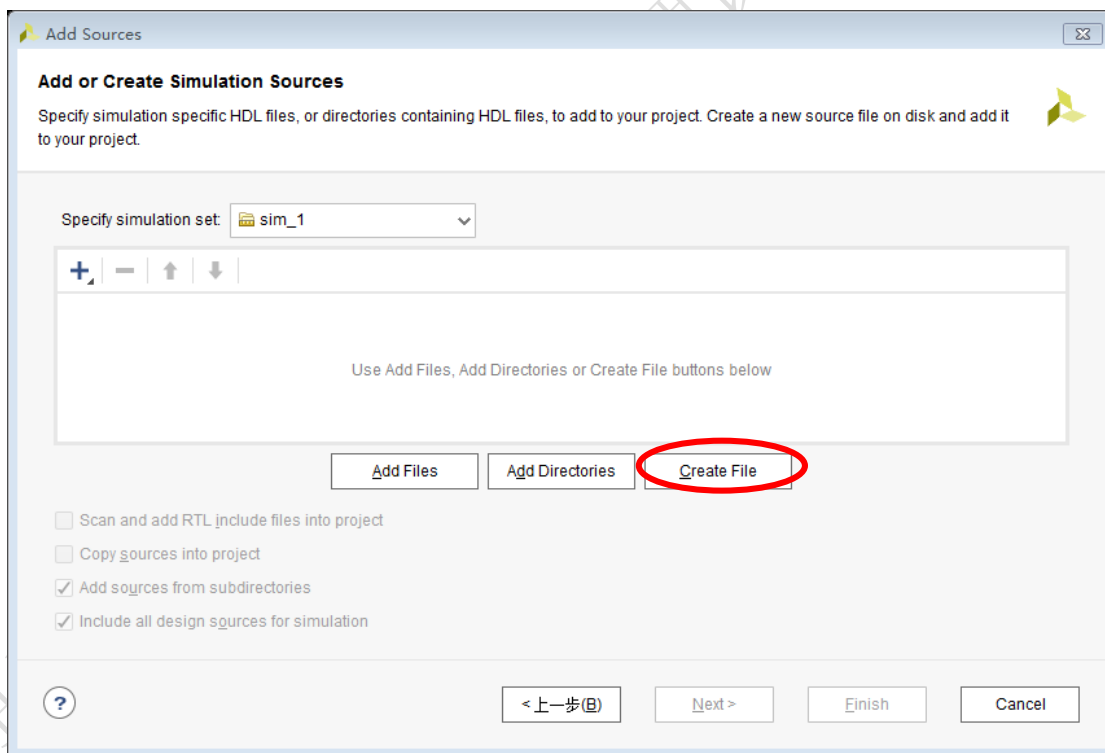


图 7-18 新建测试文件

输入激励测试文件名，点击“OK”。

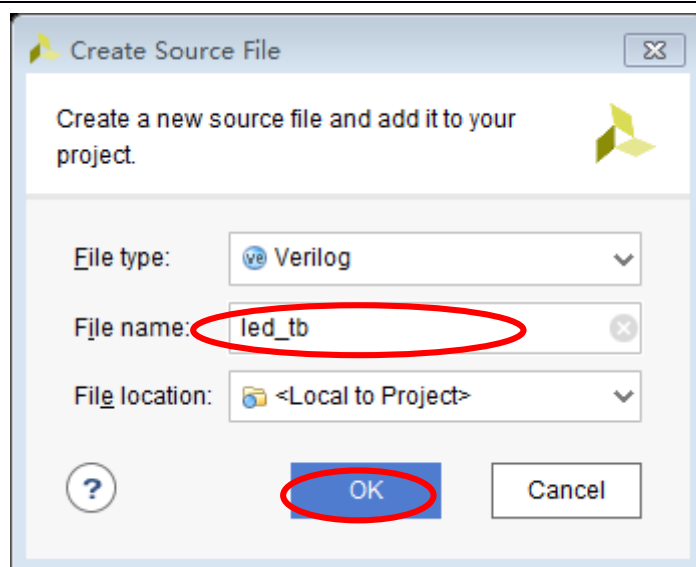


图 7-19 新建测试文件设置

完成新建测试文件，点击“Finish”。

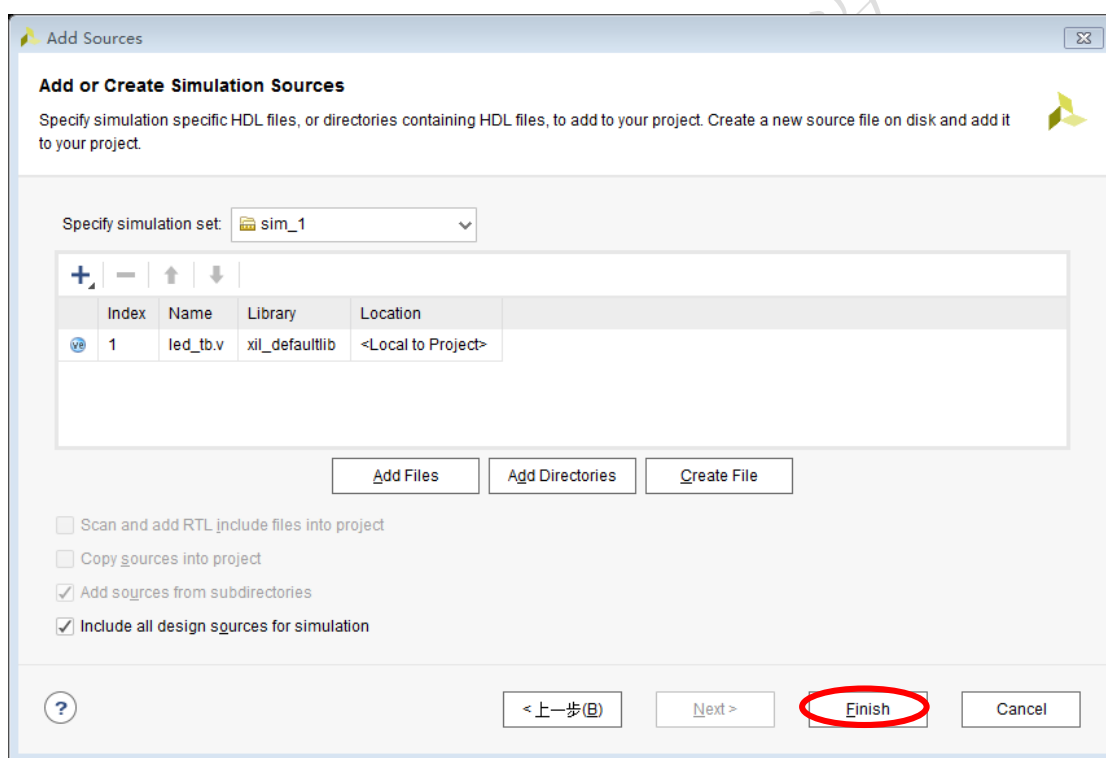


图 7-20 完成新建测试文件

对测试激励文件进行 module 端口定义，由于激励测试文件不需要有对外的接口，所以不进行 I/O 端口设置直接点击“OK”，完成空白的激励测试文件创建。

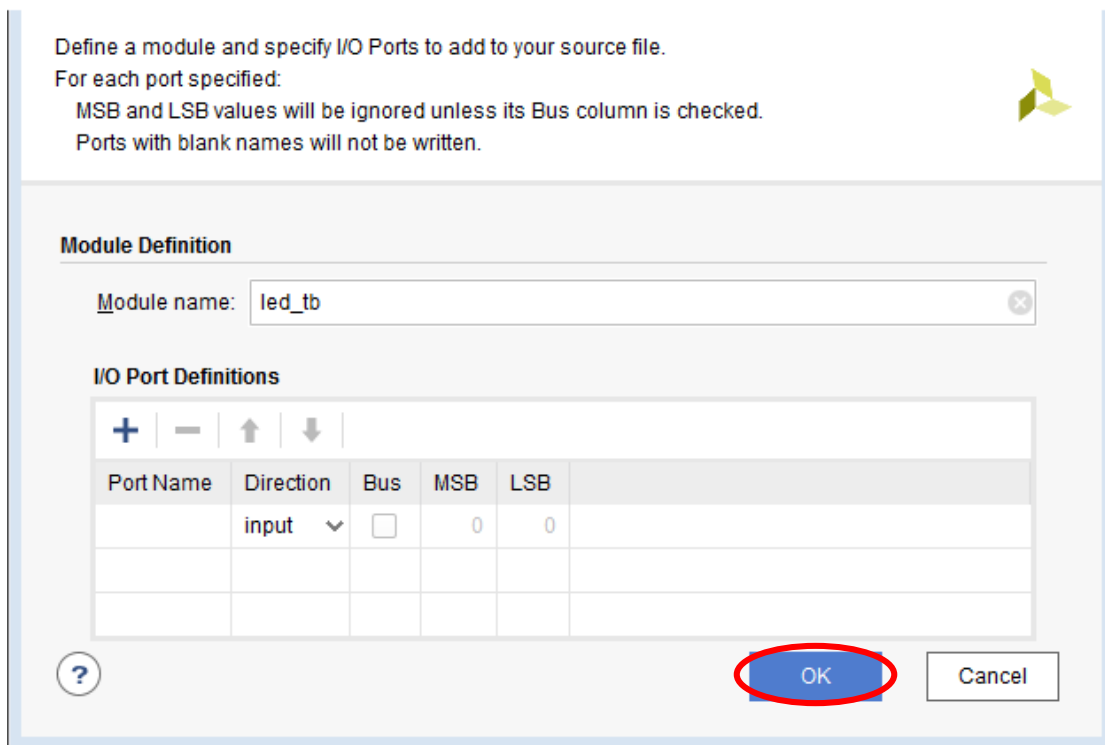


图 7-21 不需要进行端口设置

在“Source”窗口下双击打开空白的激励测试文件。测试文件 led_tb.v 位于工程目录下“\led.srcs\sim_1\new”文件夹下。

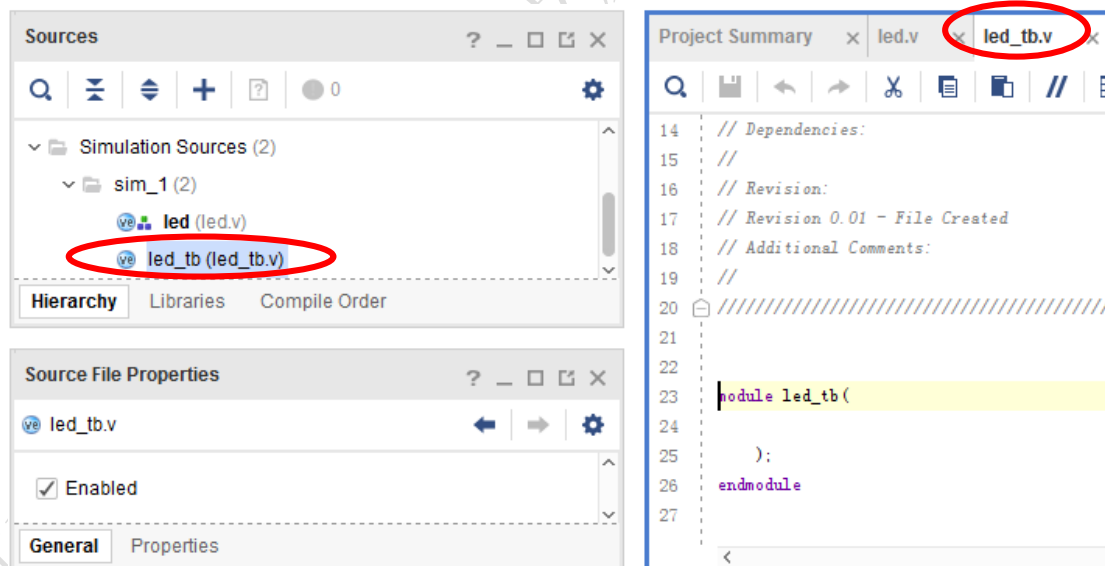


图 7-22 空白测试激励文件

完成对将要仿真的 module 的实例化和激励代码的编写，如下述代码所示。

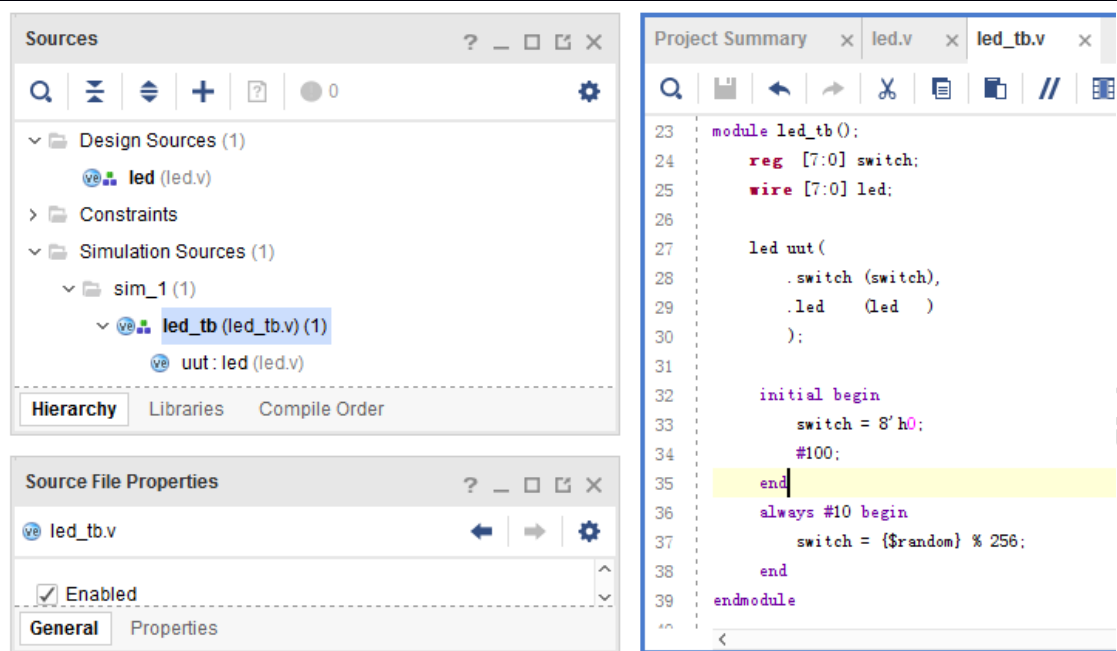


图 7-23 完成测试激励文件

进入仿真。在左侧“Flow Navigator”窗口中点击“Simulation”下的“Run Simulation”选项，选择“Run Behavioral Simulation”，进入仿真界面。

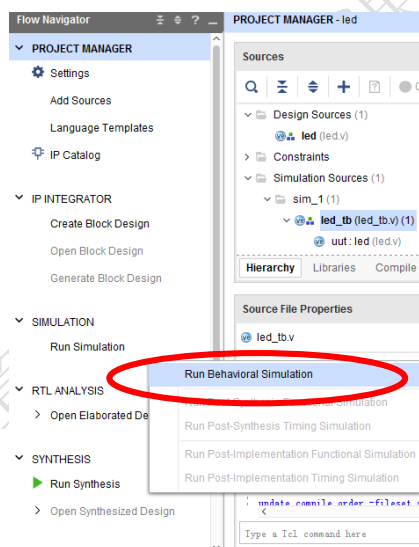


图 7-24 运行行为级仿真

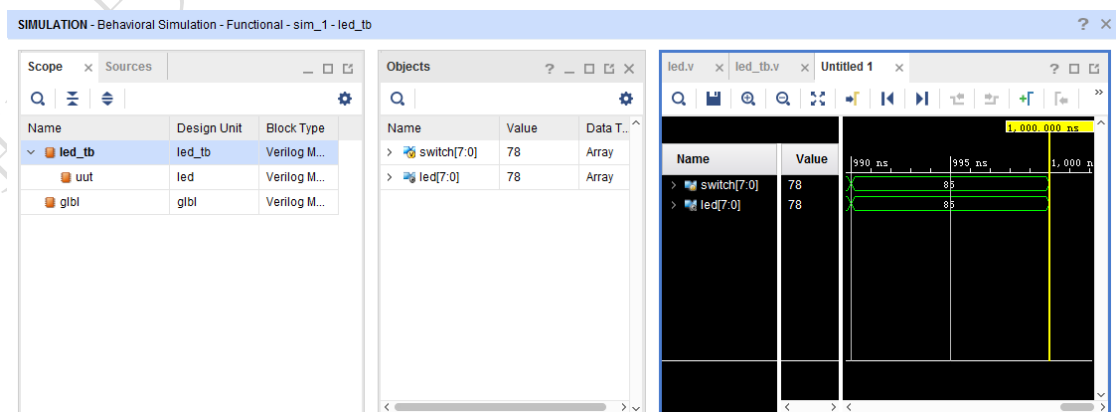


图 7-25 行为级仿真界面

可通过左侧“Scope”一栏中的目录结构定位到想要查看的 module 内部信号，在“Objects”对应的信号名称上右击选择“Add To Wave Window”，将信号加入波形图中。仿真器默认显示 I/O 信号，由于这个示例不存在内

部信号，因此不需要添加观察信号。

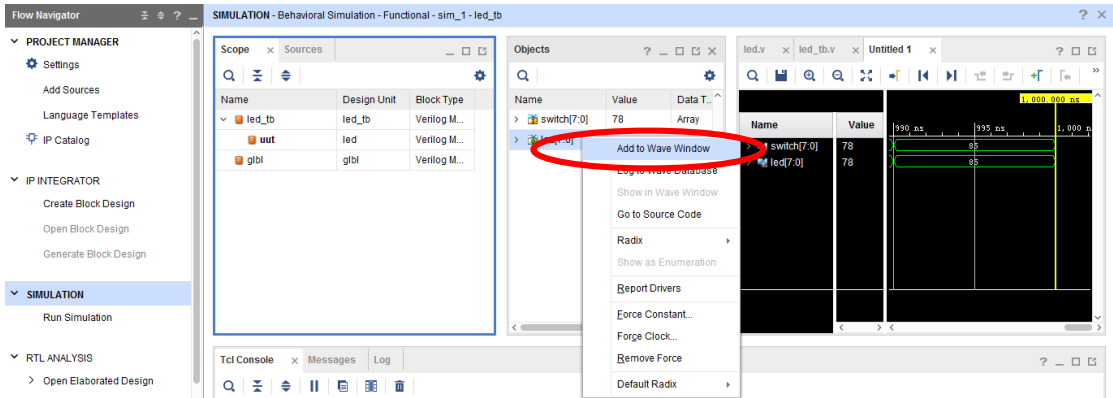


图 7-26 将内部信号添加到波形图

可通过选择工具栏中的选项来进行波形的仿真时间控制。如下图工具条，分别是复位波形（即清空现有波形）、运行仿真、运行特定时长的仿真、仿真时长设置、仿真时长单位、单步运行、暂停、重启。



图 7-27 仿真控制工具

观察仿真波形是否符合预期功能。在波形显示窗口上侧是波形图控制工具，由左到右分别是：查找、保存波形配置、放大、缩小、缩放至全显示、缩放至光标、转到时间 0、转到时间的最后、前一个跳变、下一次跳变、添加标记、前标记、下一个标记、交换光标。



图 7-28 波形图控制工具

可通过右键选中信号来改变信号的显示形态。如下图将信号改为二进制显示。

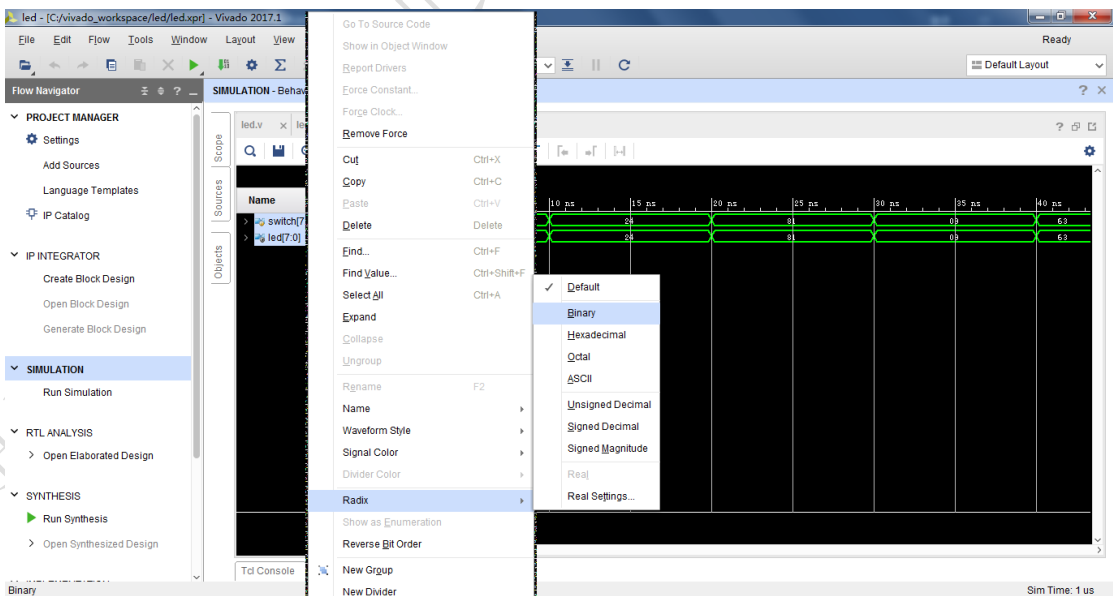


图 7-29 仿真波形窗口

查看波形检查设计功能正确性。

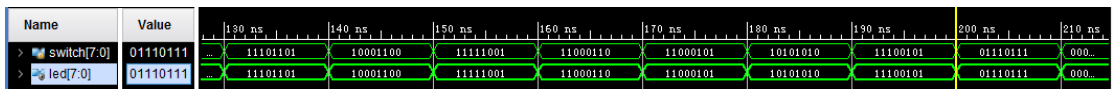


图 7-30 仿真波形结果

7.5 添加约束文件

添加约束文件有两种方法，一是利用 Vivado 中 I/O Planning 功能，二是直接新建 XDC 的约束文件，手动输入约束命令。下面分别介绍这两种方法：

7.5.1 利用 I/O Planning 生成约束文件

点击“Flow Navigator”中“Synthesis”下的“Run Synthesis”，在弹出的“Launch Runs”窗口点“OK”，先对工程进行综合。

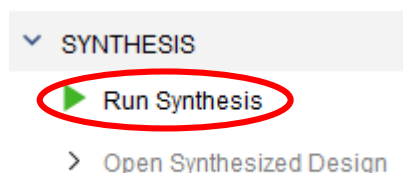


图 7-31 综合工程

综合完成之后，选择“Open Synthesized Design”，打开综合后的网表。

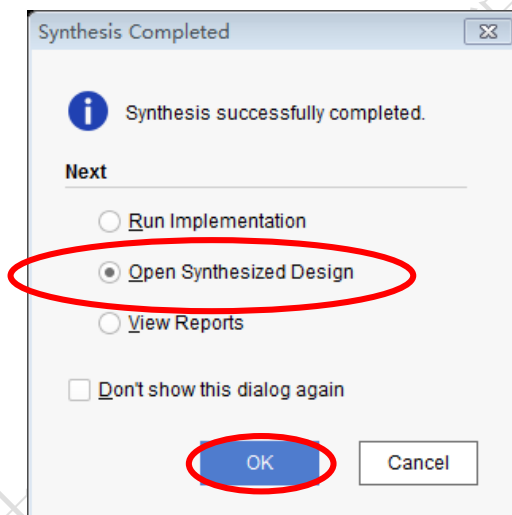


图 7-32 打开综合结果

此时应看到如下界面，如果没出现如下界面，在图示位置的“layout”中选择“I/O Planning”一项。

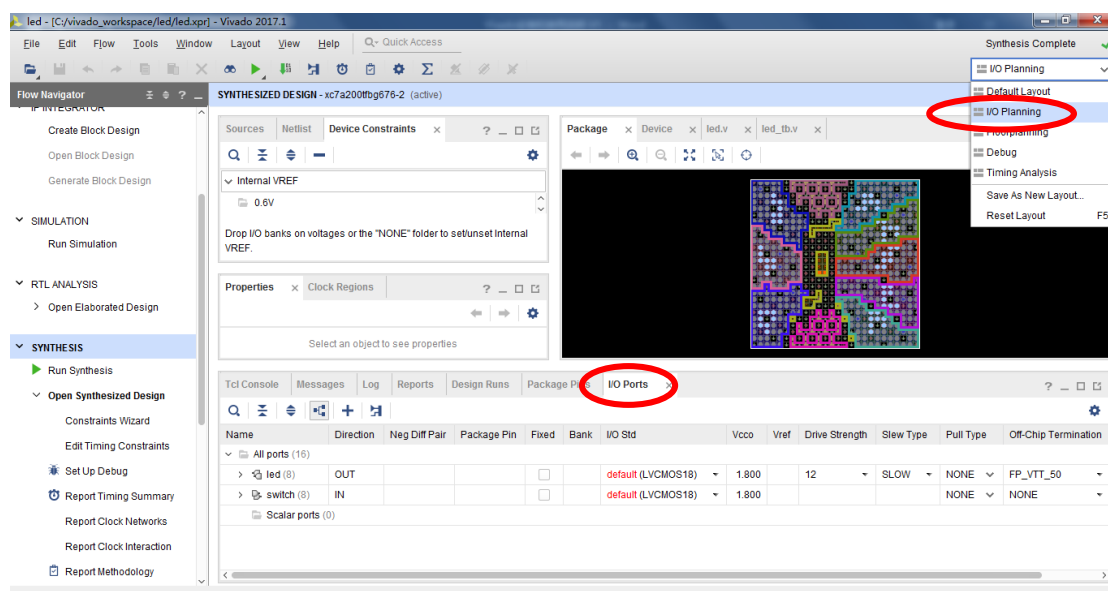


图 7-33 I/O Planning 界面

在界面右下方的选项卡中切换到“IO Ports”一栏，并在对应的信号后，在“Site”一列输入对应的 FPGA 管脚标号（或将信号拖拽到右上方“Package”图中对应的管脚上），并指定“I/O std”。具体的 FPGA 约束管脚和 I/O 电平标准，可参考对应板卡的原理图或提供的常用的引脚对应关系表（参见 龙芯 Artix-7 实验箱-原理图与引脚列表）。

	A	B	C
1			引脚对应关系
2	引脚名称	FPGA接口编号	描述
3	LED灯		
4	FPGA_LED1	H7	实验板放正，一排LED灯左起第一个。
5	FPGA_LED2	D5	依次类推
6	FPGA_LED3	A3	
7	FPGA_LED4	A5	
8	FPGA_LED5	A4	
9	FPGA_LED6	F7	
10	FPGA_LED7	G8	
11	FPGA_LED8	H8	
12	FPGA_LED9	J8	
13	FPGA_LED10	J23	
14	FPGA_LED11	J26	
15	FPGA_LED12	G9	
16	FPGA_LED13	J19	
17	FPGA_LED14	H23	
18	FPGA_LED15	J21	
19	FPGA_LED16	K23	实验板放正，一排LED灯左起第八个。

图 7-34 引脚对应关系表

完成 I/O Ports 设置如下图，“I/O Std”设置为“LVCMOS33*”表明 I/O 标准是 3.3V 的 LVCMOS。

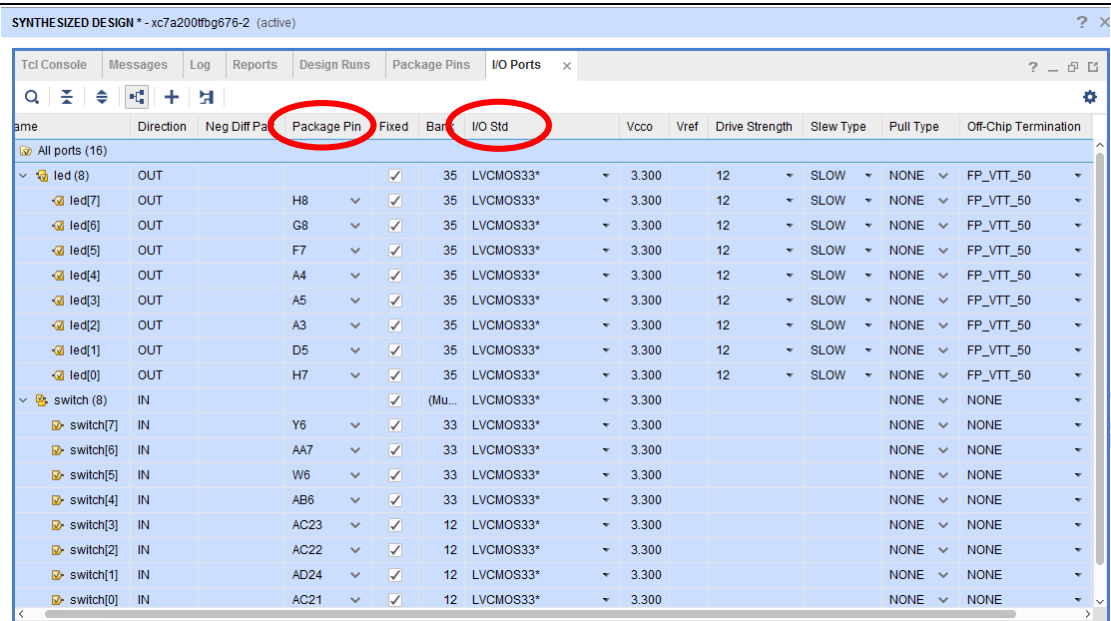


图 7-35 完成 I/O 端口设置

点击左上方工具栏中的保存按钮，提示添加新的约束文件使综合过时失效。点击“OK”。

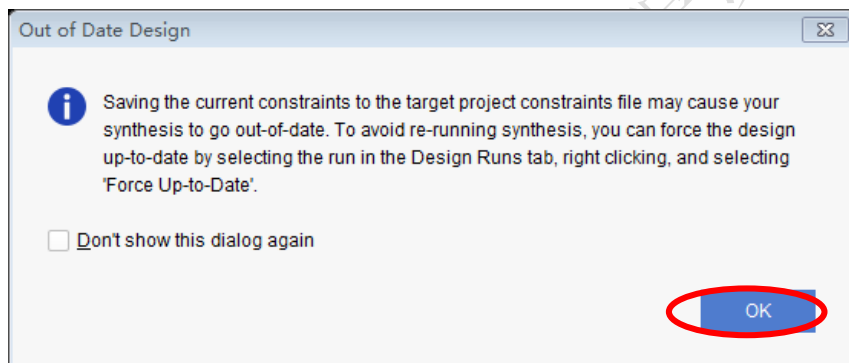


图 7-36 提示综合失效

新建 XDC 文件或选择工程中已有的 XDC 文件。选择“Create a new file”，输入“File name”，点击“OK”完成约束过程。

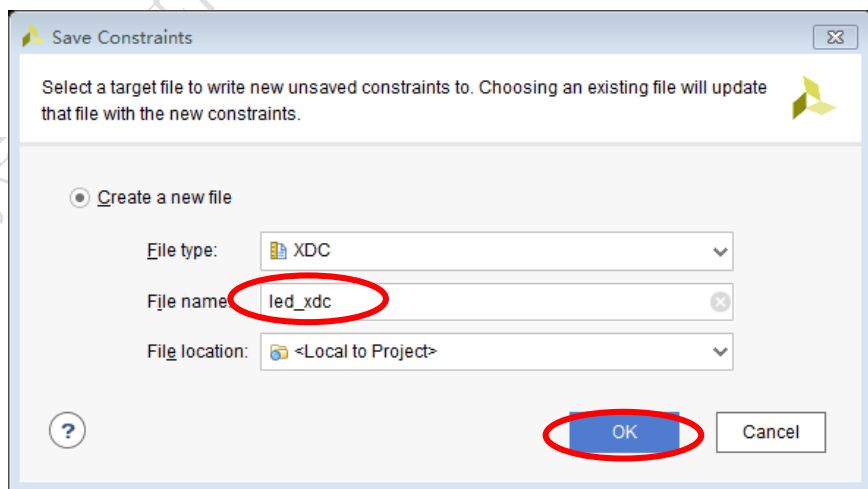


图 7-37 新建 XDC 文件设置

在“Sources”窗口“Constraints”层次下可以查看新建的 XDC 文件。

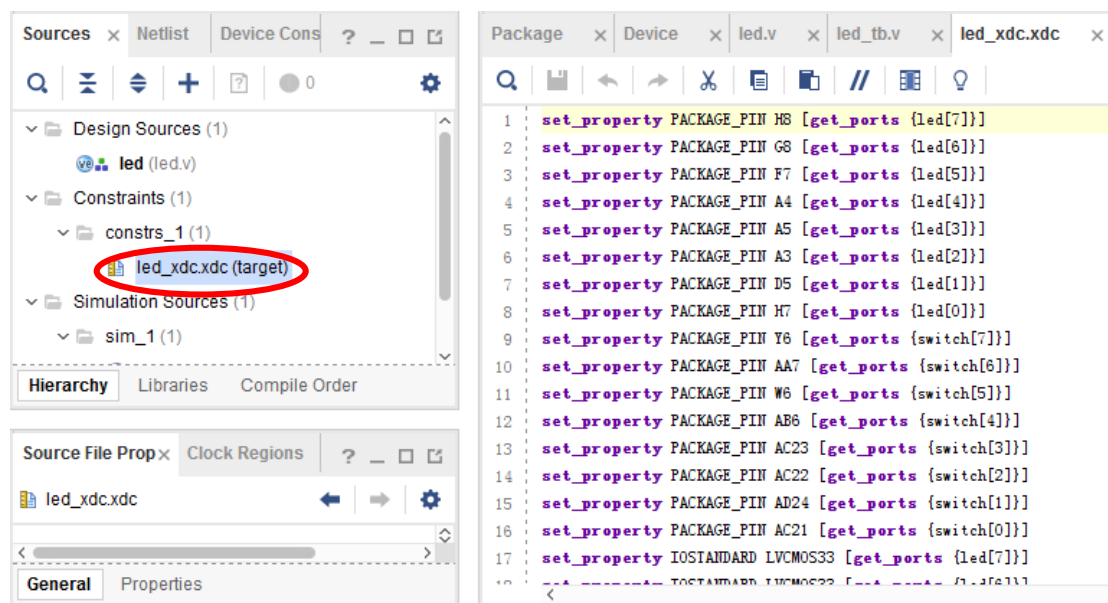


图 7-38 完成 XDC 文件创建

7.5.2 新建 XDC 文件并手动输入约束

新建约束文件。点击“Add Sources”，选择“Add or Create Constraints”，点击“Next”。

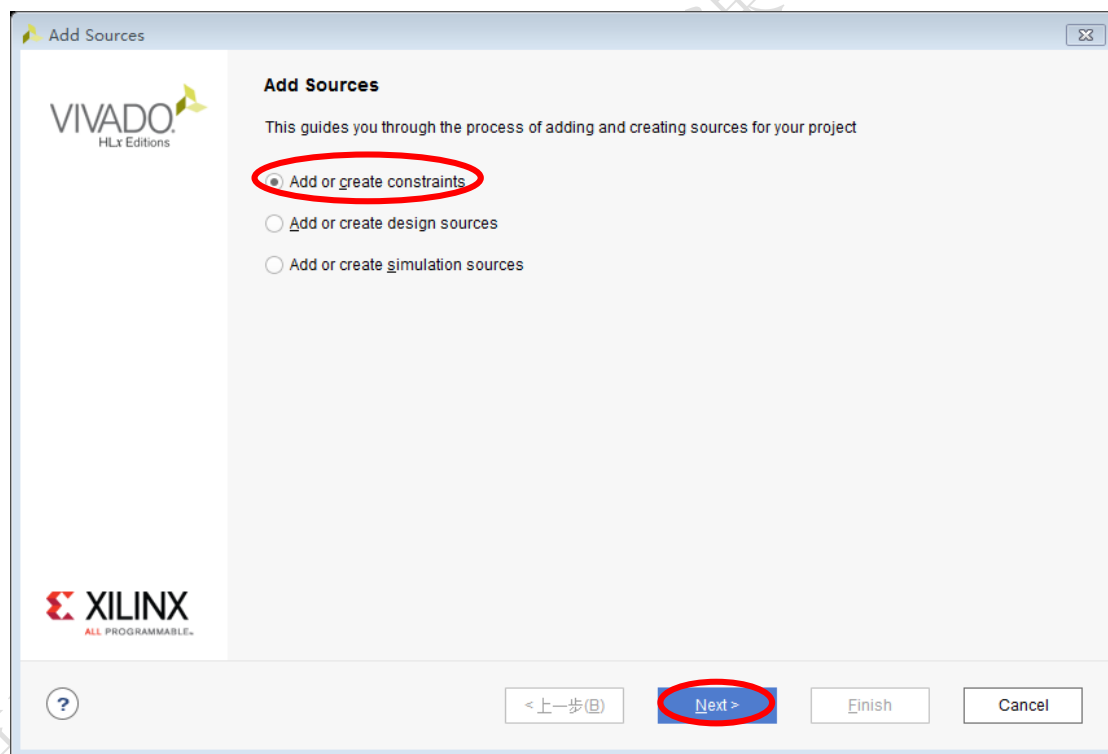


图 7-39 新建约束文件

添加或创建约束文件。点击“Create Files”添加约束文件。

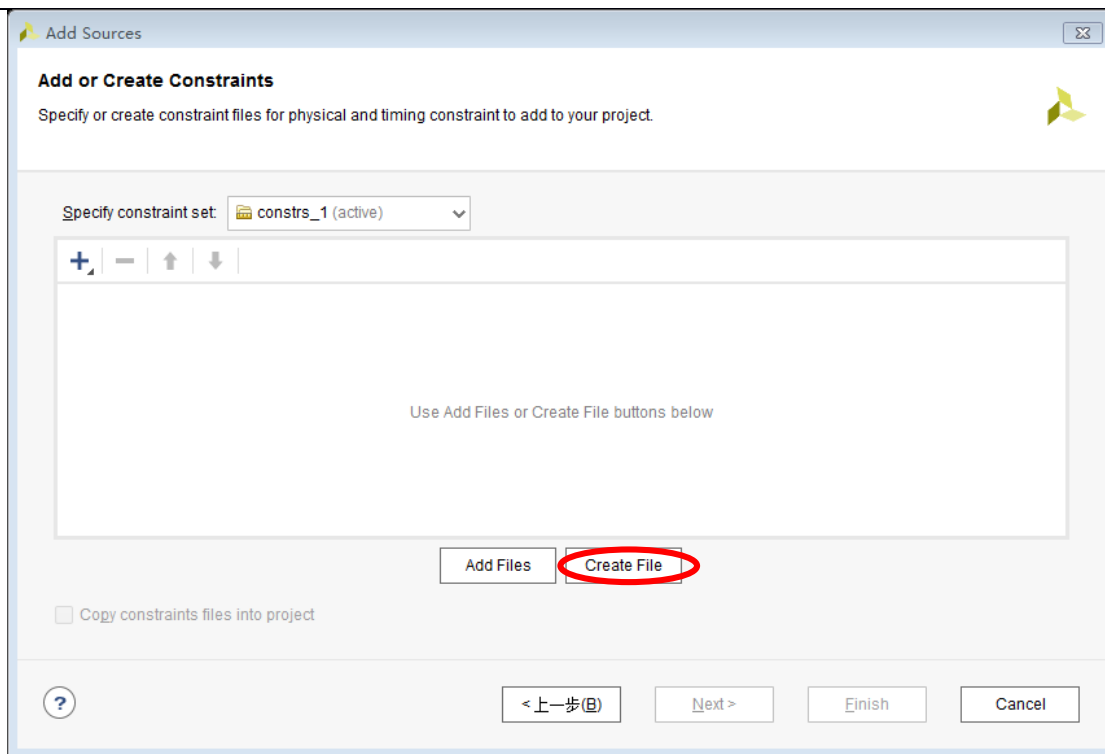


图 7-40 添加或创建约束文件

设置新建的 XDC 文件，输入 XDC 文件名，点击“OK”。默认文件在工程目录下的“\led.srcs\constrs_1\new”中。

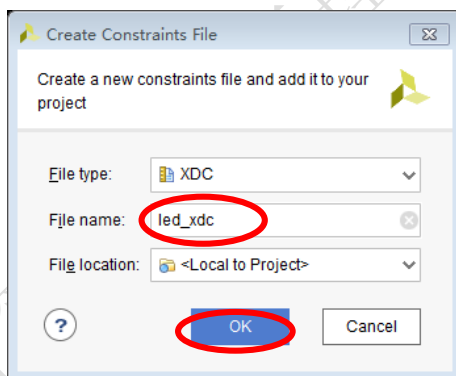


图 7-41 新建 XDC 文件设置

完成新建 XDC 文件，点击“Finish”。

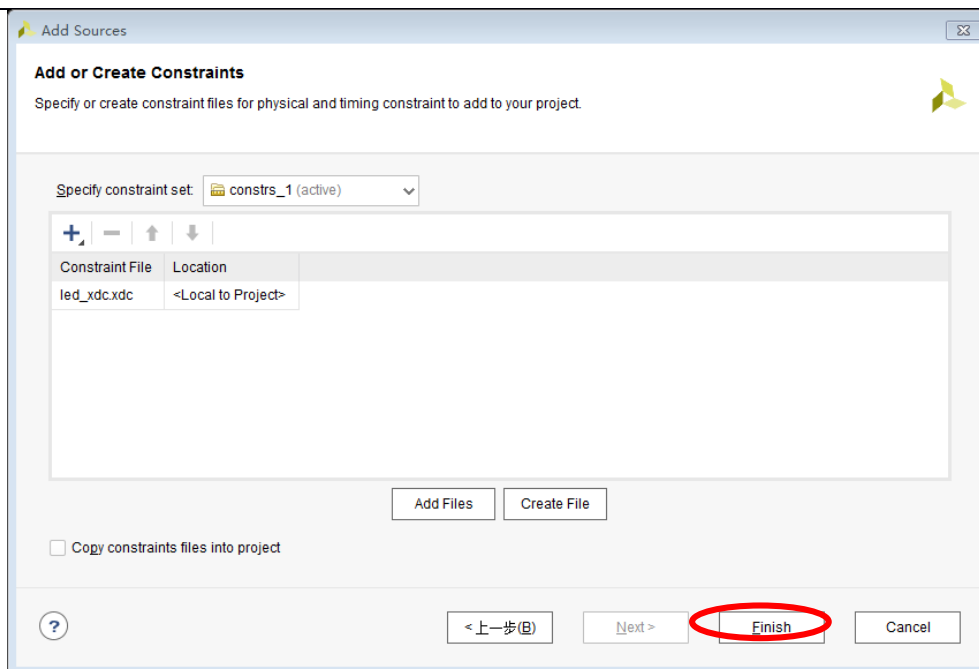


图 7-42 完成新建 XDC 文件

在“Sources”窗口“Constraints”下双击打开新建好的 XDC 文件“led_xdc.xdc”，并参照下面的约束文件格式，输入相应的 FPGA 管脚约束信息和电平标准。

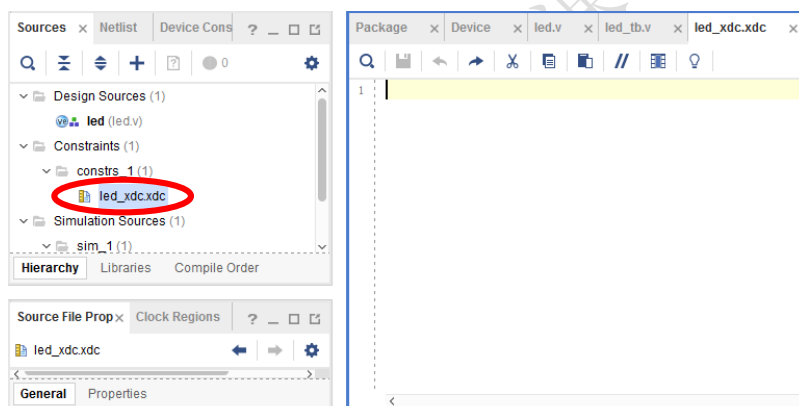


图 7-43 打开空白 XDC 文件

约束文件参考：

```

1 set_property PACKAGE_PIN H8 [get_ports {led[7]}]
2 set_property PACKAGE_PIN G8 [get_ports {led[6]}]
3 set_property PACKAGE_PIN F7 [get_ports {led[5]}]
4 set_property PACKAGE_PIN A4 [get_ports {led[4]}]
5 set_property PACKAGE_PIN A5 [get_ports {led[3]}]
6 set_property PACKAGE_PIN A3 [get_ports {led[2]}]
7 set_property PACKAGE_PIN D5 [get_ports {led[1]}]
8 set_property PACKAGE_PIN H7 [get_ports {led[0]}]
9 set_property PACKAGE_PIN Y6 [get_ports {switch[7]}]
10 set_property PACKAGE_PIN AA7 [get_ports {switch[6]}]
11 set_property PACKAGE_PIN W6 [get_ports {switch[5]}]
12 set_property PACKAGE_PIN AB6 [get_ports {switch[4]}]
13 set_property PACKAGE_PIN AC23 [get_ports {switch[3]}]
14 set_property PACKAGE_PIN AC22 [get_ports {switch[2]}]
15 set_property PACKAGE_PIN AD24 [get_ports {switch[1]}]
16 set_property PACKAGE_PIN AC21 [get_ports {switch[0]}]

```

```

17 set_property IOSTANDARD LVCMOS33 [get_ports {led[7]}]
18 set_property IOSTANDARD LVCMOS33 [get_ports {led[6]}]
19 set_property IOSTANDARD LVCMOS33 [get_ports {led[5]}]
20 set_property IOSTANDARD LVCMOS33 [get_ports {led[4]}]
21 set_property IOSTANDARD LVCMOS33 [get_ports {led[3]}]
22 set_property IOSTANDARD LVCMOS33 [get_ports {led[2]}]
23 set_property IOSTANDARD LVCMOS33 [get_ports {led[1]}]
24 set_property IOSTANDARD LVCMOS33 [get_ports {led[0]}]
25 set_property IOSTANDARD LVCMOS33 [get_ports {switch[7]}]
26 set_property IOSTANDARD LVCMOS33 [get_ports {switch[6]}]
27 set_property IOSTANDARD LVCMOS33 [get_ports {switch[5]}]
28 set_property IOSTANDARD LVCMOS33 [get_ports {switch[4]}]
29 set_property IOSTANDARD LVCMOS33 [get_ports {switch[3]}]
30 set_property IOSTANDARD LVCMOS33 [get_ports {switch[2]}]
31 set_property IOSTANDARD LVCMOS33 [get_ports {switch[1]}]
32 set_property IOSTANDARD LVCMOS33 [get_ports {switch[0]}]

```

7.6 工程实现

在“Flow Navigator”中点击“Program and Debug”下的“Generate Bitstream”选项，工程会自动完成综合、布局布线、Bit 文件生成过程，完成之后，可点击“Open Implemented Design”来查看工程实现结果。

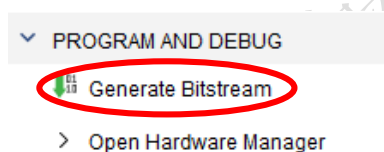


图 7-44 生成 bit 流文件

提示综合过时重新运行综合和实现，点击“Yes”。在弹出的“Launch Runs”窗口点“OK”。

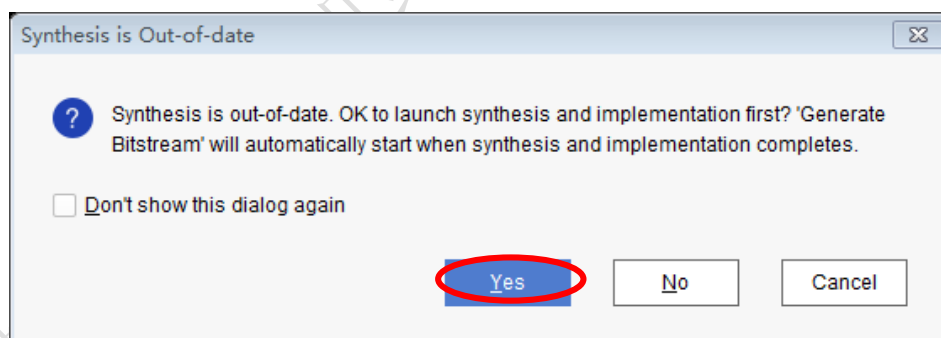


图 7-45 提示重新运行综合和实现

在比特流文件生成完成的窗口选择“Open Hardware Manager”，进入硬件管理界面。连接 FPGA 开发板的电源线和与电脑的下载线，打开 FPGA 电源。

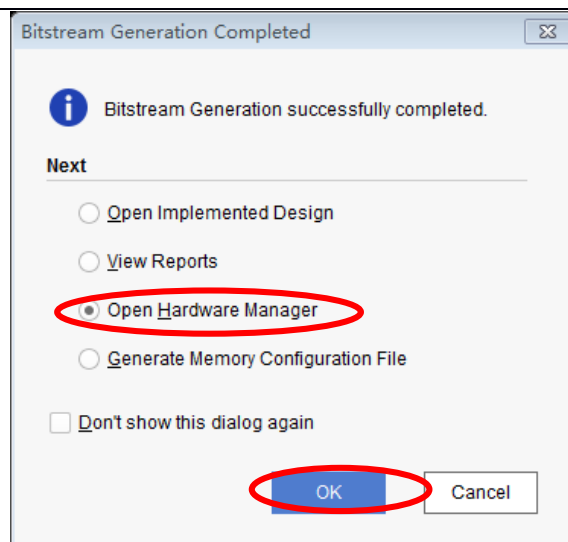


图 7-46 打开硬件程序和调试管理

在“Hardware Manager”窗口的提示信息中，点击“Open target”的下拉菜单的“Open New Target”（或在“Flow Navigator”下“Program and Debug”中展开“Open Hardware Manager”，点击“Open Target”->“Open New Target”）。也可以选择“Auto Connect”自动连接器件。

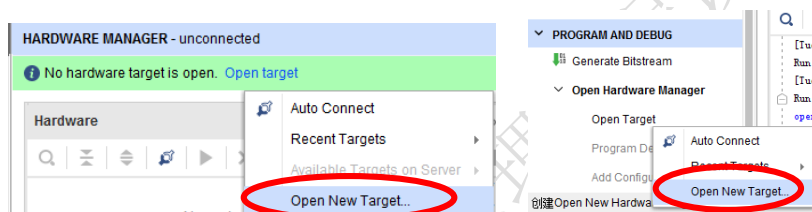


图 7-47 打开新目标

在“Open Hardware Target”向导中，先点击“Next”，进入 Server 选择向导。

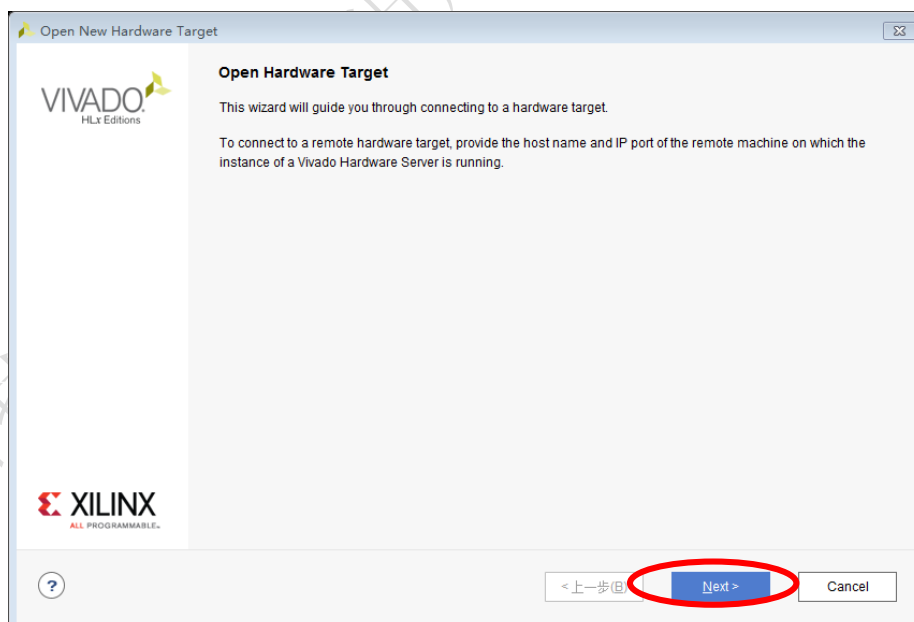


图 7-48 Open Hardware Target 向导

选择连接到“Local server”，点击“Next”。

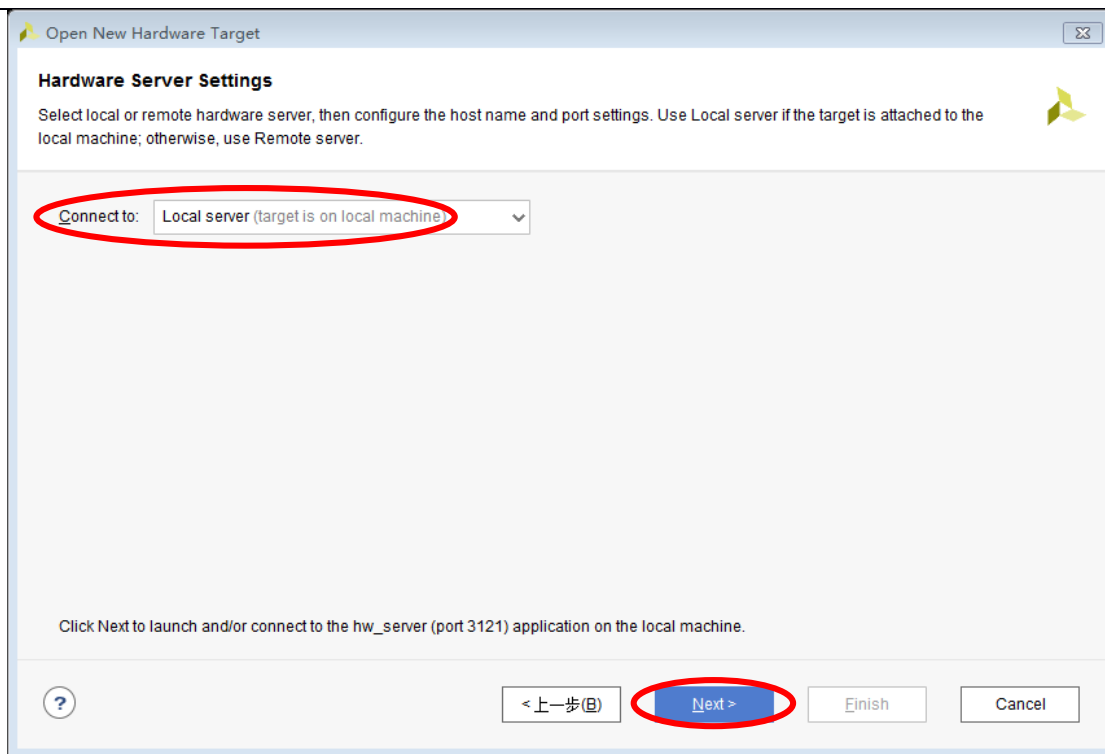


图 7-49 连接 Local server

选择目标硬件，点击“Next”。

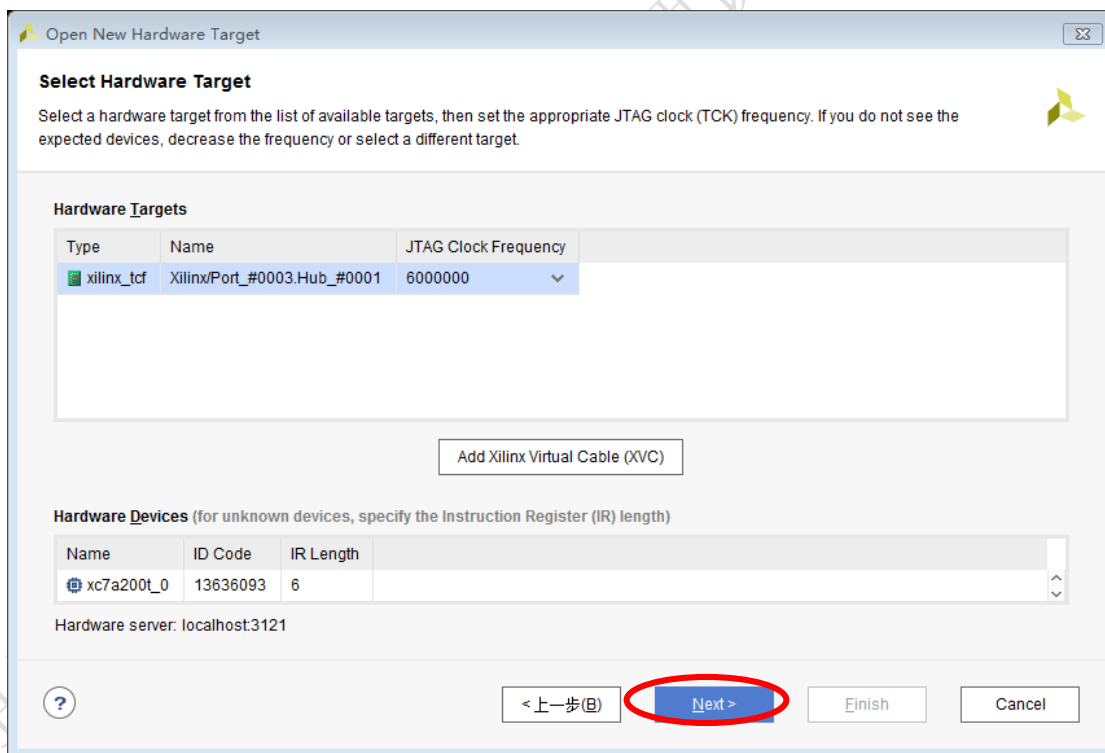


图 7-50 选择目标硬件

完成目标硬件打开。点击“Finish”。

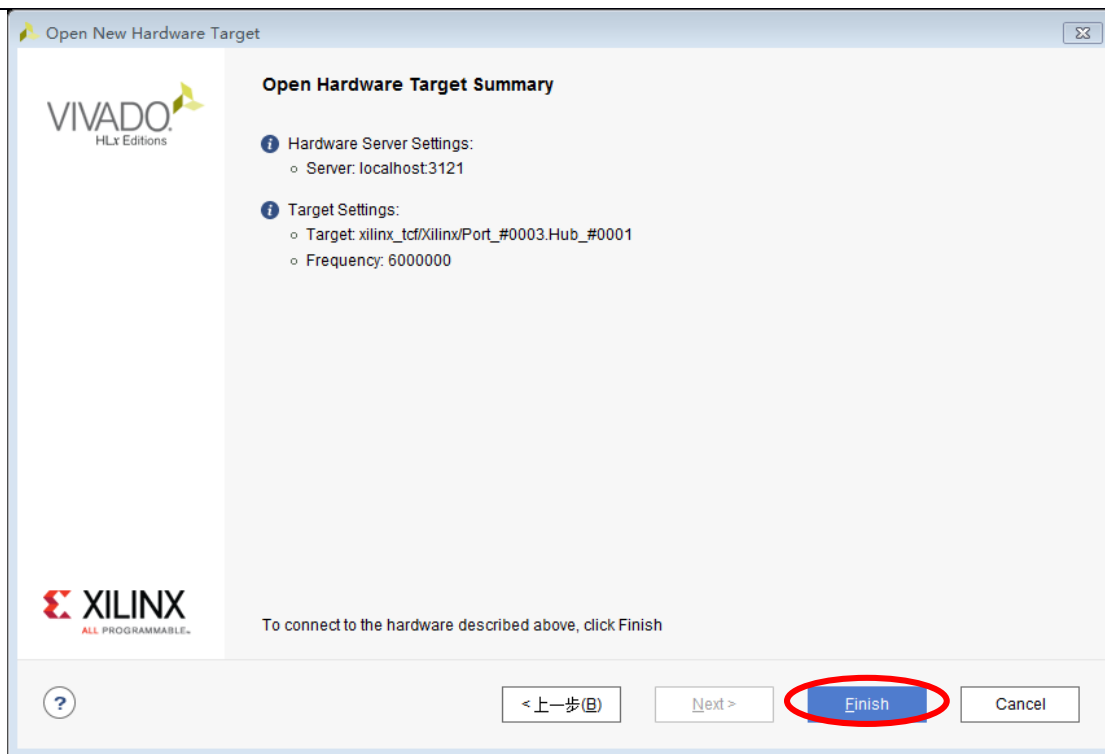


图 7-51 完成目标硬件打开

对目标硬件编程。在“Hardware”窗口右键单击目标器件“xc7a200t_0”，选择“Program Device...”。或者“Flow Navigator”窗口中“Program and Debug”->“Hardware Manager”->“Program Device”。

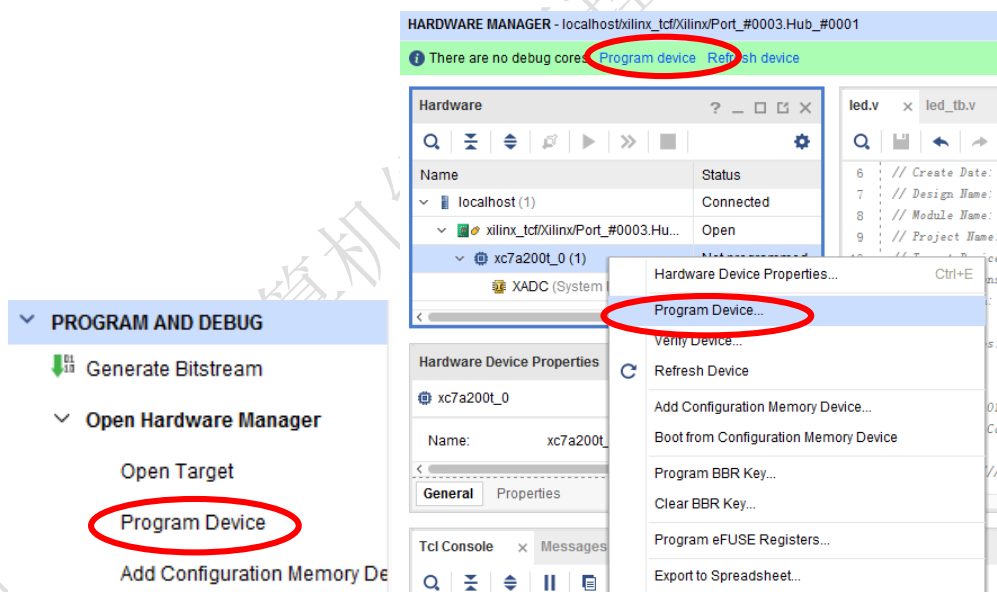


图 7-52 对目标器件编程

选择下载的 bit 流文件，点击“Program”。

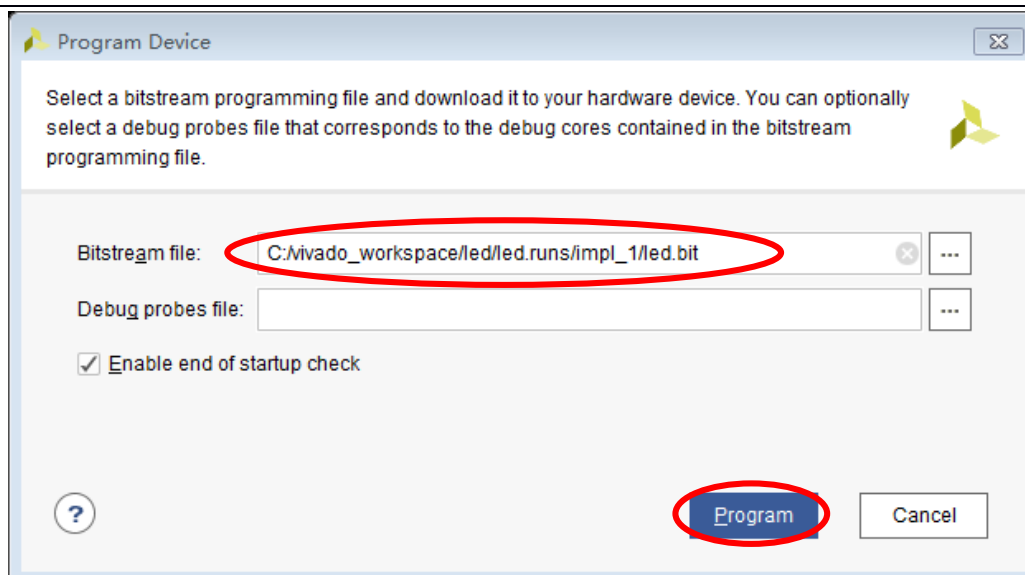


图 7-53 选择 bit 流文件

完成下载后，“Hardware”窗口下的“xc7a200t_0”的状态变成“Programmed”。

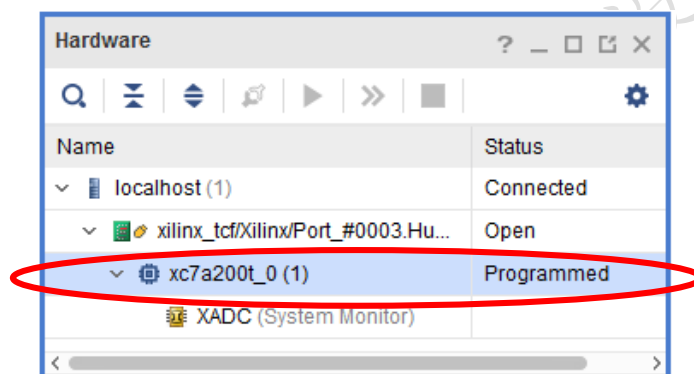


图 7-54 完成 FPGA 编程

FPGA 开发板上，8 个拨码开关 SW18~SW25 对应控制 8 个 led 灯 LED1~LED8 的亮灭。设计完成。

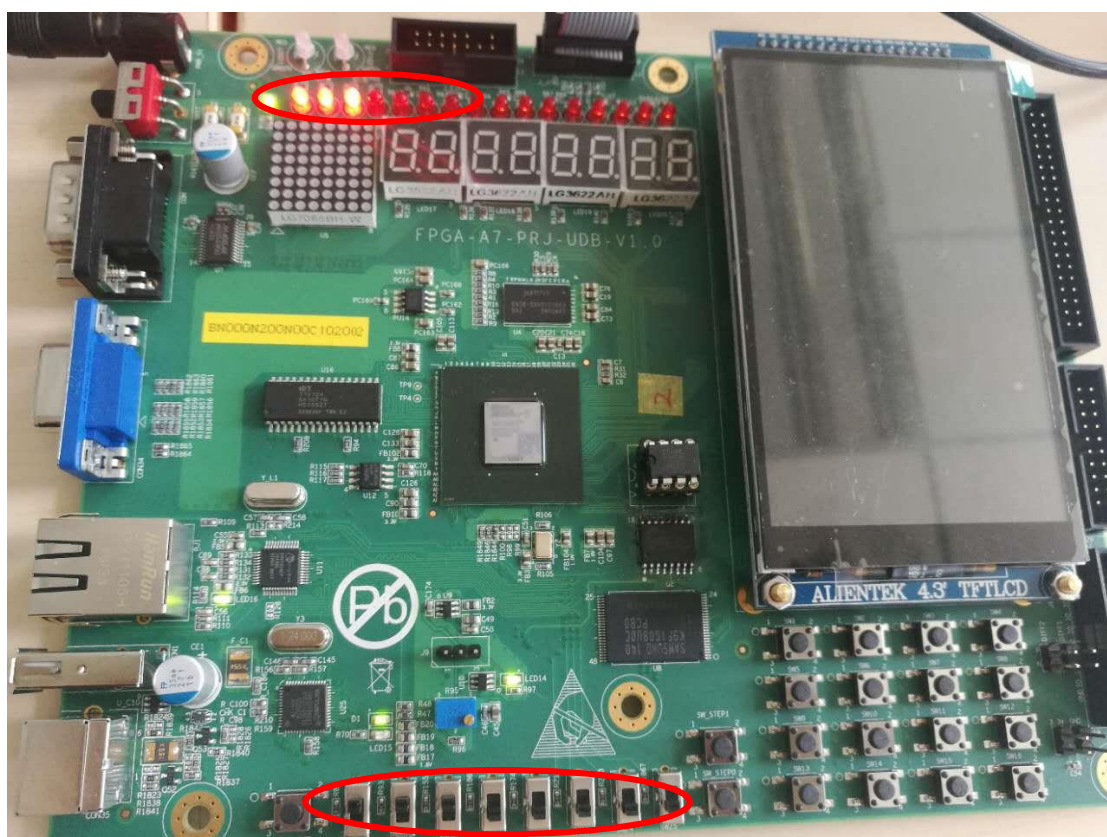


图 7-55 FPGA 运行结果

8 附录四：“计算机组成原理课程设计” MIPS 指令系统规范

参考本目录下 Appendix 目录下的文档 A02 和 A04。

北京科技大学计算机组成原理课程设计19-20秋季

9 附录五：实验报告模板

参考本目录下 Appendix 目录下 A07 文档。

北京科技大学计算机组成原理课程设计19-20秋季

10 附录六：交叉编译工具链安装

在学习并尝试本章节前，你需要具有以下环境：

- (1) Windows 或 Linux 操作系统的电脑一台。
- (2) 连通的网络。

通过本章节的学习，你将获得：

- (1) Windows 下虚拟机 VirtualBox 的安装包和 Ubuntu 镜像。
- (2) MIPS 交叉编译工具链的安装。

10.1 交叉编译环境搭建

10.1.1 Windows 下安装

首先在 windows 下安装虚拟机，分享一个 VirtualBox 的安装包和一个 Ubuntu 虚拟机镜像。网盘下载链接：<https://pan.baidu.com/s/10PRHAGAA1770ZTkTAexXqw> 密码：lmnp。其中 VirtualBox-5.1.16-113841-Win.exe 为 VirtualBox 安装程序，ubuntu16.04_vbox.zip 为虚拟机镜像。该

其次为虚拟机里启动 Linux 镜像，安装 GCC 交叉编译工具链，同 10.1.2 节。

注意：如果使用上述提供的下载链接里的 Ubuntu 镜像，则已包含上述 gcc-4.3-ls232 交叉编译工具链，无需再安装。

10.1.2 Linux 下安装

GCC4.3，下载地址：<http://ftp.loongnix.org/toolchain/gcc/release/gcc-4.3-ls232.tar.gz>

下载后解压，安装步骤如下：

在终端下进入 gcc-4.3-ls232.tar.gz 所在目录，运行：

```
[abc@www ~]$ sudo tar -zxvf gcc-4.3-ls232.tar.gz -C /
```

执行后，请确保目录 “/opt/gcc-4.3-ls232/bin” 存在，随后执行：

```
[abc@www ~]$ echo "export PATH=/opt/gcc-4.3-ls232/bin:$PATH" >> ~/.bashrc
```

对于 64 位系统，还要安装 lsb-core：

```
[abc@www ~]$ sudo apt-get install lsb-core
```

完成上述工作后如果可以输入 mipsel-linux-gcc -v 命令，如果可以正确查看版本号则说明配置正确。

11 附录七：乘除法器 IP 定制

在学习并尝试本章节前，你需要具有以下环境和指令：

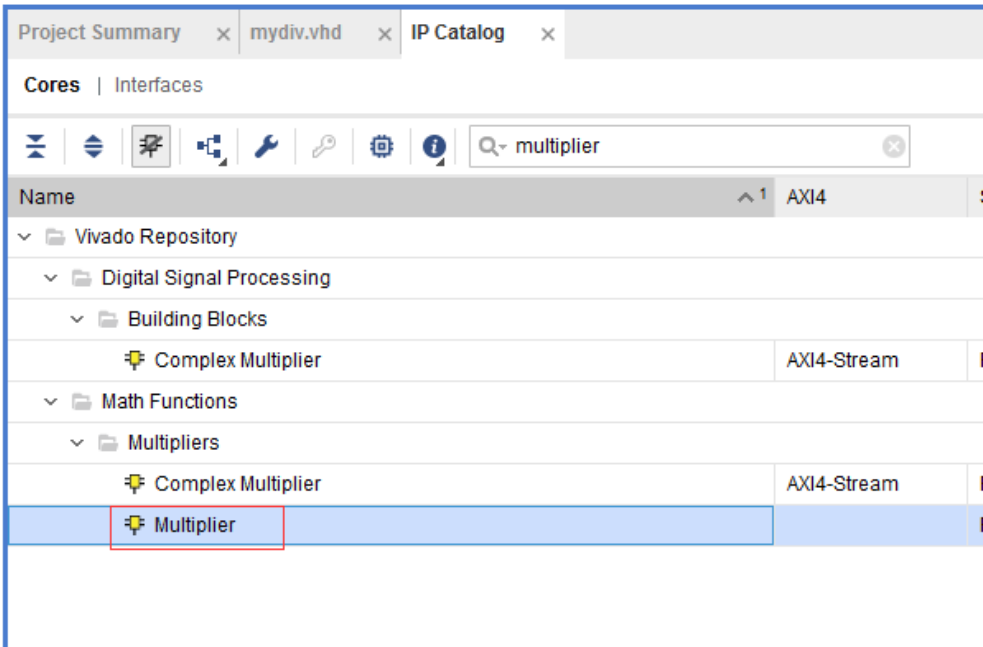
- (1) Vivado 环境。
- (2) Vivado 定制 IP 的方法。

通过本章节的学习，你将获得：

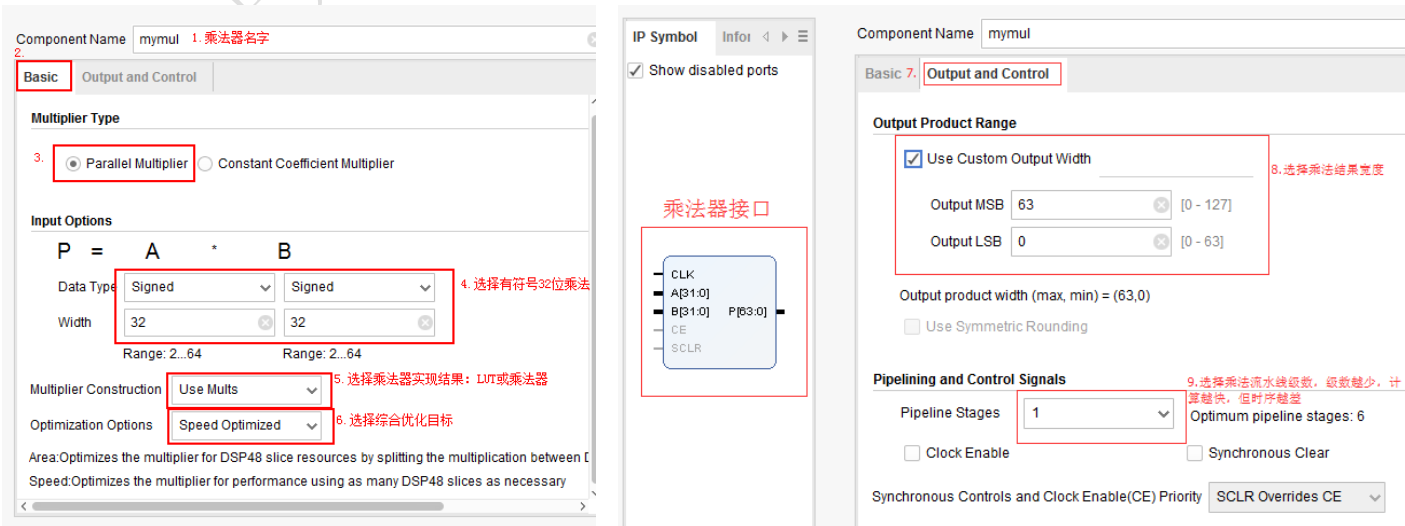
- (1) Vivado 定制乘法器 IP 的方法。
- (2) Vivado 定制除法器 IP 的方法。

11.1 乘法器 IP 定制

最简单的，可以直接写乘号“*”，会自动调用乘法器 IP。也可以根据需求定制指定属性的乘法器 IP。打开 Vivado 工程，在左侧点击“IP Catalog”，并搜索 multiplier，如下图：



双击“Multiplier”，打开乘法器设置界面，如下：



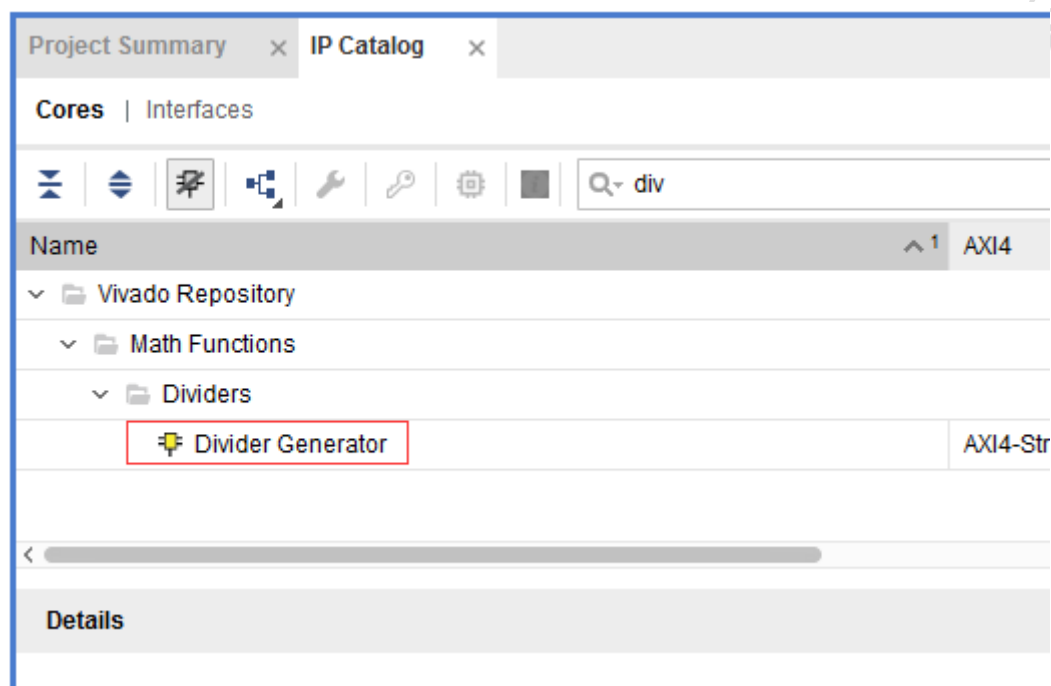
其中，第 1、5、6、9 点标注可以进行自定义修改，但其他几点标注，建议不要修改。关于第 9 点标注，如果

选为乘法器为 1 级流水，可能时序偏差些，可以根据实际情况和自己的设计需求选择。

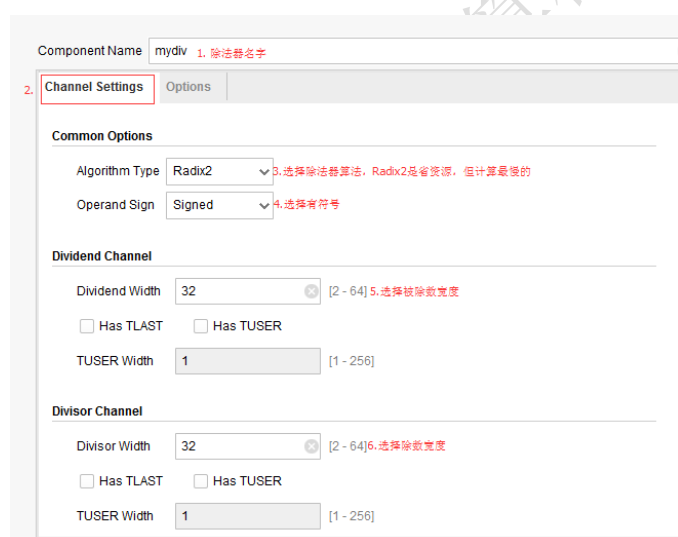
在上图右侧，给出了乘法器接口，信号的意义是很明显的，就不再赘述。需要注意的是，如果乘法器设置为 1 级流水的，则说明乘法一拍完成，也就是在第 n 拍送入 A 和 B ，那么在第 n 拍就能获得乘积 P ；如果乘法器是设置为 2 级流水的，则在第 n 拍送入 A 和 B ，需要在第 $n+1$ 拍才能获得乘积 P 。所以，大家需要根据自己定制的乘法的流水级级数，确定自己送入 A 、 B ，和取回乘积的时机。

11.2 除法器 IP 定制

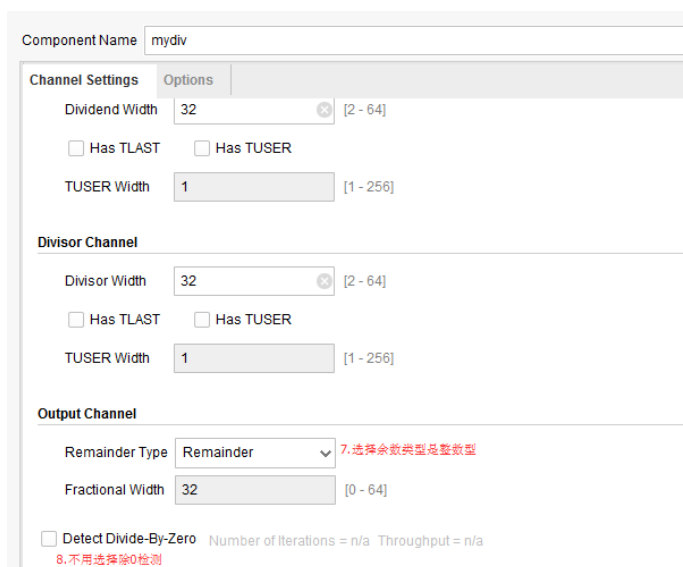
打开 Vivado 工程，在左侧点击“IP Catalog”，并搜索 div，如下图：



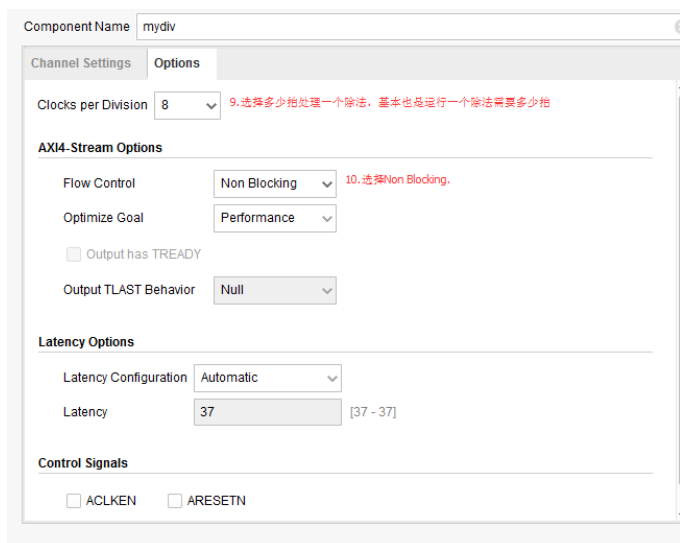
双击“Divider Generator”，打开除法器设置界面，如下：



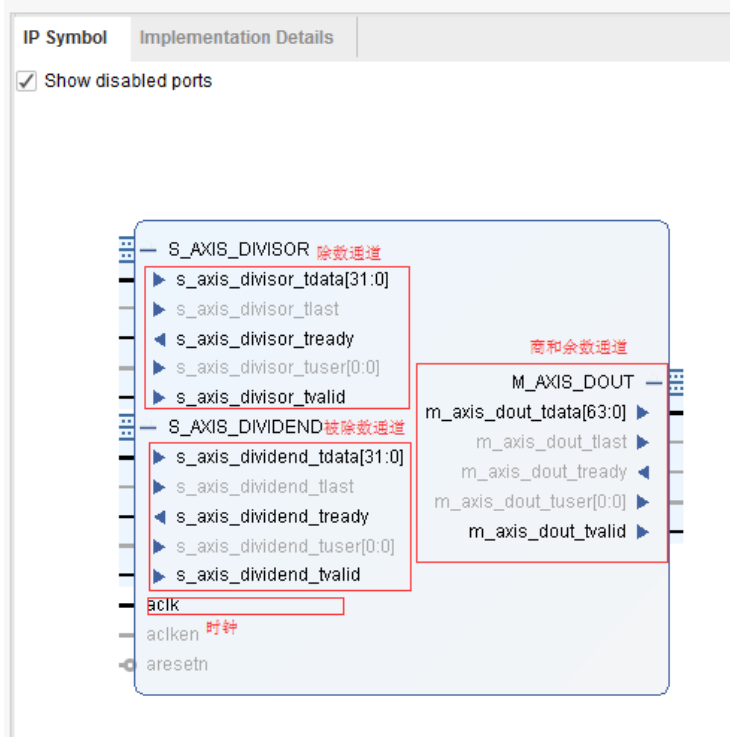
(1)



(2)



(3)



(4)

上 4 幅图中第 1、3、9 点标注是可以进行自定义更改的，但其他几点标注，建议不要修改。其中第 4 点标注，选择是有符号除法，是建议大家将无符号除法转换为有符号除法。

上述第 4 副图中，列出了除法器生成的端口，注意箭头方向代表了信号方向是 input 还是 output。

在本次实验调用时，应该被除数和除数同时给，被除数和除数被该 IP 接受是通过一对握手信号 tvalid 和 tready 完成的。当 CPU 需要向该 IP 传递除法命令时，将被除数送到 s_axis_dividend_tdata，除数送到 s_axis_divisor_tdata，同时置上 s_axis_dividend_tvalid 和 s_axis_divisor_tvalid。随后发现 s_axis_dividend_tready 为 1 则立马撤掉 s_axis_dividend_tvalid，类似的，发现 s_axis_divisor_tready 为 1，则立马撤掉 s_axis_divisor_tvalid。一次除法传输，必须确保两组 tvalid 和 tready 同时为 1（握手成功），有且只有一个 clock，否则等于传递了多次除法命令。

当传递除法完成后，需要等待除法计算完成，在收到 m_axis_dout_tvalid 时，表示除法计算完成，m_axis_dout_tdata[63:32]为商，m_axis_dout_tdata[31:0]为余数。在等待除法完成时，其实是可以传递下一个除法，因为除法 IP 是流水完成的。

12 附录八：SoC_up 使用环境说明

12.1 基于 GS232 搭建的 SoC_up 说明

12.1.1 GS232 开源版本简介

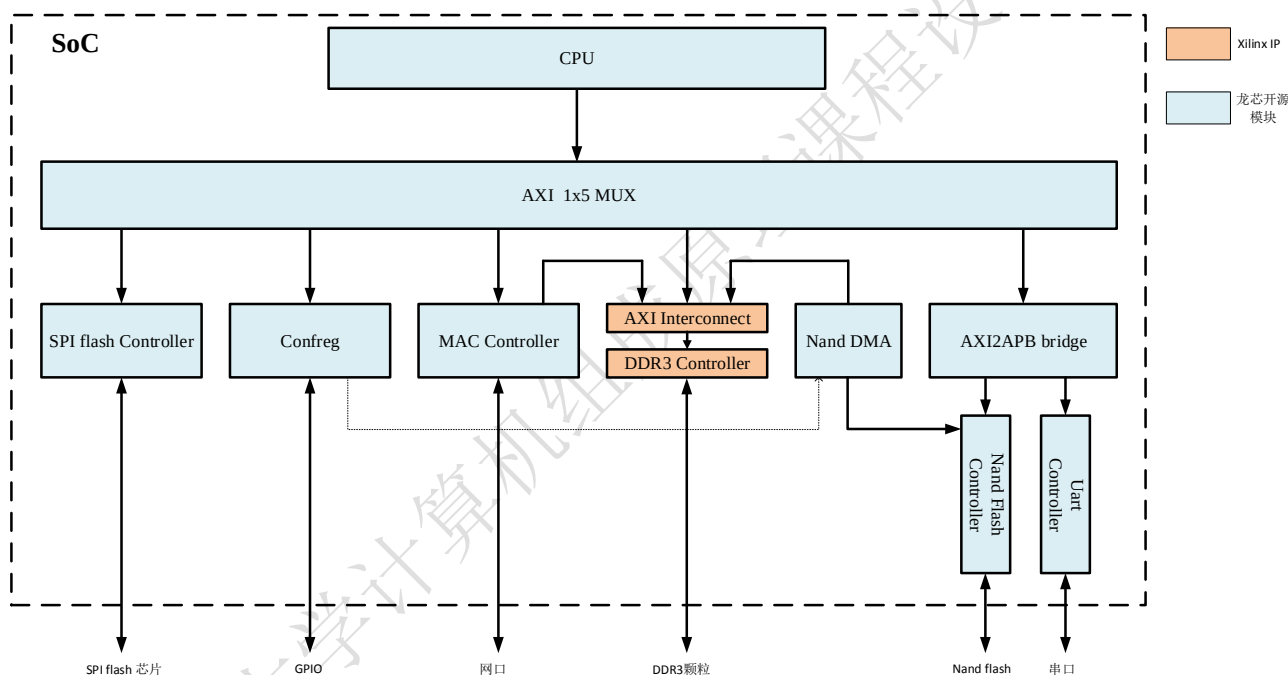
GS232 开源版本不包含 DSP、浮点部件等。

TLB 大小为 32 项。

指令和数据 Cache 为 4 路组相连，每路大小为 4KB，Cache 行大小为 32 bytes。

对外接口为 32 位 AXI 接口。

12.1.2 SoC_up 结构



SoC_up 如上图所示。开源 LS232 对外有一个 AXI 接口，连接到 AXI 互连网络上与外设相连。

SoC_up 对外连接的设备共有 6 个：SPI flash、GPIO(数码管、LED 灯、开关灯)、网口、DDR3 颗粒、Nand flash 和串口。这些外设在教学实验板上均已集成。

12.1.3 地址空间分配

各外设地址空间分配如下：

各控制器模块名	分配的虚拟地址段	地址空间大小
SPI flash	0xbfc0_0000_0xbfcf_ffff 和 0xbfe4_0000~0xbfe4_ffff	1MB(flash 存储空间) 64KB(控制器寄存器空间)
GPIO	0xbfd0_0000~0xbfd0_ffff	64KB
MAC	0xbff0_0000~0xbff0_ffff	64KB

DDR3	4GB 空间中的剩余地址 ¹	128MB
Nand flash	0xbfe7_8000~0xbfe7_bfff	16KB
Uart	0xbfe4_0000~0xbfe4_3fff	16KB
地址空洞 (软件可以不关注)	0xbfe7_0000~0xbfe7_7fff 0xbfe7_c000~0xbfe7_ffff 0xbfe4_4000~0xbfe4_ffff	分配给 APB 设备的，但由于 APB 设备只有 uart 和 nand，故存在较多的地址空洞，留作后续新增 APB 设备使用。

12.1.4 NAND DMA 控制器

其一端通过 64 位 AXI 接口接到 DDR3 内存上，一端通过 APB 接口接到 APB 设备上（可认为接到 NAND 控制器上）。

该 DMA 只用于 nand flash 与内存交换数据。

该 DMA 的配置寄存器 ORDER_ADDR_IN 位于 CONFREG 模块，地址还是为 0xbfd0_1160。

12.1.5 NAND FLASH 控制器

通过 APB 接口接在 APB 桥上。

该 NAND FLASH 控制器不支持上电从 flash 启动和校验纠错。

FPGA 板上 NAND FLASH 颗粒 K9F1G08U0C-PCB0 的 main 区容量为：1K blocks * 64 pages/block * 2K Bytes/page = 128M bytes。也就是共有 64k 页，一页为 2k bytes。每页的 spare 区为 64bytes。

12.1.6 CONFREG 模块

包含 8 个 32 位内存映射读写寄存器和一个 dma 的 order_addr_in 寄存器（0xbfd0_1160）。

12.1.7 MAC 控制器

一个从端 32 位 AXI 接口，接到 AXI 互网络上供 CPU 访问。一个主端 32 位 AXI 接口，接到 DDR3 内存上，自带 DMA 功能。

12.1.8 DDR3 控制器

为 Xilinx IP。

不需要软件配置。一上电，整个 SoC 会先等 DDR3 完成复位，才会撤掉 CPU 的复位信号。

其实分为，一个 3x1 的 AXI 仲裁器，和一个 DDR3 控制器，均为 XilinxIP。

其中 3x1 的 AXI 仲裁器为 2 个 32 位 AXI 接口：一个接 AXI 互网络供 CPU 访问，一个接 MAC 控制器供网口访问；还有一个 64 位 AXI 接口供 NAND DMA 访问。

12.1.1 SPI FLASH 控制器

通过一个 32 位 AXI 接口接到 AXI 互网络上。

¹ 剩余地址：指 4GB 地址空间中，除了其他外设分配的地址外的地址。由于实验板上 DDR3 颗粒大小为 1Gb，即 128MB，故软件访问时注意不要越界，建议软件使用虚拟地址 0x8000_0000~0x87ff_ffff 对其进行访问。

12.1.2 UART 控制器

仅支持一个串口设备。

12.1.3 中断连接

由于教学 SoC 比较小，故没有集成中断控制器，也就只实现 1 级中断。

其中 6 个硬件中断（CauseIP7-2）高位未连接外部中断，其余 5 位硬件中断和外设连接关系如下。

硬件中断名	来源
CauseIP6	dma_int: NAND DMA 的中断请求
CauseIP5	nand_int: NAND FLASH 的中断请求
CauseIP4	spi_int: SPI FLASH 的中断请求
CauseIP3	uart0_int: 串口的中断请求
CauseIP2	mac_int: 网口的中断请求

目前 SoC 的各控制器的中断连接为上表的方案。

12.2 串口软件

12.2.1 Linux 下

(1) 配置

在终端下运行：

```
[abc@www ~]$ minicom -s
```

选择 Serial port setup:

```

+-----[configuration]-----+
| Filenames and paths          |
| File transfer protocols      |
| Serial port setup            |
| Modem and dialing            |
| Screen and keyboard          |
| Save setup as dfl             |
| Save setup as..              |
| Exit                         |
+-----+

```

进入配置界面：

```

+-----+
| A - Serial Device      : /dev/ttyUSB0 |
| B - Lockfile Location  : /var/lock    |
| C - Callin Program     :              |
| D - Callout Program    :              |
| E - Bps/Par/Bits       : 57600 5N1    |
| F - Hardware Flow Control : No        |
| G - Software Flow Control : No        |
|                          |
| Change which setting?  |
+-----+

```

其中 E 行依据开发板上串口控制器的初始化代码中设置的波特率进行选择，F 和 G 行选择 NO。

配置完成后按 Enter 返回，选择 Save setup as dfl 保存为默认设置。

(2) 运行

将 USB 转串口一端连到电脑上，一端连到串口线上，串口线另一端连接到开发板上串口接口上。在 Linux 终端运行如下命令，开启电脑上的串口界面，开发板即可与电脑进行交换了。

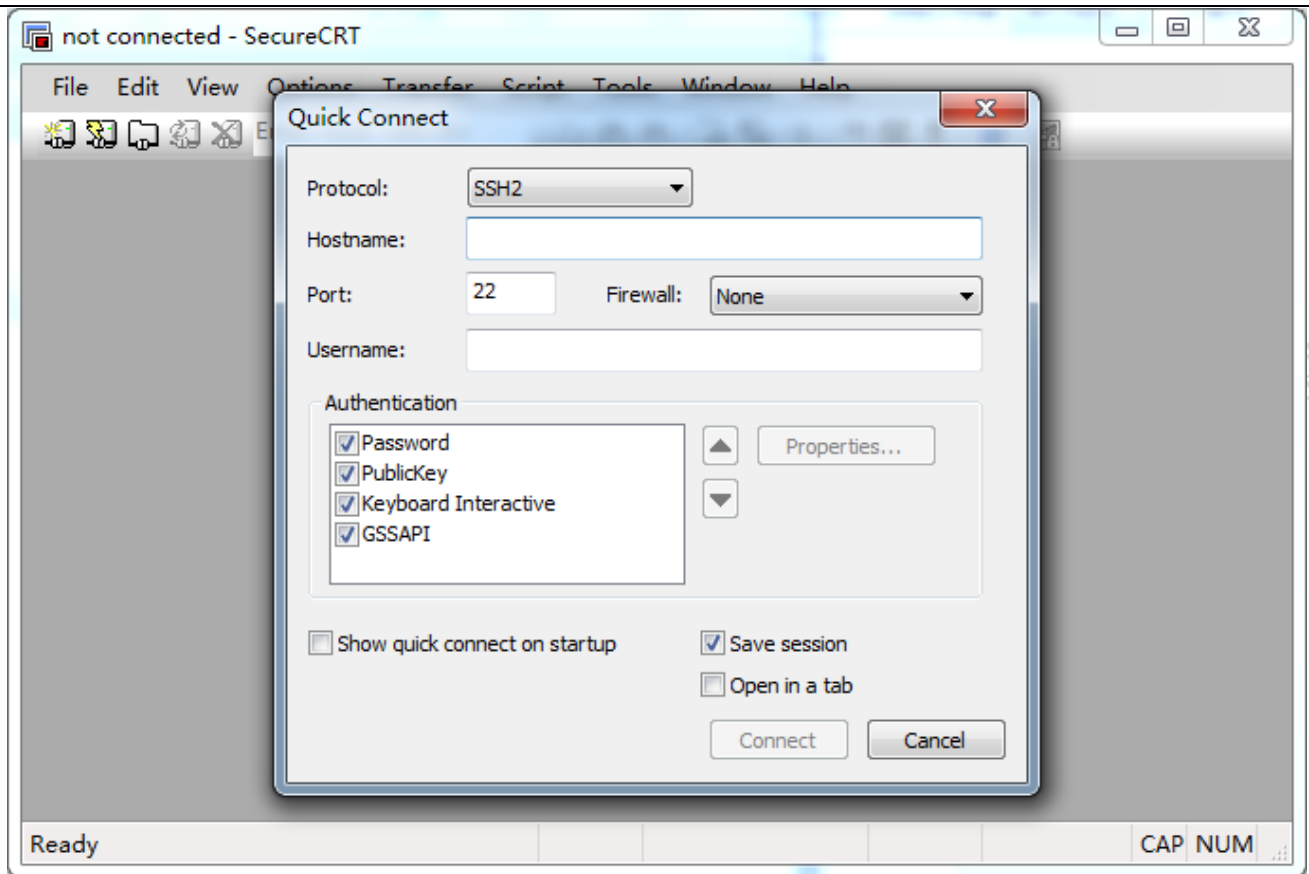
```
[abc@www ~]$ sudo minicom
```

12.2.2 Windows 下

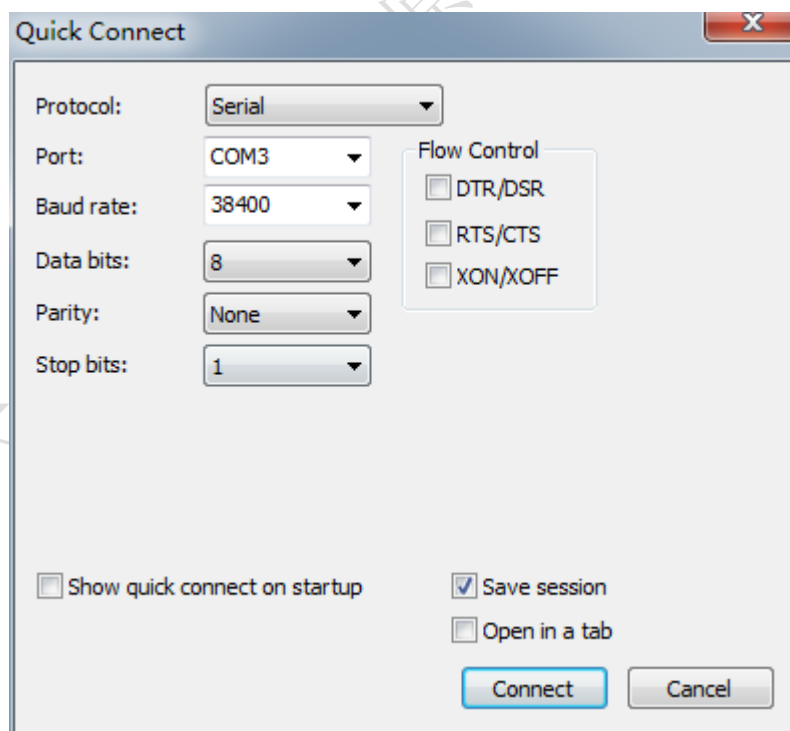
Windows 下可以使用免安装的 SecureCRTPortable 串口软件(见 lab_environment/uart_soft)。

先使用 USB 转串口和串口连接线将电脑和开发板相连。

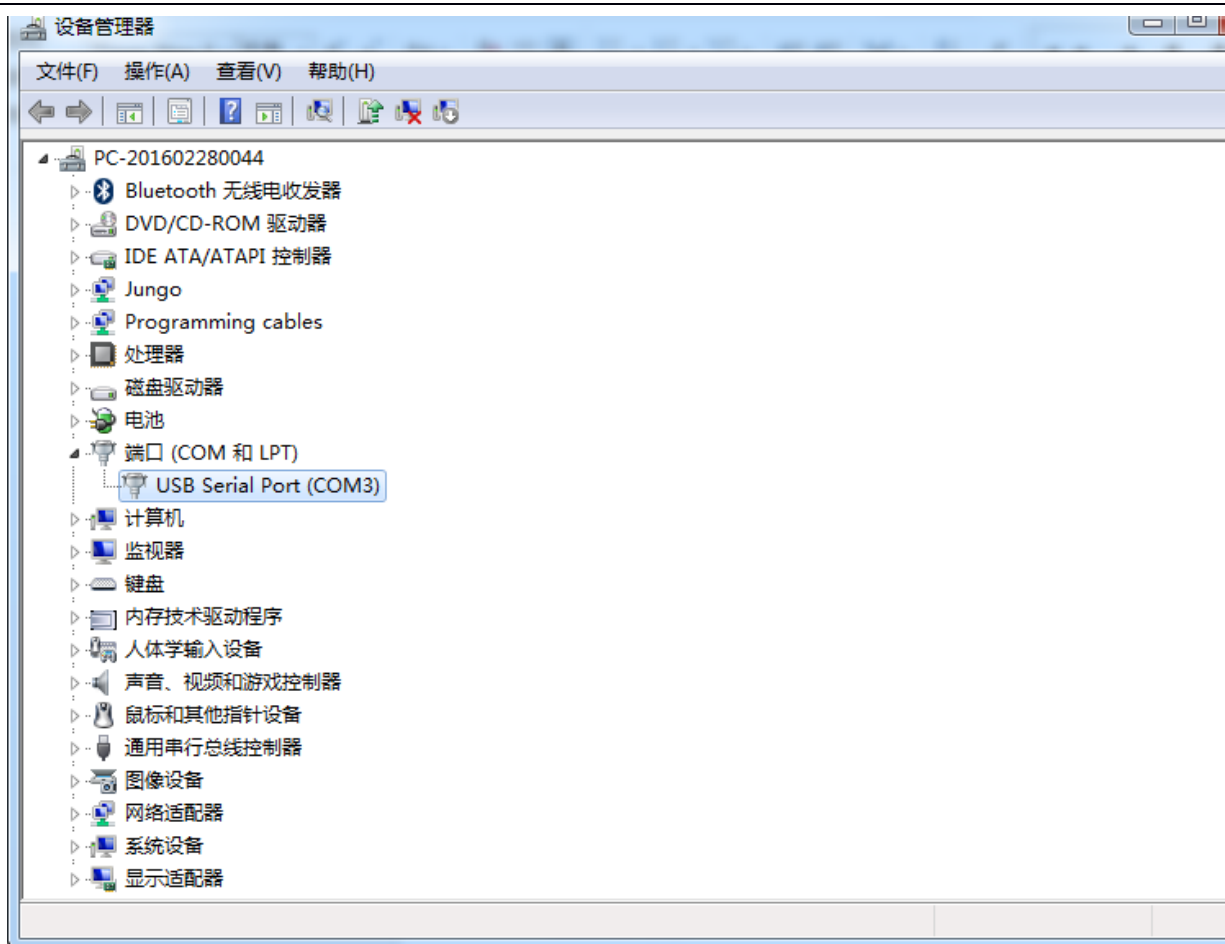
双击程序打开，第一次启动界面如下：



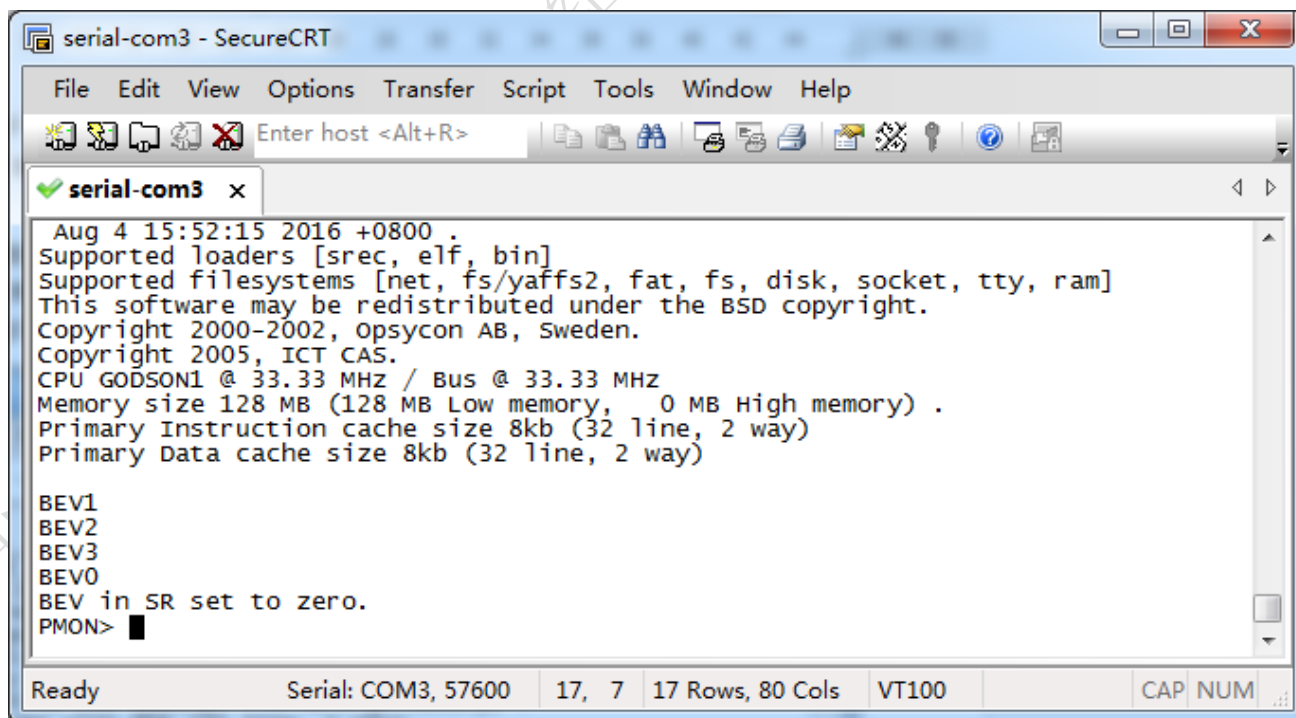
第一行 **Protocol** 下拉选择 **Serial**，如下：



其中 Baud rate 为选择波特率，需根据开发板上串口控制器的初始化代码中设置的波特率进行选择。右侧 **Flow Control** 全不选。Port 的选择需根据 Windows 电脑上的端口进行选择，可以右键电脑选择设备管理器进入**设备管理器**查看：



配置好串口后，点击 **connect**，即可进入串口界面，在波特率设置正确的情况下，可以通过串口与开发板进行交互，如下：



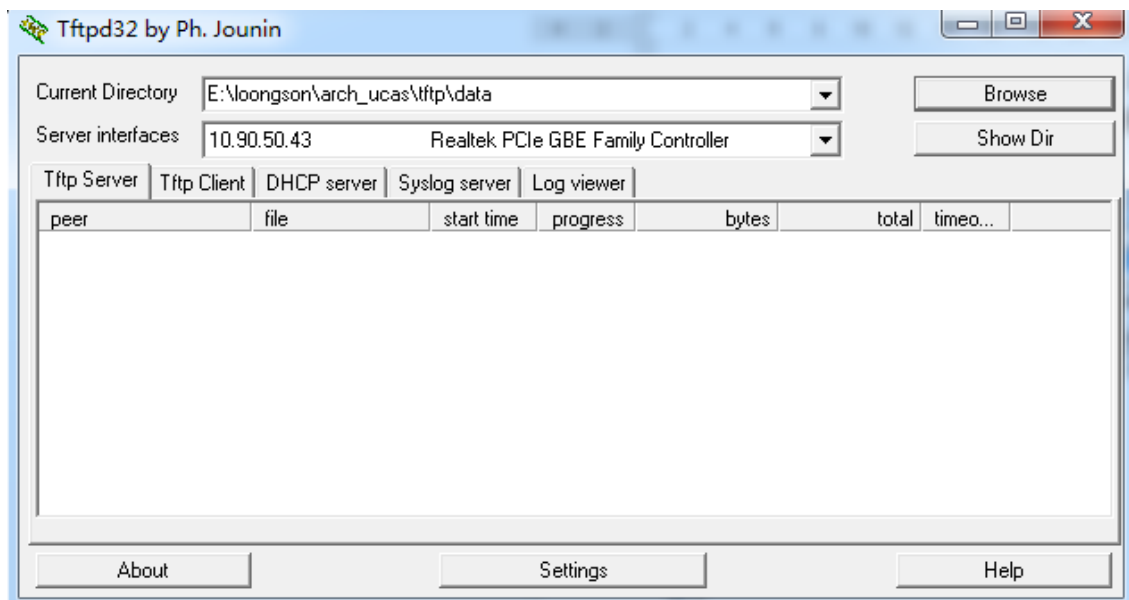
12.3 Tftp 服务器搭建

由于运行 linux 时，最初的内核需要使用网口 load 进入内存执行，因此需要先搭建 Tftp 服务器。

此处只给出 Windows 搭建 tftp 服务器的方法，Linux 下的搭建方法也很简单，请自行学习。

Windows 下可以使用 tftpd32 软件(实验环境包里有该软件)。

双击打开应用程序 tftpd32:



其中 **Current Directory** 为 tftp 服务器的根目录，可以点击 **Browse** 进行更改。点击 **Show Dir** 可以查看该根目录下的文件。

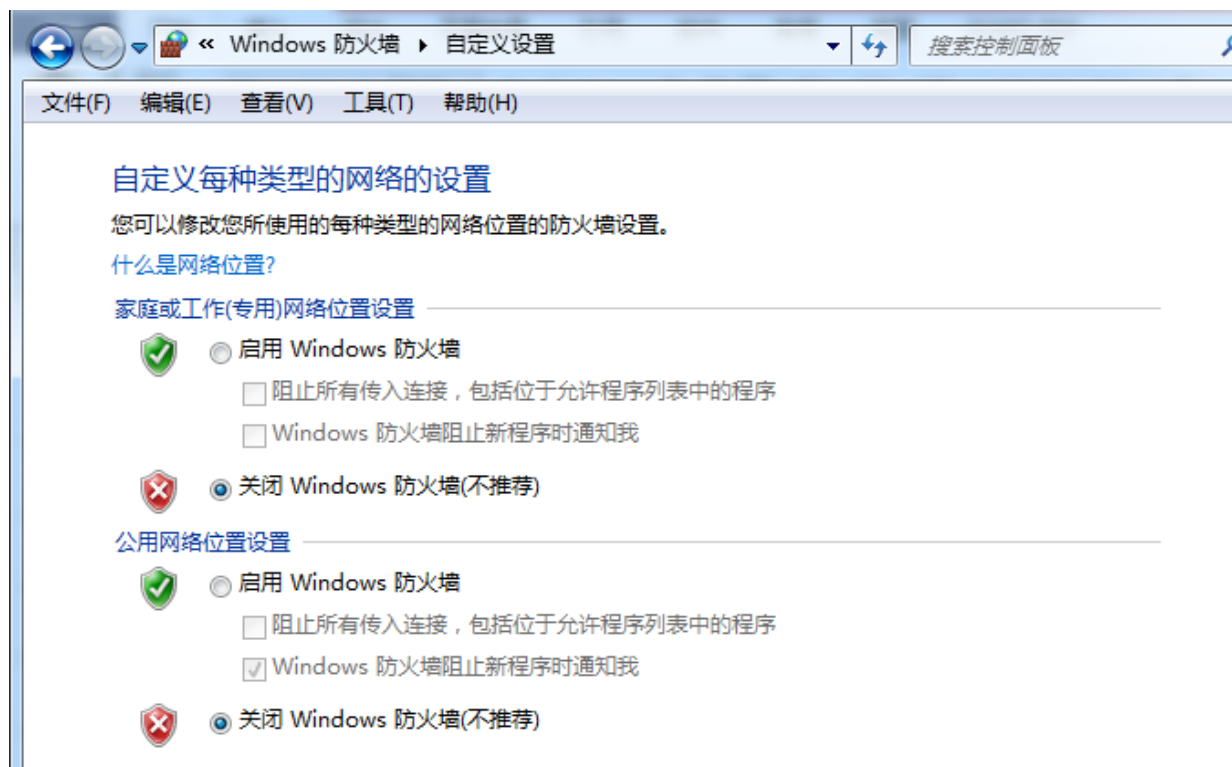
Server interfaces 为选择网卡作为 tftp 服务器的网络入口，可以下拉进行选择，示例中选择了有线网卡接入的 IP: 10.90.50.43。

至此，Windows 上的 tftp 服务器已正常开启了，虚拟机里的 ubuntu 可以正常访问，但局域网里的其他设备还无法访问，需要关闭电脑上的防火墙。

在控制面板中找到 Windows 防火墙，选择“打开或关闭 Windows 防火墙”:



选择关闭 Windows 防火墙即可。



这样同一局域网上的设备就可通过 `tftp://10.90.50.43` 访问电脑上搭建的 tftp 服务器了，可以从根目录下载文件，或上传文件到根目录下。

同样虚拟机里的 ubuntu 可以再终端下输入如下命令，可登录到搭建的 tftp 服务器上：

```
[abc@www ~]$ tftp 10.90.50.43
tftp>
tftp> binary
tftp> put gzrom.bin
Sent 299024 bytes in 0.7 seconds
tftp> get gzrom.bin
Received 299024 bytes in 0.7 seconds
tftp>
```

使用 **put** 命令可以上传一个文件到 tftp 服务器的根目录下，使用 **get** 命令可以下载一个文件到本地。当传输 bin 文件时，需要先输入 **binary** 命令更改传输模式为二进制模式，否则 bin 文件传输后会出错。

12.4 Flash 烧写

目前提供两种可插拔 flash 芯片烧写方法：使用 flash 编程器和使用 FPGA 上编程 bit 流。

推荐大家使用方法二，因为这样不需要插拔 flash 芯片，防止 flash 芯片引脚损坏。

12.4.1 使用 EZP2010 编程器：

需要具有该编程器设备。

使用该编程器，需手动安装驱动。

烧写时，需拔下 Flash 芯片，之后烧写完成，再插入到 FPGA 板上。

12.4.2 使用 FPGA 编程 bit 流

该方法是使用龙芯开源的 gs132 核搭建了一个小的 soc，该 soc 外设有串口、flash 芯片和指令数据 ram。该 soc 在 FPGA 上生成的 bit 流文件可实现通过串口在线编程 flash 芯片。

编程过程中，不需要拔下 flash 芯片，且速率达到 6KB/sec。具体使用方法如下：

(1) 现有一个使用 gcc 编译好的 binary 文件(后缀名为.bin)，准备使用串口将其下载至 FPGA 上的 flash 芯片中。

(2) 准备工具：

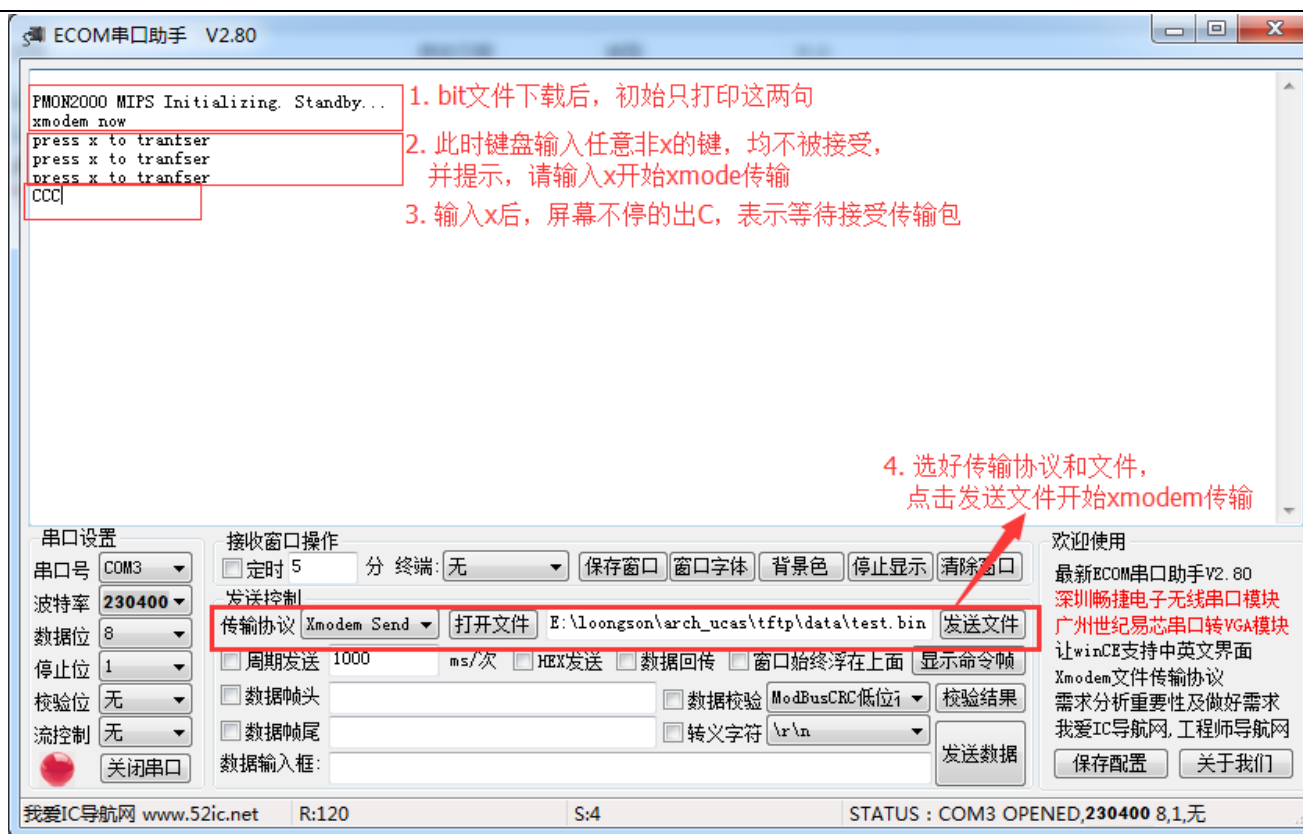
- FPGA 开发板。
- FPGA 电源线。
- FPGA 下载线。
- flash 芯片。
- 串口线（可能还需要 usb 转串口线）。
- xilinx 下载工具。
- 串口软件（ECOM 或 SecureCRT）

(3) 烧写步骤：

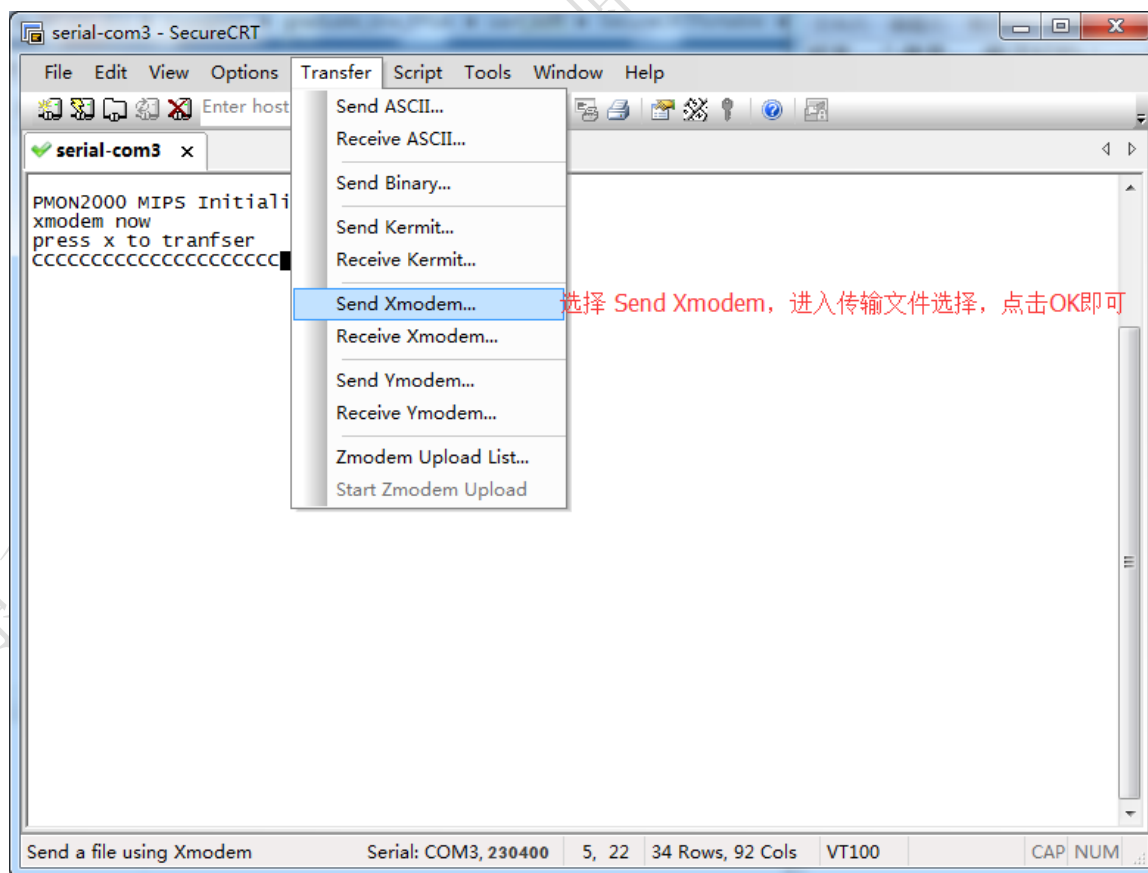
- flash 芯片正确放置 FPGA 开发板上。
- FPGA 开发板与电脑连接下载线、串口线。
- 脑上打开串口软件，波特率选为 230400
- FPGA 板上电，如正常下载 bit 流文件一样下载 programmer_by_uart.bit 至 FPGA 上。
- 确认串口看到打印信息，可以按任意键，根据提示信息，键盘输入 “X” 表示准备开始 xmodem 传输
- 串口软件使用 xmodem 模式传输 bin 文件。等待传输完成。

Linux 环境下可以使用串口进行 xmodem 传输完成 flash 烧写的。

Windows 下，如果串口软件使用的是 ECOM 软件，烧写步骤如下。ECOM 传输 xmodem 的进度会弹出一个界面进行显示，可以看到每个传输包的情况。



如果是串口软件使用的是 SecureCRT 软件，烧写步骤如下。SecureCRT 会在串口界面显示传输百分百。



注意：每次需要烧写 flash 时都需要下载一次 programmer_by_uart.bit 文件，不是很方便。可以考虑将 bit 文件

转换为 mcs 文件，固化到 FPGA 开发板上。固化方法，参见本章 12.5.1 节。

12.5 FPGA 板上 SoC_up 下载

使用 FPGA 开发板运行教学 SoC 时，需要下载对应的 bit 流文件。

目前教学 SoC 在 Artix-7 实验板上运行的频率为 33MHz，DDR3 控制器的工作频率为 400MHz，上升下降沿均采集数据。

下载工具可选：

(1) ISE 工具套装里的 ChipScope。

ise13.2 的 ChipScope 不支持下载 Artix-7 的 bit 流文件，建议使用更高版本的 ChipScope，目前实验了 14.3 版本的是可以用的。

(2) vivado 工具里的 Open Hardware Manager。

12.5.1 Artix-7 教学实验板固化方法

本节给出基于 Artix-7 的教学实验板上固化一个 FPGA 设计的方法。

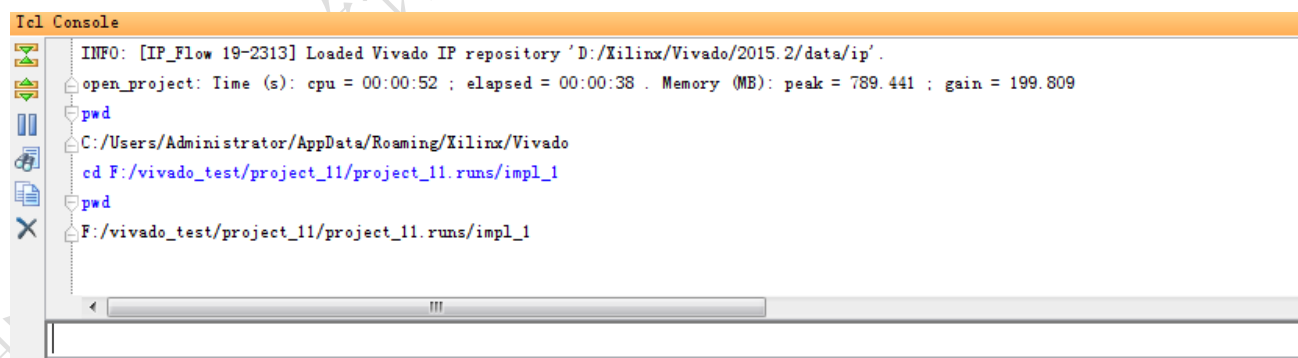
固化后，每次上电时，实验板会自动加载设计到 FPGA 芯片上，因此断电重新上电后不需要重新下载 bit 流文件，极大方便了实验板上软件编程。

固化的流程：先将一个 bit 流文件转换为 mcs 文件，将 mcs 文件下载到板上一个 SPI flash 上。

12.5.2 生成 mcs、prm 文件

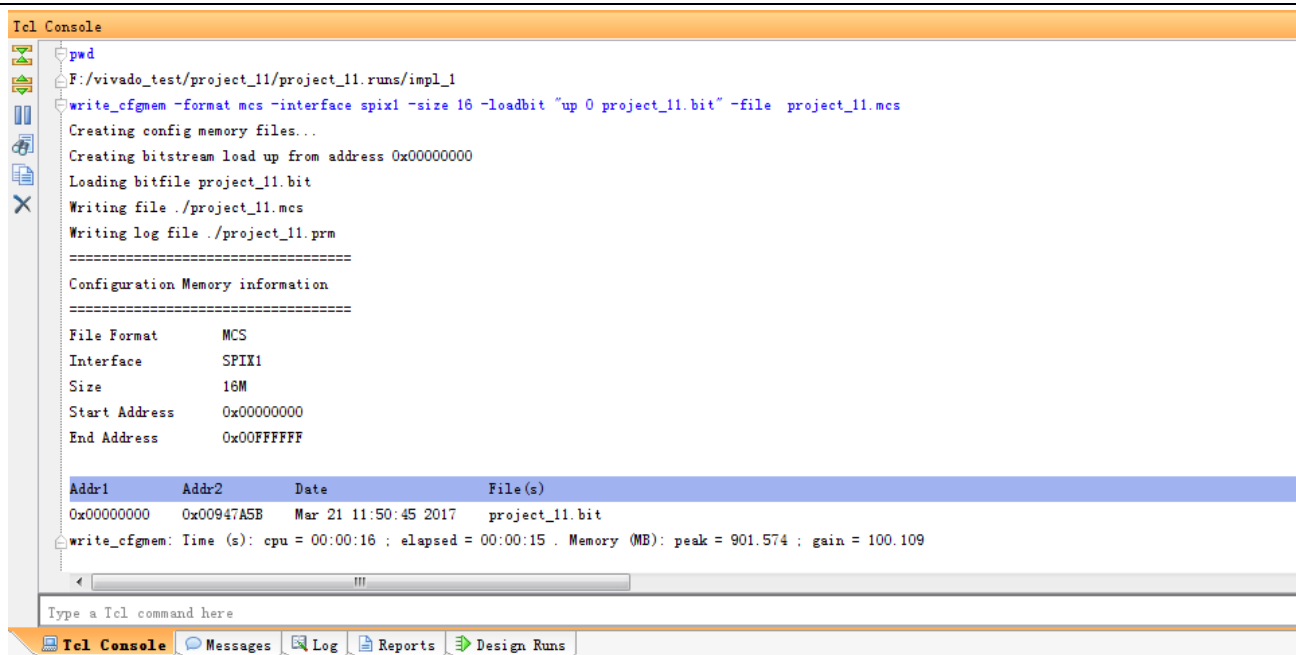
首先，需要确保 FPGA 设计的 bit 流文件已经生成。但 bit 流文件是用于直接下载到 FPGA 芯片里的文件，而不能下载到 flash 芯片里，需要转换为 mcs 文件。

在 ISE 工具里，可以再图形界面下点击选择就可生成 mcs 文件，但在 Vivado 工具里，生成 mcs 文件需要在命令控制器（tcl console）里输入命令。如下图：



上图中蓝色的为输入的命令，pwd 用于查看目录。随后使用 cd 命令进入 bit 流文件所在的目录。

假设生成的 bit 流文件为 project_11.bit，则输入命令串 **write_cfgmem -format mcs -interface spix1 -size 16 -loadbit "up 0 project_11.bit" -file project_11.mcs** 即可生成 mcs 文件，如下图。



其中命令里的 project_11.bit 为待转换的 FPGA 设计的 bit 文件，project_11.mcs 为生成的 mcs 文件名，可以自定义。

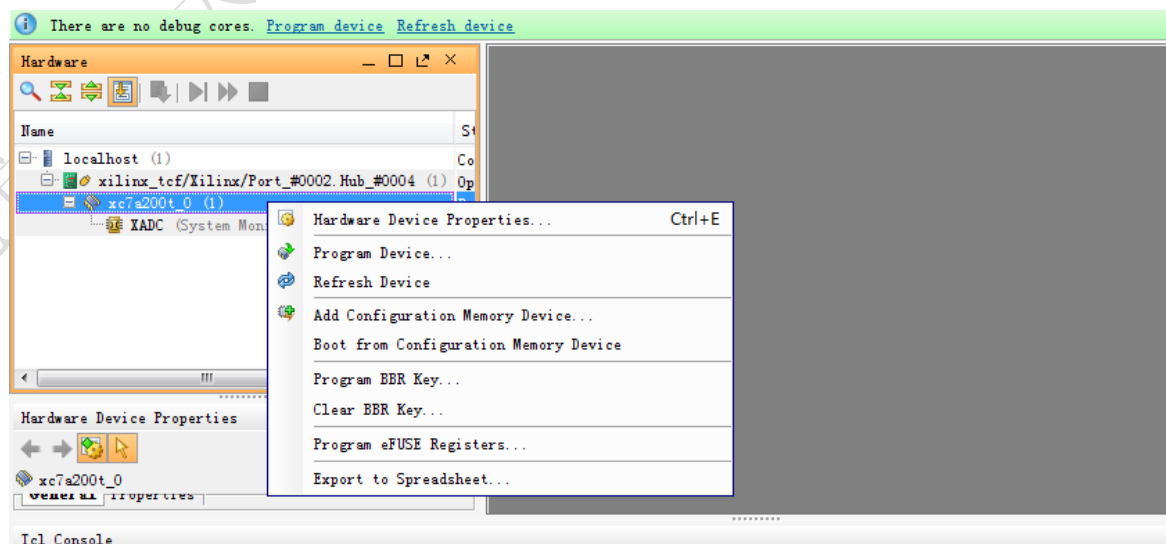
上述命令，是先输入 cd 命令将目录切换到了 bit 流文件的目录，再使用 write_cfgmem 命令将 bit 流文件转换为 mcs 文件。这两步可以使用命令串 **write_cfgmem -format mcs -interface spix1 -size 16 -loadbit "up 0 F:/vivado_test/project_11/project_11.runs/impl_1/project_11.bit" -file F:/vivado_test/project_11/project_11.runs/impl_1/project_11.mcs** 一步到位。这一命令串中明确指定了 bit 流文件的目录，和生成的 mcs 文件的保存目录，bit 流文件的目录和文件名必须正确，但 mcs 文件的目录和文件名可以自定义。

在生成 mcs 的同时，也会生成一个 prm 文件，后续都需要下载。

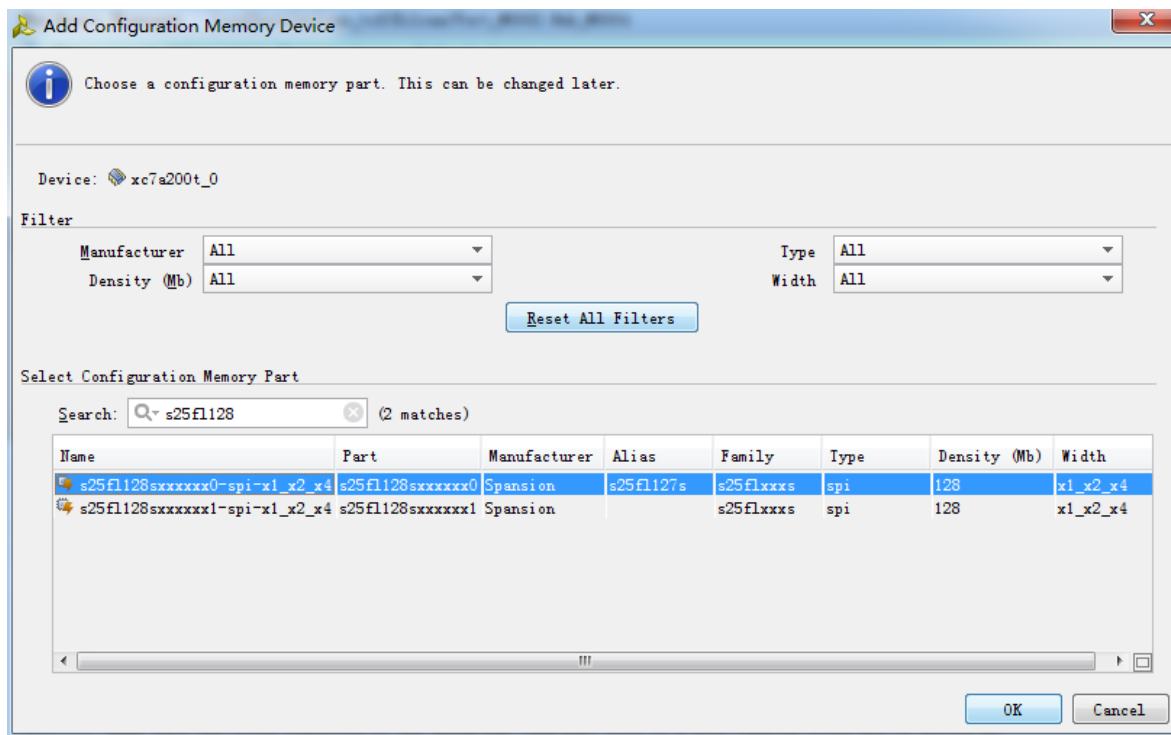
12.5.3 下载 mcs、prm 文件

生成好 mcs、prm 文件后，就需要将它们下载到实验板上 SPI flash 上。

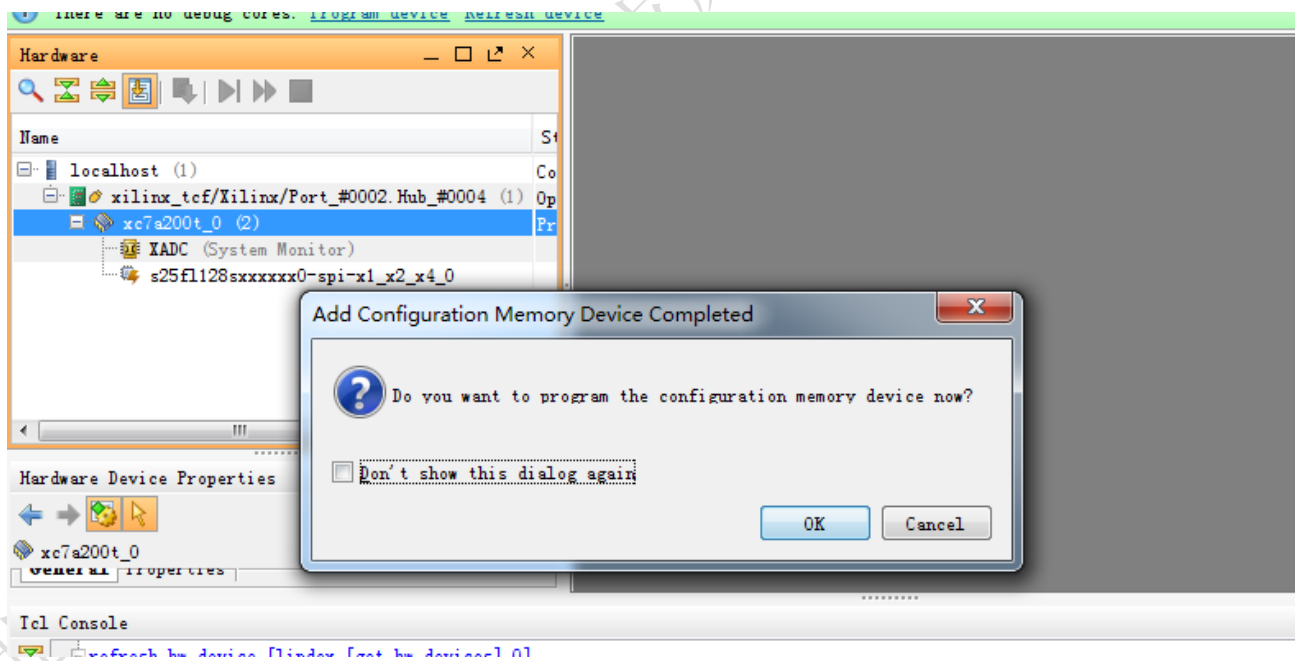
像下载 bit 流文件一样，打开 Vivado 工具里的“Open Hardwar Target”，连接设备。



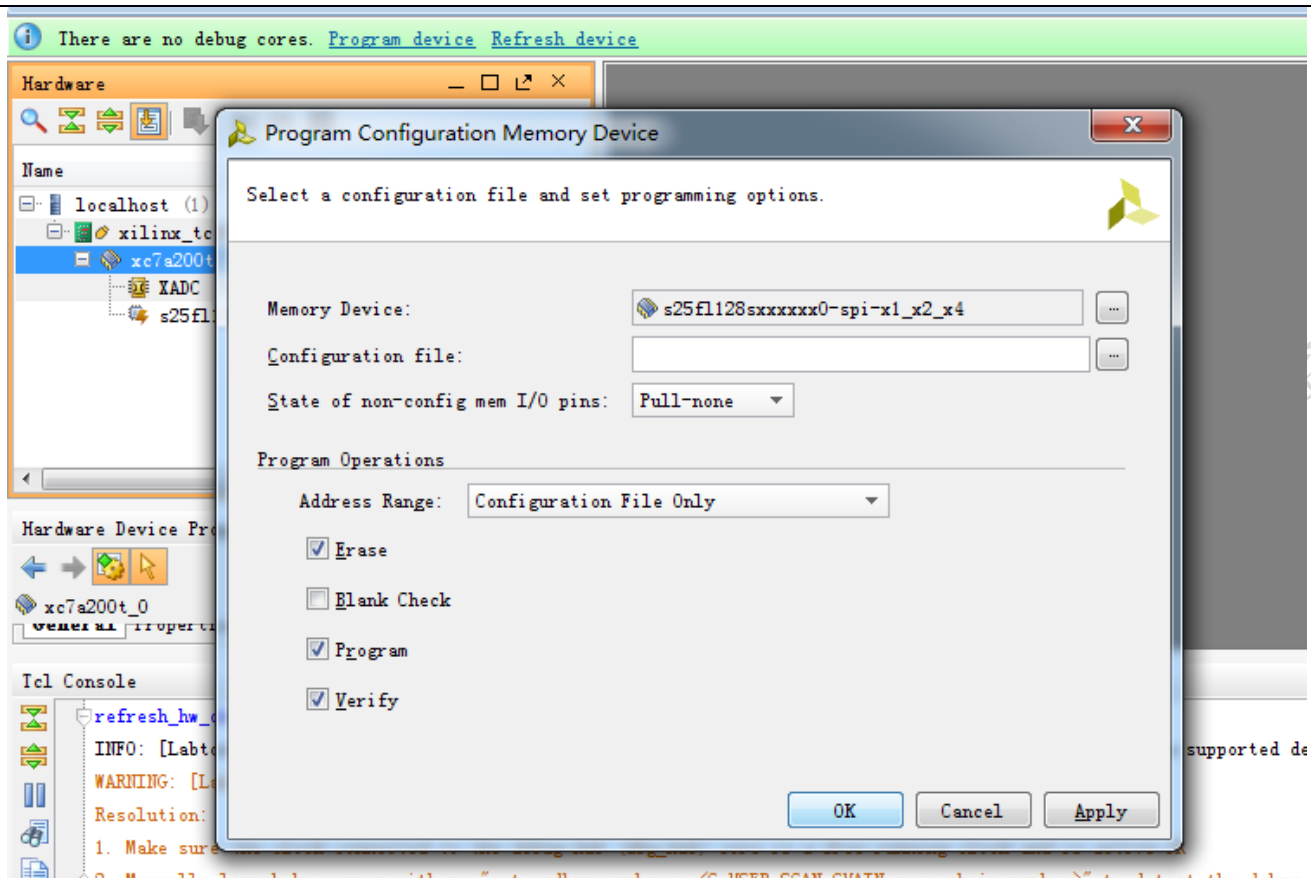
选中 xc7a200t 后，右键选择“Add Configuration Memory Device”，出现如下界面：



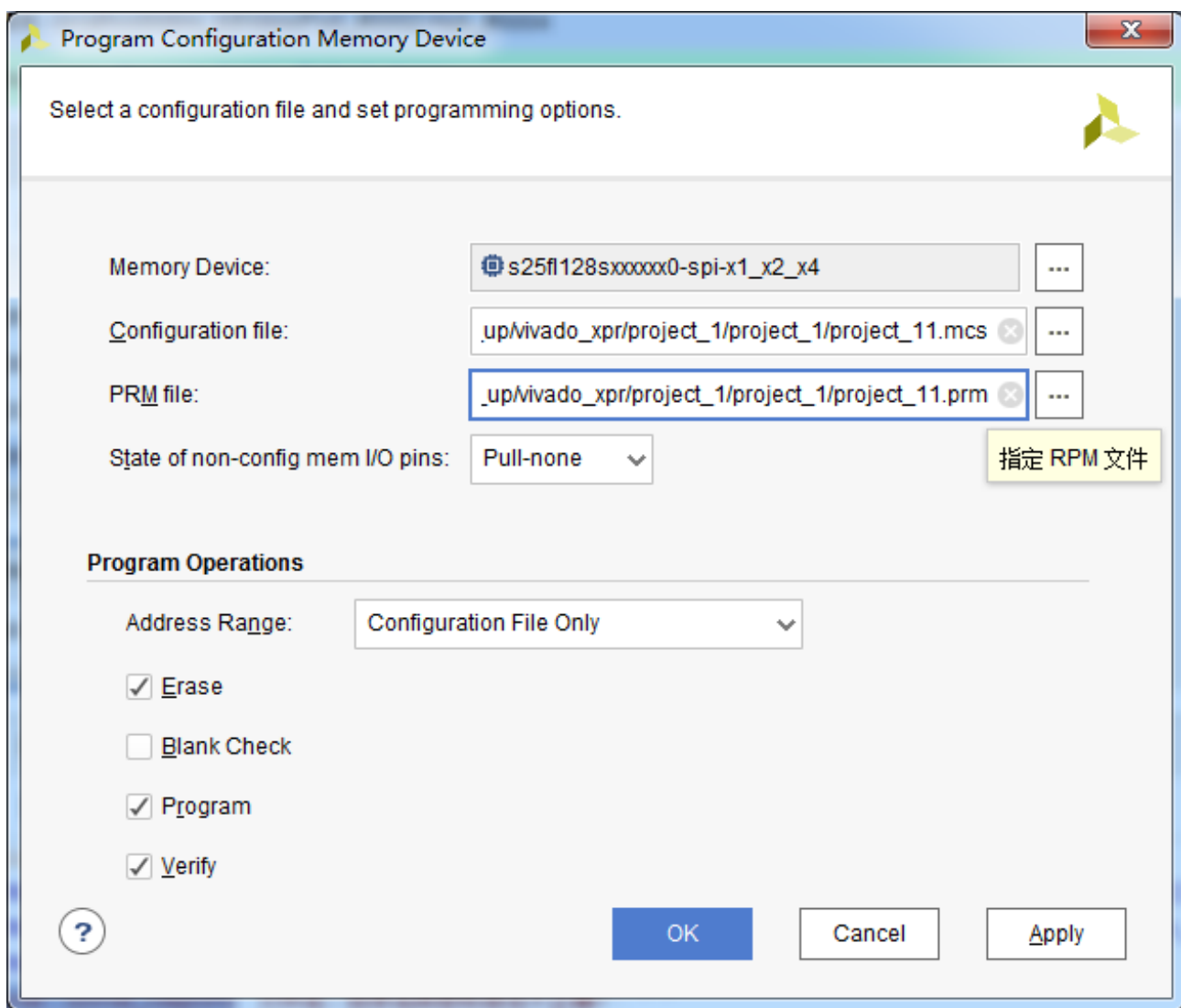
在 search 栏输入 s25fl128，选中出现的第一个 spi flash 芯片“s25fl128sxxxxx0-spi-x1_x2_x4”(需要与板上固定的 flash 芯片型号相同，如果第一个不行，就选第二个 spi flash)，点击 OK，弹出如下窗口，询问是否现在编程 flash：



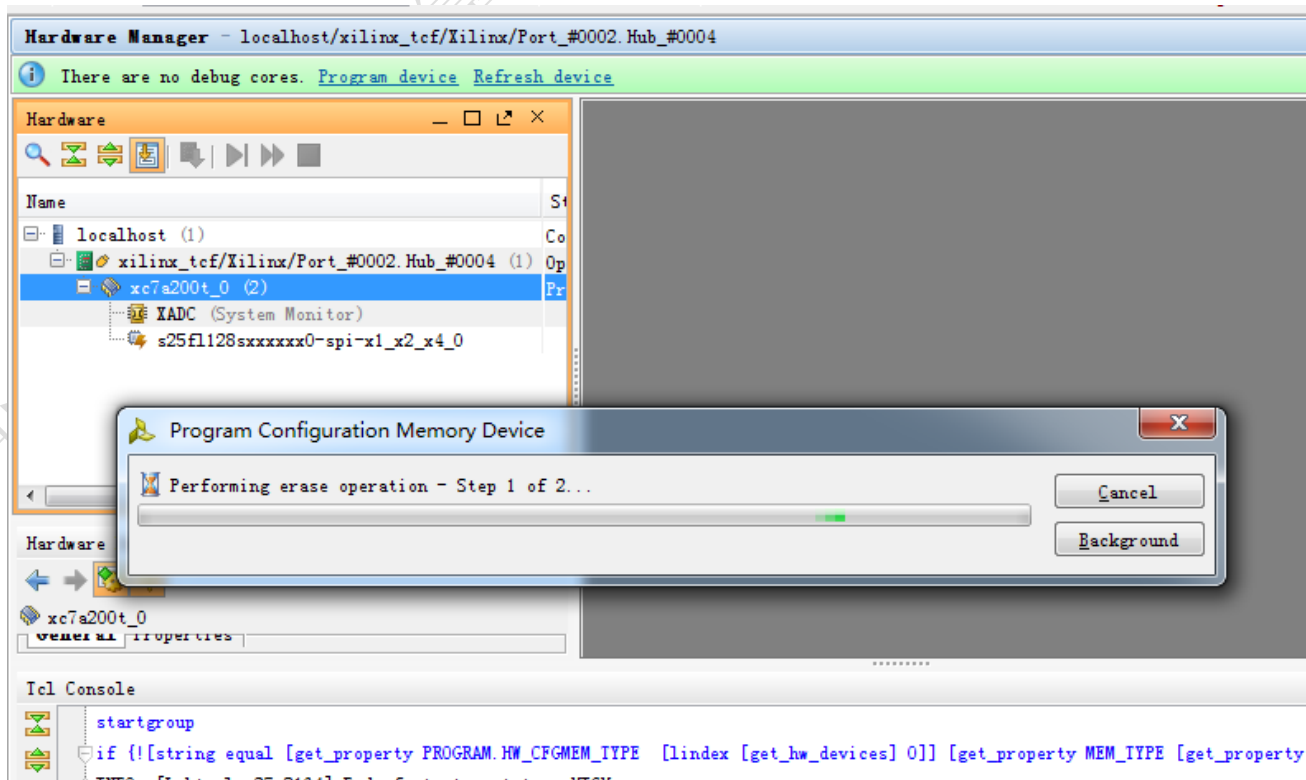
点击 OK，出现编程 flash 的界面：



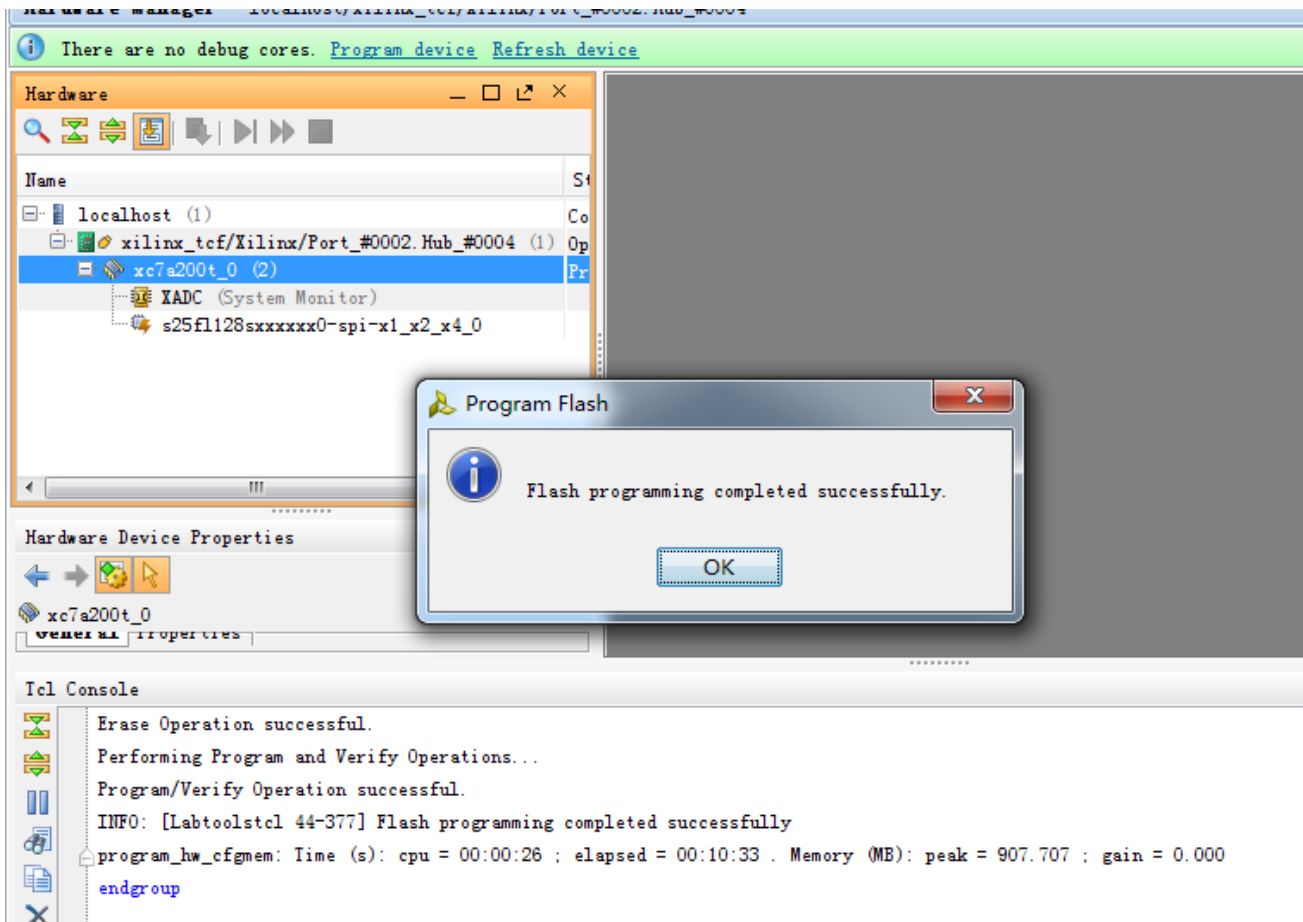
在 Configuration file 那栏选到生成的 mcs 文件，在 PRM file 那栏选到生成的 prm 文件，如下图：



点击 OK 即可。后续等待下载 mcs 到 flash 芯片完成即可，如下图：



Flash 芯片会先进行擦除，在进行编写，完成后会提示 completed Successfully，如下图：



此时烧写就完成了，需要将实验板断电重新上电，等待一段时间，就可发现固化进实验板的 FPGA 设备自动加载完成了。（注意：断点上电后，需要关闭“Open Hardwar Target”，FPGA 才会完成自动加载固化后的设计）

12.6 加载内存到 NandFlash

当前 SoC_up 支持 128MB 的 NandFlash 作为电脑中的硬盘功能。因此可以将 Linux 内核加载到 NandFlash 上。

如果可插拔 SPI 中 Flash 芯片的 PMON 是完善的且已配置后，且实验箱中的 SoC_up_33Mz.bit 也已固化。则下次实验箱上电后，会先自动加载 SoC_up_33M.bit 运行 SoC_up，随后自动运行 PMON 对设备进行初始化，随后 PMON 会自动加载 NandFlash 中的 Linux 内核进行启动，这就是通常电脑启动的过程。

Linux 内核加载至 NandFlash 中并配置 PMON 的方法如下：

- (1) 实验箱运行至 PMON；
- (2) 设置分区空间大小，PMON 命令：set mtdparts nand-flash:50M@0(kernel)ro,-(rootfs)；
- (3) 设置网口 IP，PMON 命令：ifconfig dmfe0 x.x.x.x，其中 x.x.x.x 为配置的 IP；
- (4) 擦除 NandFlash，PMON 命令：mtd_erase /dev/mtd0r

mtd_erase /dev/mtd1r

以上两个命令均要执行一次。

- (5) 拷贝内核文件，PMON 命令：devcp tftp://x.x.x.x /vmlinux /dev/mtd0，其中 x.x.x.x 为搭建的 tftp 服务器的 IP；

(6) 设置启动分区及参数，PMON 命令：

```
set al /dev/mtd0;
```

```
set append "console=ttyS0,115200 rdinit=/sbin/init initcall_debug=1 loglevel=20"
```

(7) 重启 FPGA 实验箱，会自动完成本章开头描述的启动过程，自动运行到 Linux 内核状态：

北京科技大学计算机组成原理课程设计19-20秋季

13 附录九：FPGA 在线调试说明

在 FPGA 开发过程中，经常碰到仿真上板行为不一致，更多时候是仿真正常，上板异常。由于上板调试手段薄弱，导致很难定位错误。这时候可以借助 Xilinx 的下载线进行在线调试，在线调试是在 FPGA 上运行过程中探测预定好的信号，然后通过 USB 编程线缆显示到上位机上。

本文档给出简单使用在线调试的方法：RTL 里设定需探测的信号，综合并建立 Debug core，实现并生产 bit 流文件，下载 bit 流和 debug 文件，上板观察。

13.1 抓取需探测的信号

在 RTL 源码中，给想要抓取的探测信号声明前增加(*mark_debug = "true"*)。

比如，我们想要在 FPGA 板上观察 debug 信号、PC 寄存器和数码管寄存器，需要在代码里这样设置。

```
//debug interface
(*mark_debug = "true"*)output [31:0] debug_wb_pc;
(*mark_debug = "true"*)output [31:0] debug_wb_rf_wen;
(*mark_debug = "true"*)output [4:0] debug_wb_rf_wnum;
(*mark_debug = "true"*)output [31:0] debug_wb_rf_wdata;

wire inst_ex_dib;
(*mark_debug = "true"*)reg [31:0] inst_pc_r;
reg [31:0] inst_code_r;
```

(1) 抓取写回信息

```
reg [31:0] led_rgo_data;
reg [31:0] led_rgl_data;
(*mark_debug = "true"*)reg [31:0] num_data;
wire [31:0] switch_data;
wire [31:0] btn_key_data;
```

(2) 抓取 PC 寄存器

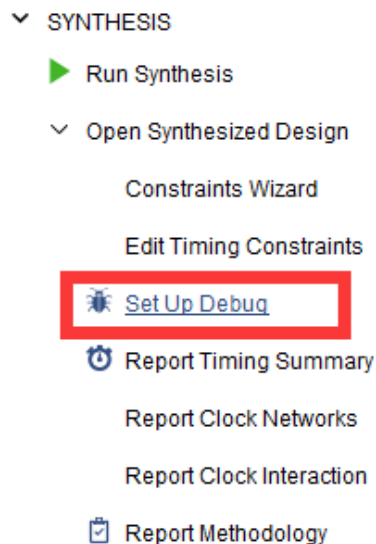
(3) 抓取数码管寄存器

设定完成后，就可以运行综合了。

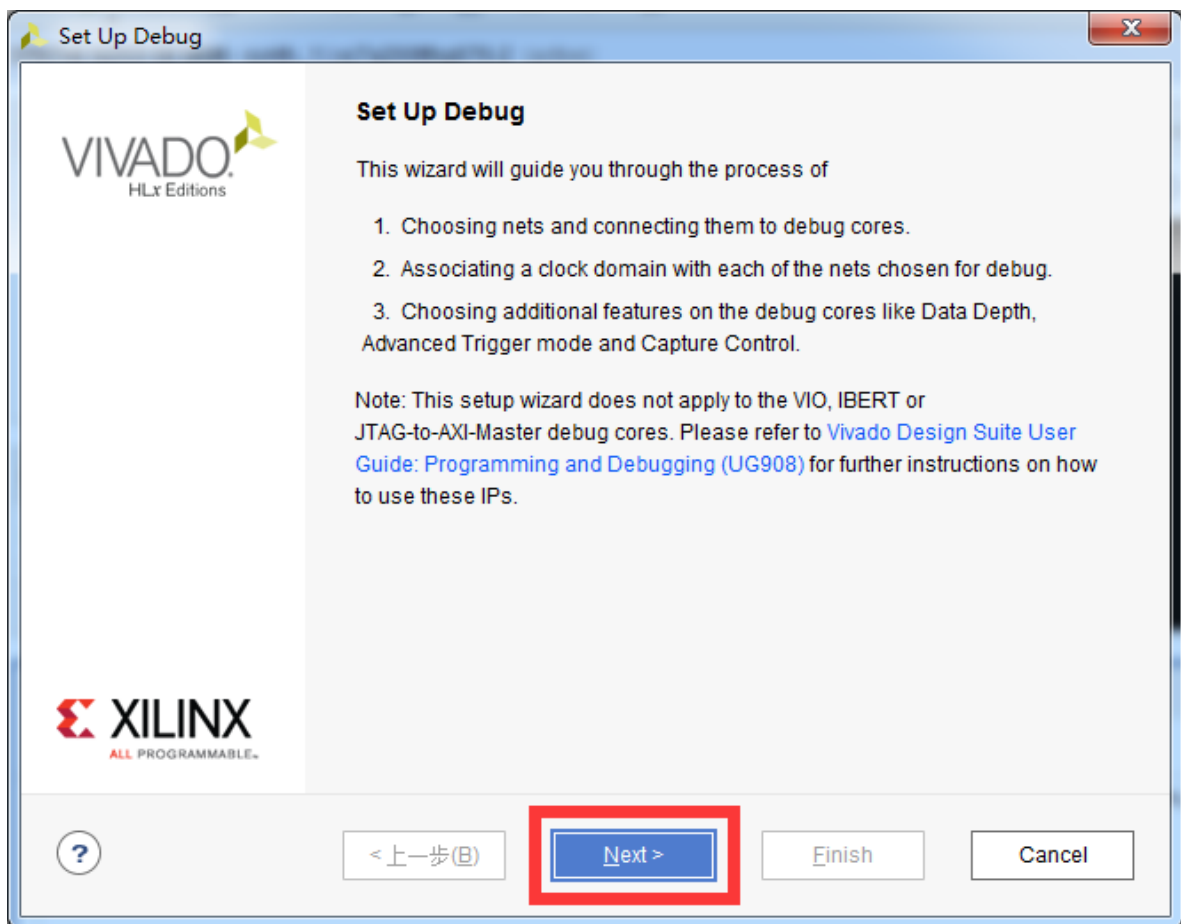
13.2 综合并建立 debug

在综合完成后，需建立 debug。

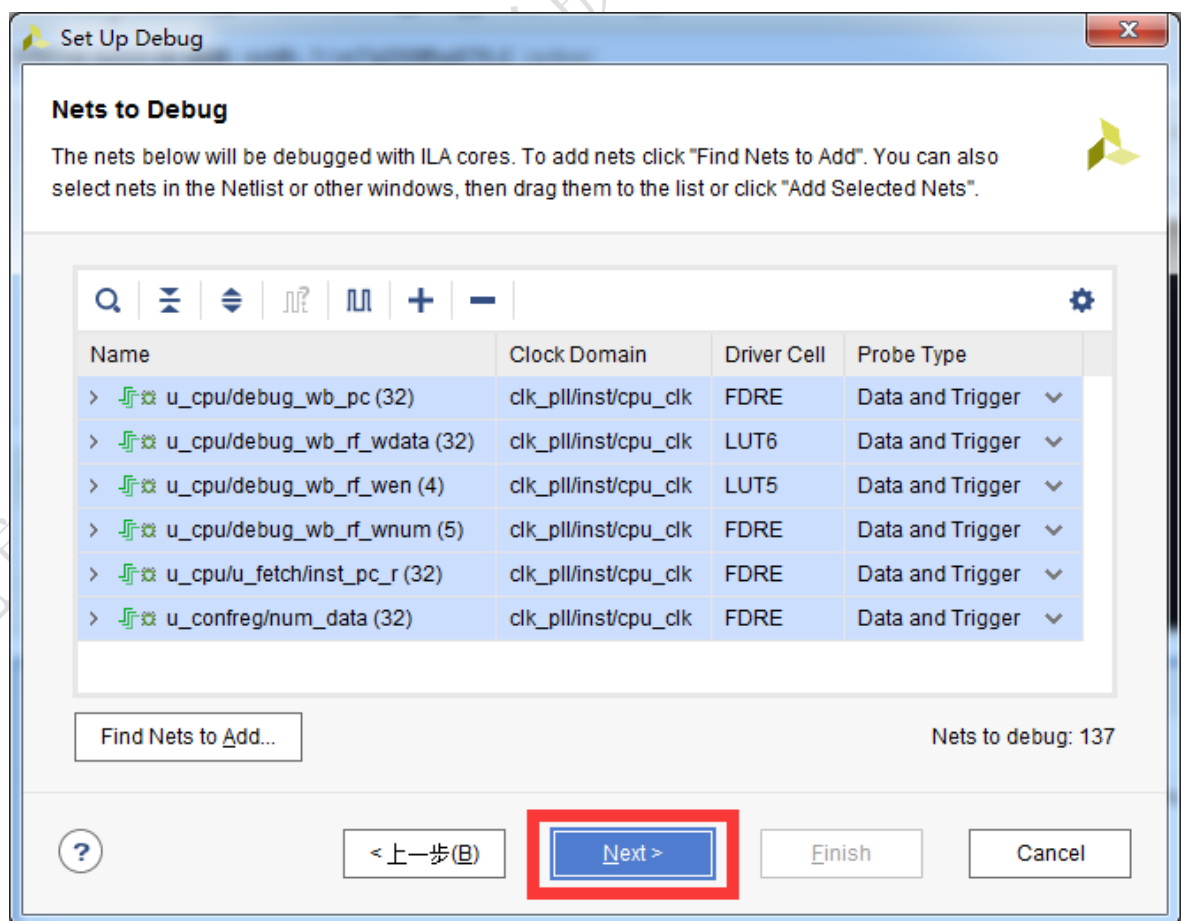
点击工程左侧 synthesis->Open Synthesized Desgin->Set Up Debug。



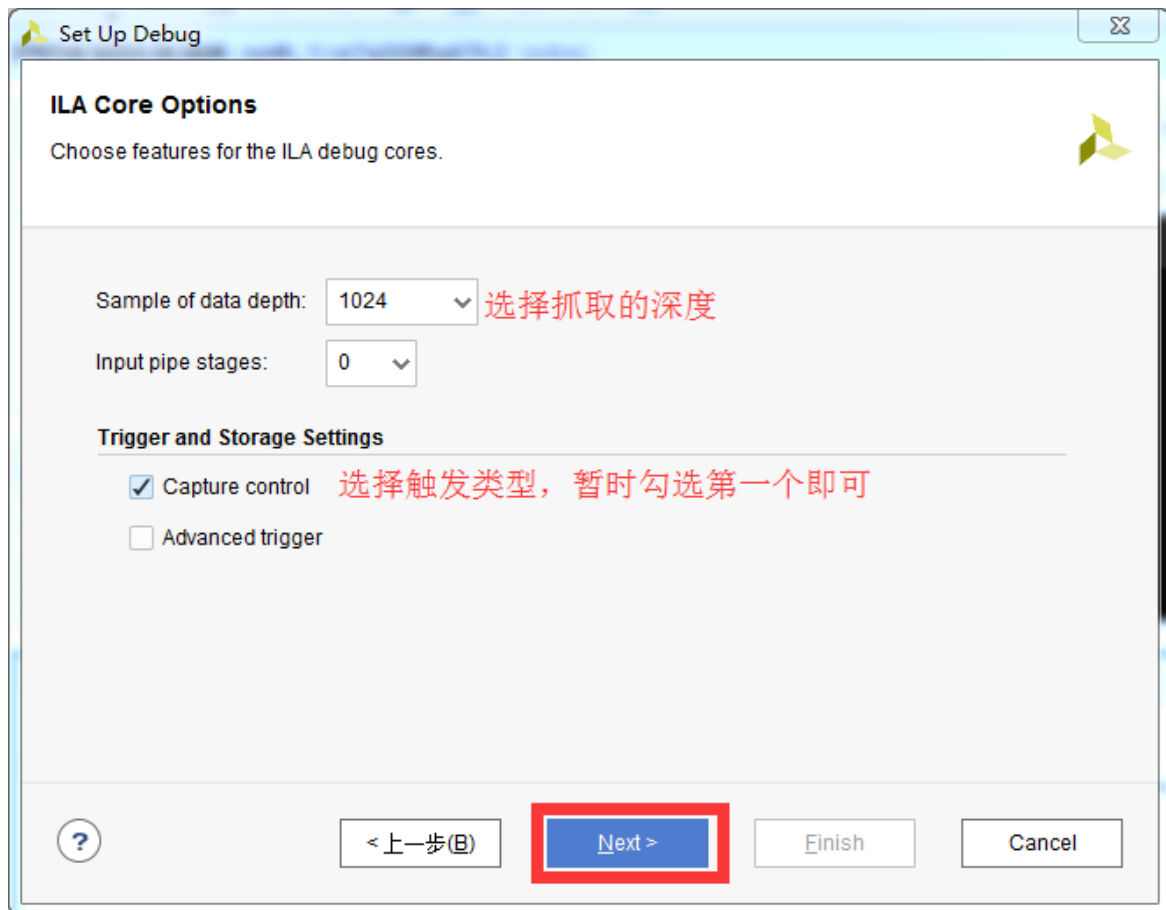
随后会出现如下界面，点击 Next:



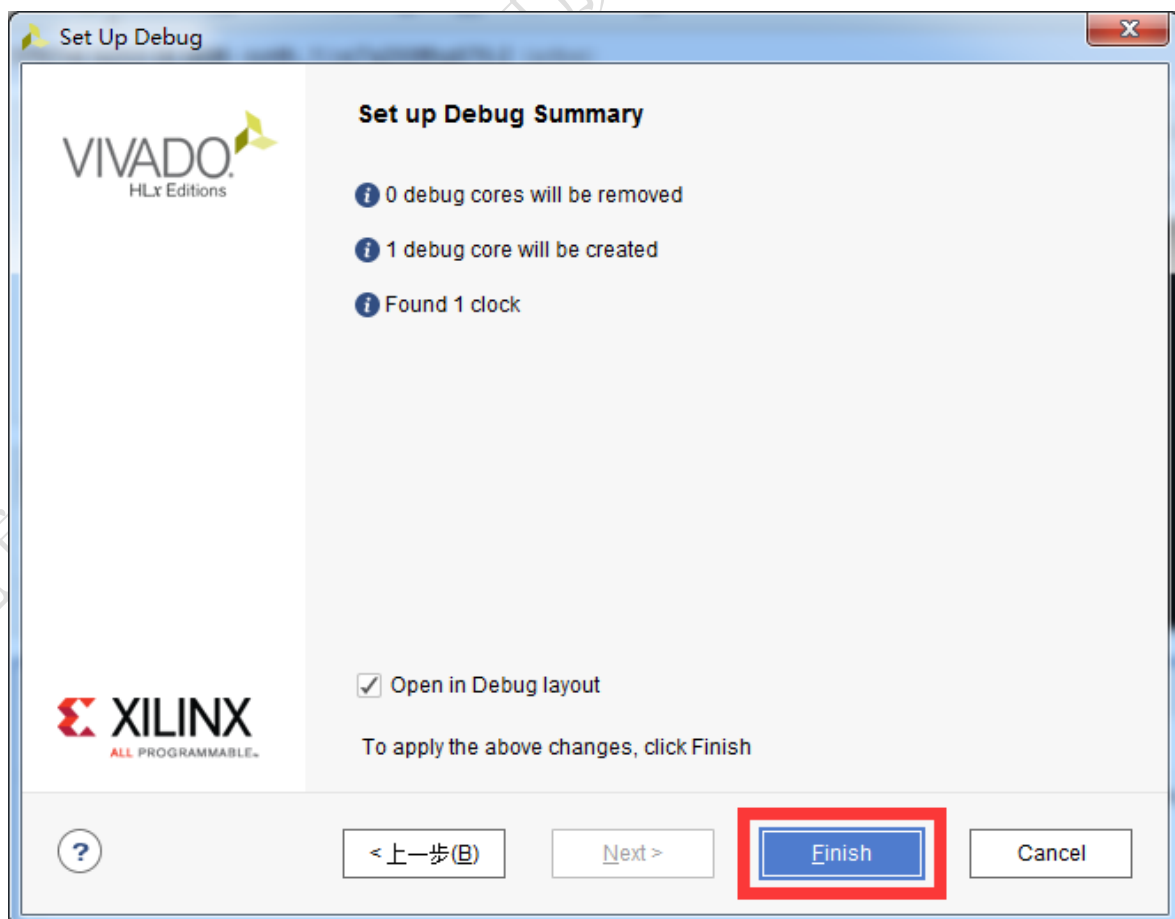
随后会列出抓取的 Debug 信息，点击 Next



选择抓取的深度和触发控制类型，点击 Next:

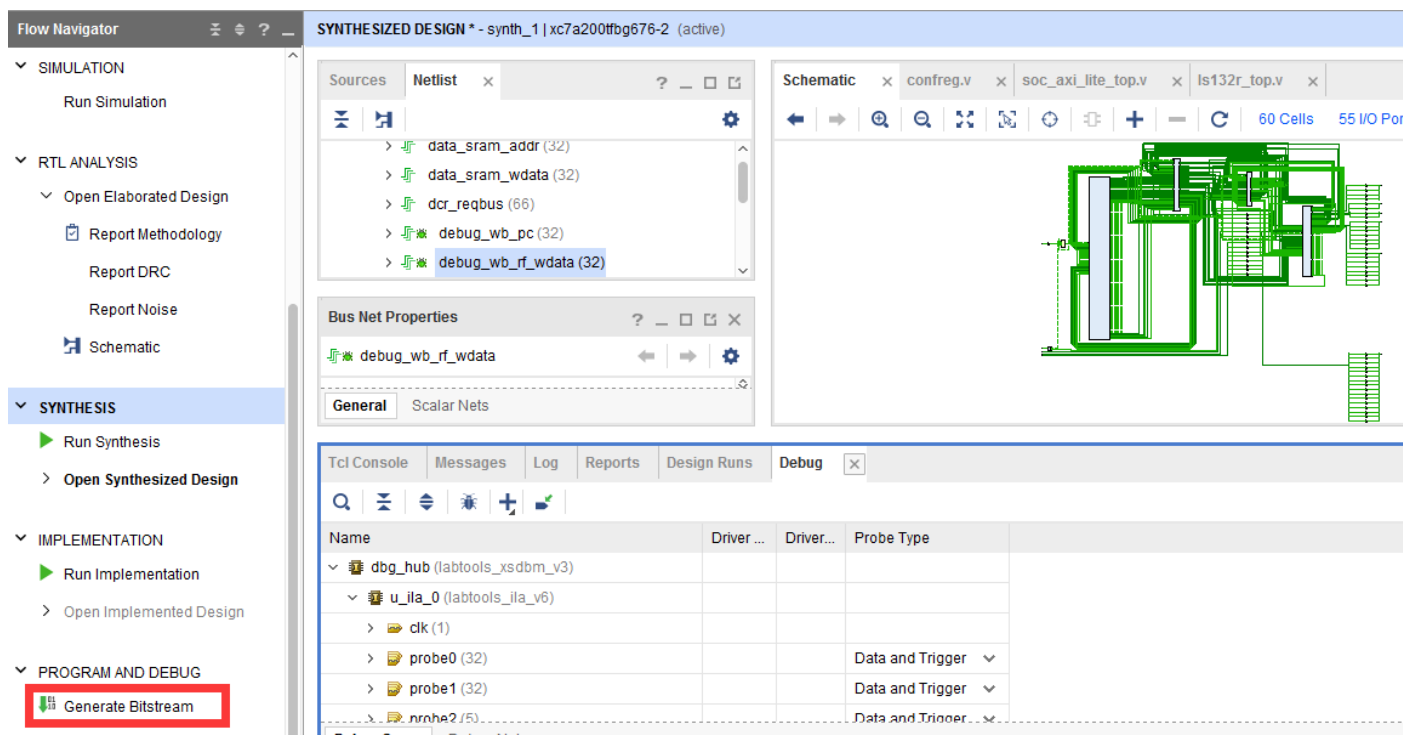


最后，点击 Finish:

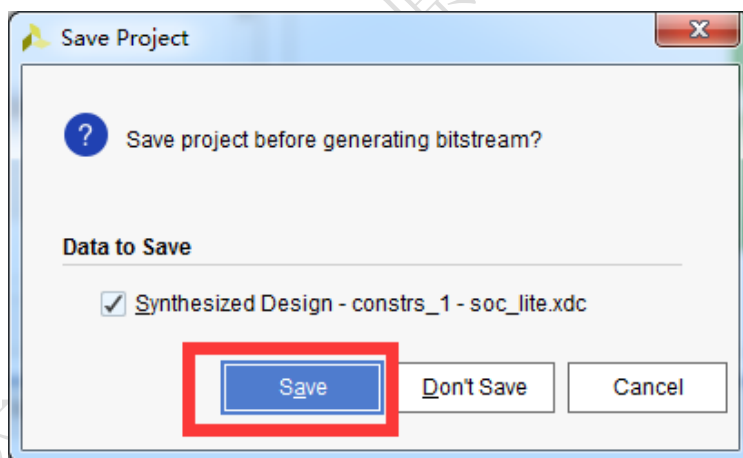


13.3 实现并生产 bit 流文件

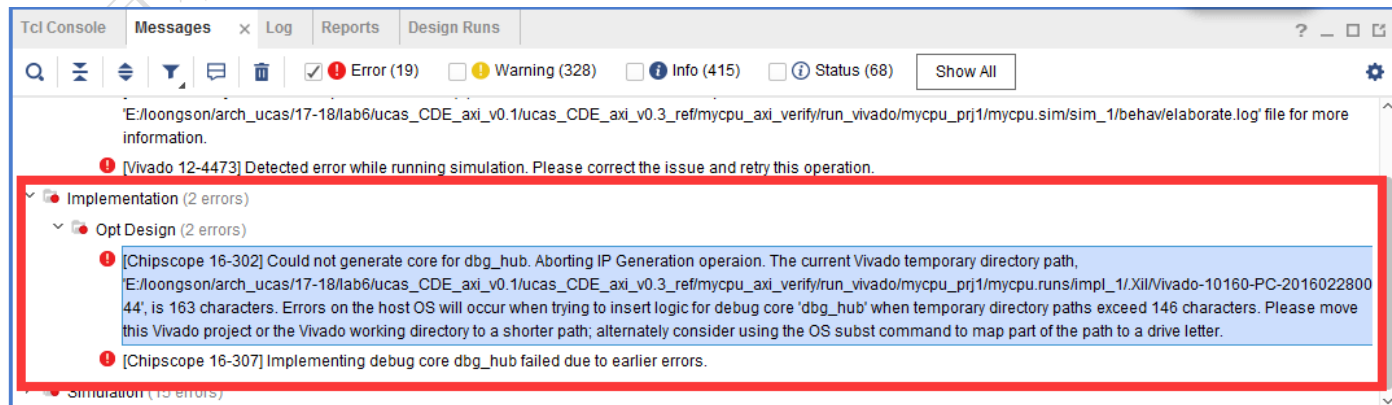
在 13.2 节完成后会出现类似下图界面，直接点击 Generate Bitstream:



弹出如下界面，点击 Save:

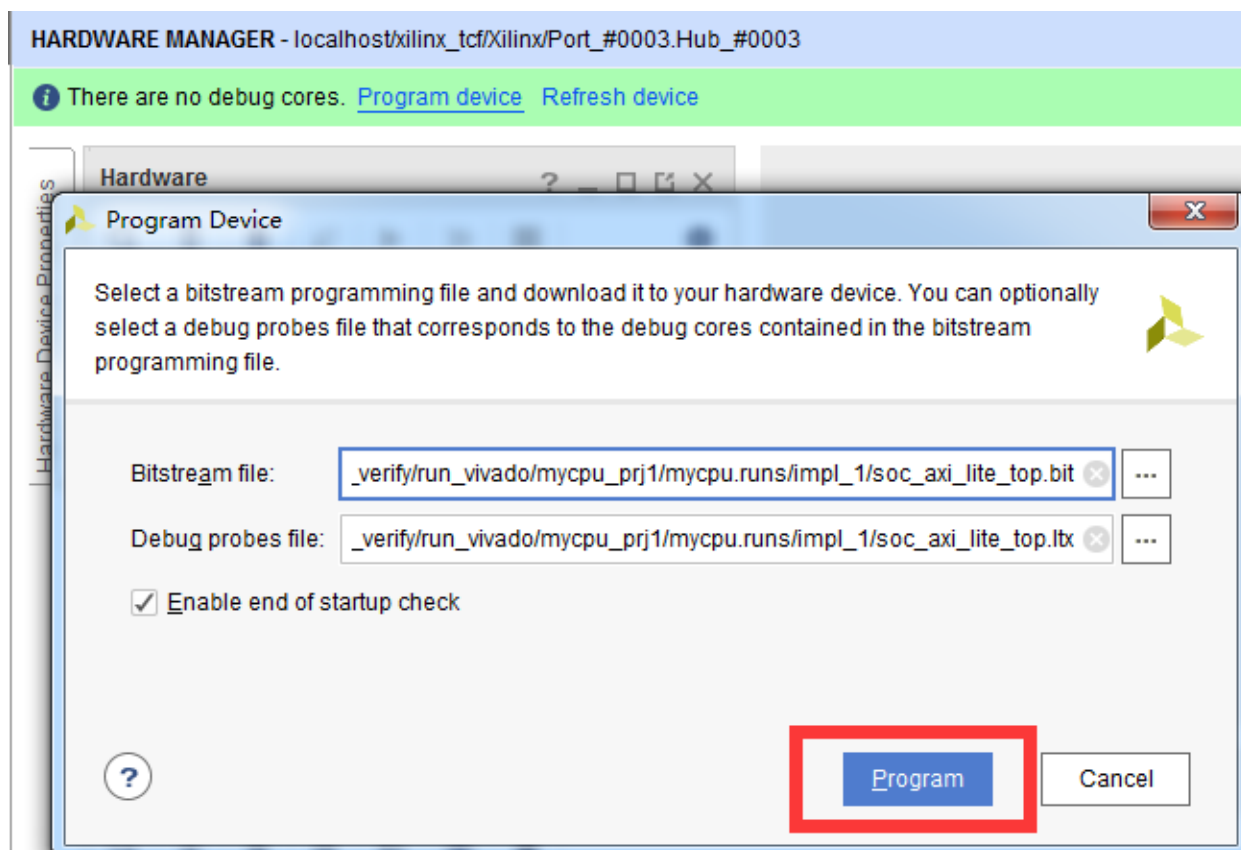


如果有后续弹出界面，继续点击 OK 或 Yes 即可。这时就进入后续生产 bit 文件的流程了，此时 Vivado 界面里的 synthesis design 界面就可以关闭了。如果发现以下错误，则是因为路径太深，引用起来名字太长，降低工程目录深度即可：



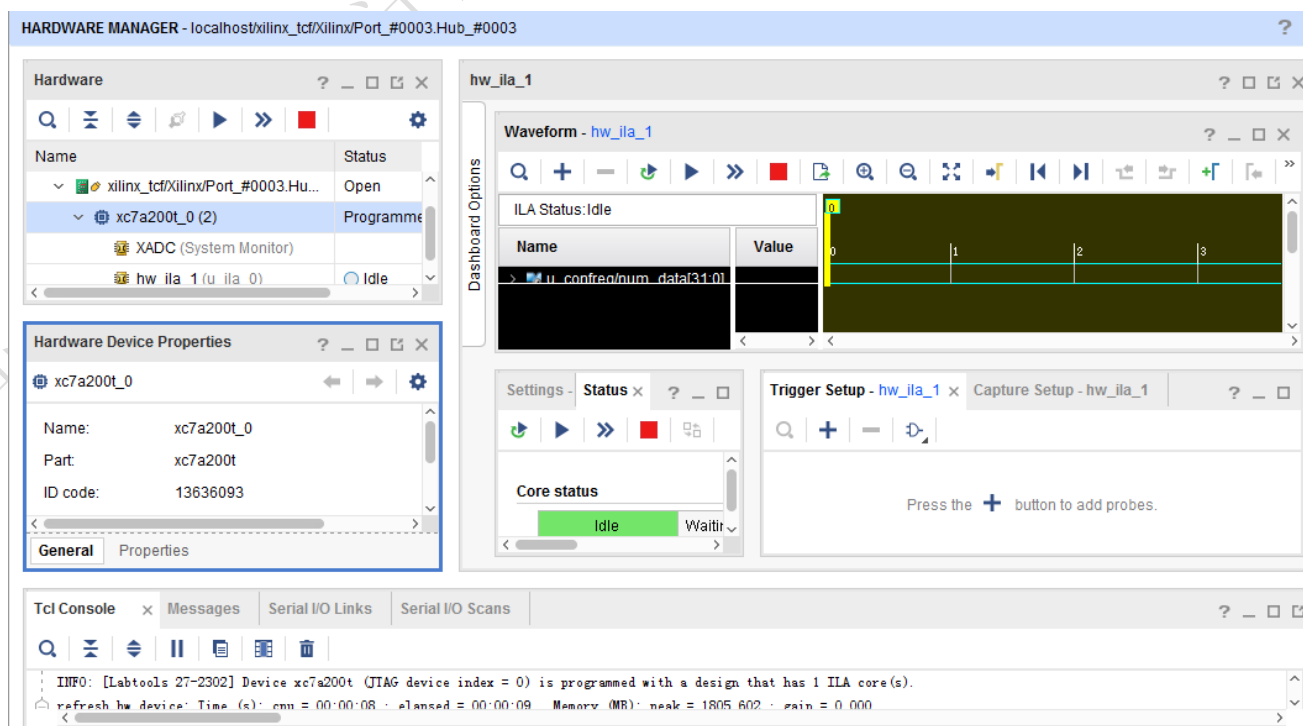
13.4 下载 bit 流和 debug 文件

在完成 13.3 节后，会生成 bit 流文件和调试使用 ltx 文件。这里，打开 Open Hardware Manager，连接好 FPGA 开发板后，选择 Program device，如下图。自动加载了 bit 流文件和调试的 ltx 文件。选择 Program，等待下载完成。

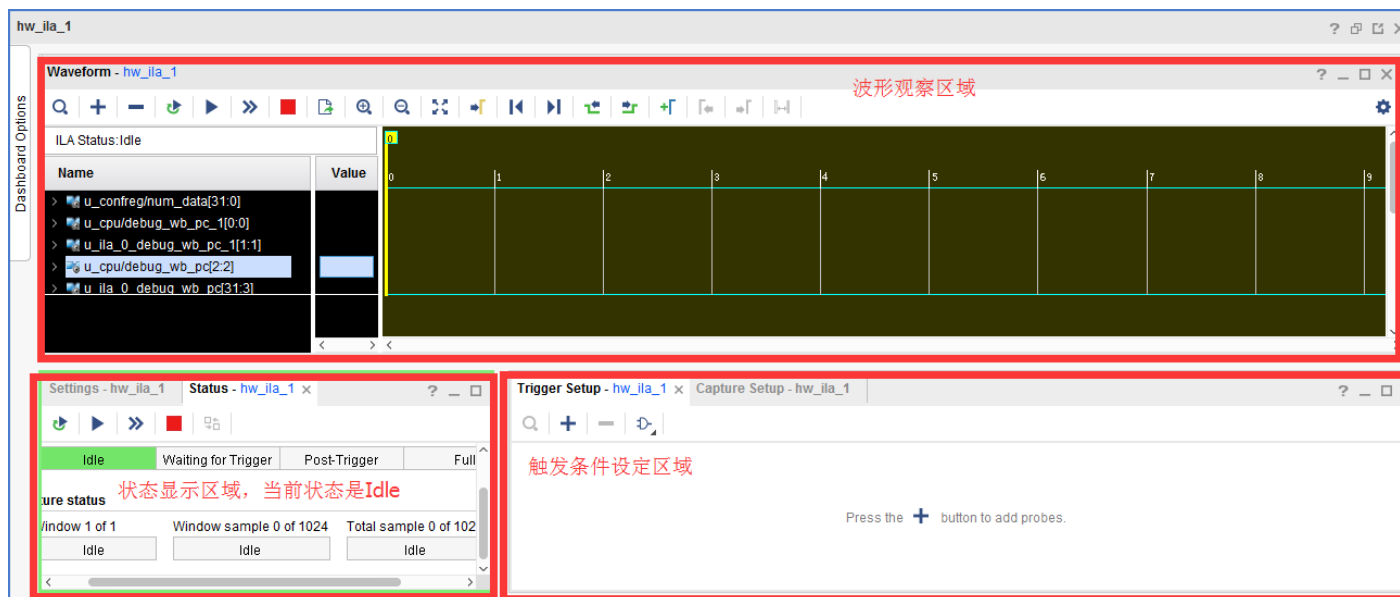


13.5 上板观察

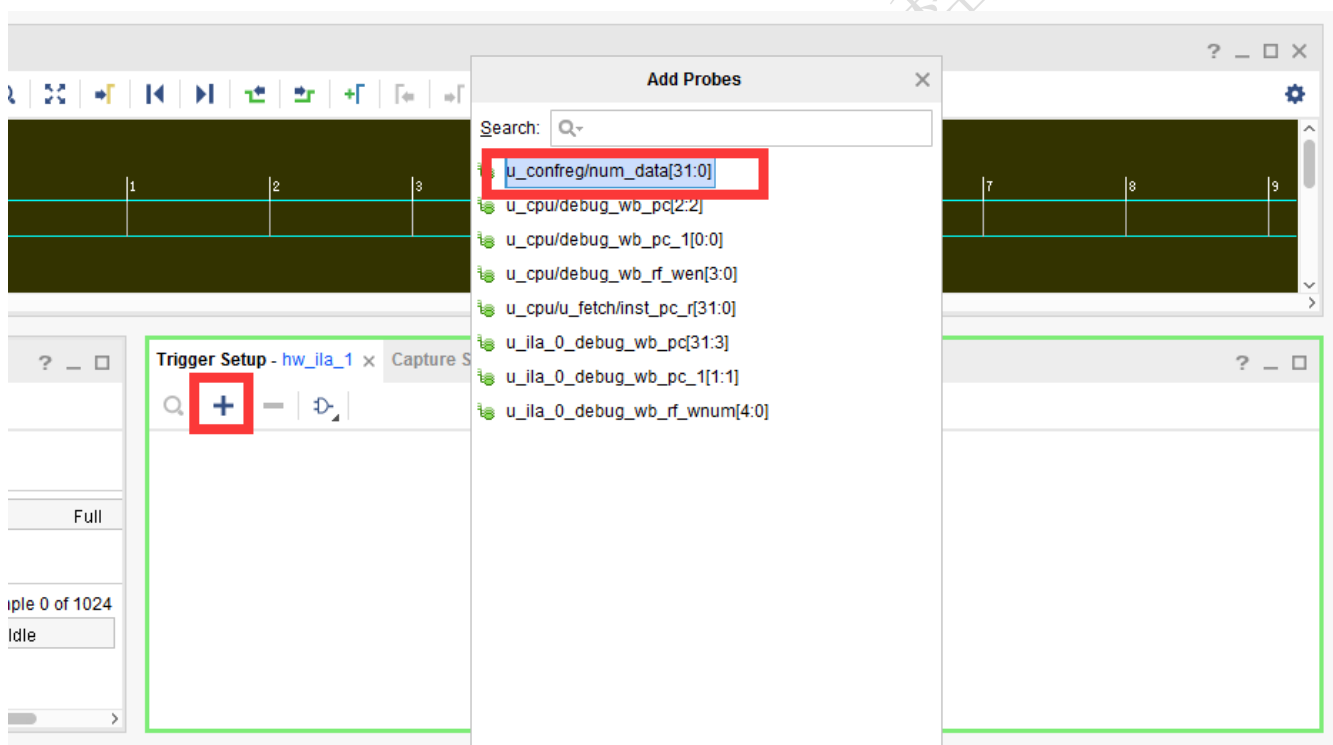
在下载完成后，vivado 界面如下，在线调试就是在 hw_ila_1 界面里进行的。



hw_ila_1 界面主要分为 3 个界面，分别如下：



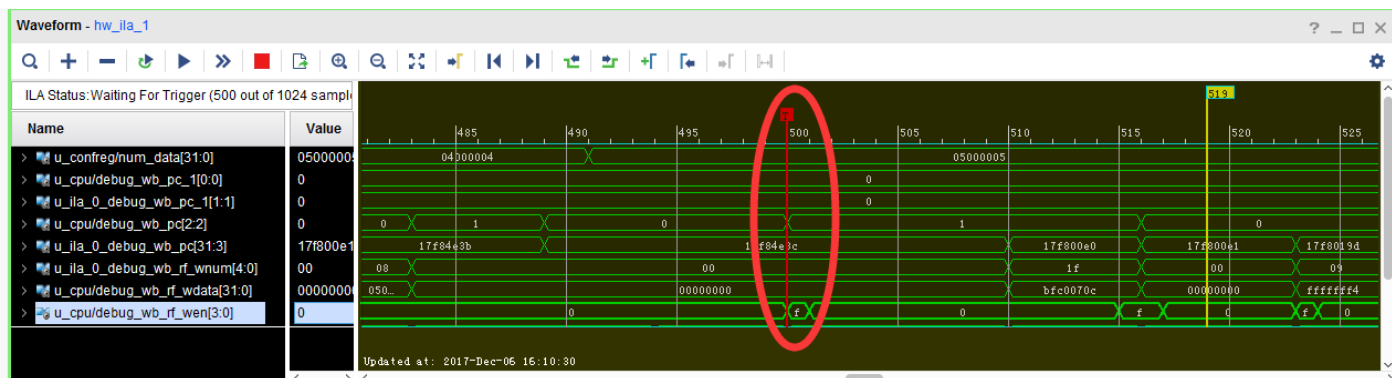
首先，我们需要在右下角区域设定触发条件。所谓触发条件，就是设定该条件满足时获取波形，比如我先设定触发条件是数码管寄存器到达 0x0500_0005。在下图中，先点击“+”，在双击 num_data。



之后会出现如下界面，设定好触发条件。



从上图也能看到, num_data 是 0x5c00_005c, 表示一次测试已经完成了。所以这时候, 点击第二个触发按键等待触发, 会发现波形窗口没有反应。这是因为触发条件没有被满足, 这时按下 FPGA 板上的复位键即可。结果如下图。红色圈出的就是触发条件: num_data==32'h5c00_005c && rf_wen==4'hf。



剩下的 debug 过程, 就和仿真 debug 类似了, 去观察波形。但在线调试时, 你无法添加在 13.1 节未被添加 debug mark 的信号。在线调试过程中, 可能需要不停的更换触发条件, 不停的按复位键。

13.6 注意事项

在 13.1 节中添加要抓取的信号时, 不要给太多信号标注 debug mark 了。在线调试时抓取波形是需要消耗电路资源和存储单元的, 因此能抓取的波形大小是受限的, 应当只给必要的 debug 信号添加 debug mark。

在 13.2 节中抓取波形的深度, 不宜太深。如果设定得太深, 那么会存在存储资源不够, 导致最后生成 bit 流和 debug 文件失败。

抓取的信号数量和抓取的深度是一对矛盾的变量。如果抓取深度相对较低, 抓取的信号数量就可以相对多些。

在 13.5 节中, 相对仿真调试, 在线调试对调试思想和技巧有更高的要求, 请好好整理思路, 多多总结技巧。特别强调以下几点:

- 触发条件的设定有很多组合, 请根据需求认真考虑, 好好设计。
- 通常只需要使用单触发模式, 但循环触发有时候也很有用, 必要时好好利用。
- 在线调试界面里很多按键, 请自行学习, 可以网上搜索资料, xilinx 官网上搜索, 查找官方文档等。

最后再提醒一点, 仿真通过但上板失败时, 请先重点排查其他问题, 最后再使用在线调试的方法。也就是仿真通过, 上板异常时, 应按以下流程排查:

- 1) 复核生成、下载的 bit 文件是否正确。
- 2) 复核仿真结果是否正确。
- 3) 检查实现时的时序报告 (Vivado 界面左侧 “IMPLEMENTATION” “Open Implemented Design” “Report Timing Summary”)。
- 4) 认真排查综合和实现时的 Warning
- 5) 排查 RTL 代码规范, 避免多驱动、阻塞赋值乱用。
- 6) 使用 Vivado 的逻辑分析仪进行板上在线调试

北京科技大学计算机组成原理课程设计19-20秋季